

server.mjs Textbook Guide

A JavaScript-first, feature-based explanation of the OMB Portfolio Builder local backend.

This PDF is written for a reader who is new to JavaScript but wants to understand how the actual backend works. It begins with a long JavaScript primer, then returns to `server.mjs` and explains the file by features: how requests are routed, how drafts are saved, how files are uploaded, how code is compiled, how GitHub publishing is authenticated, how AI requests are handled, and how updates are launched.

The explanations avoid line-number references because the code will change. Instead, each chapter names the behavior being built and embeds the relevant variables and functions inside the surrounding explanation. Source excerpts are included only where they help the reader connect the prose to the real implementation.

How To Read This PDF

The first half is a JavaScript introduction. It is intentionally long because a backend file is easier to understand when the language stops feeling mysterious. Read it like a textbook chapter: each subsection introduces one language idea, explains the intuition, shows a small example, and then connects that idea to the backend.

The second half is organized by features rather than by raw source order. If you want to understand what happens when the builder saves a draft, go to the Save Draft feature. If you want to understand the Compile Code workspace, go to the Compile Code feature. This mirrors how a real user experiences the app.

The code excerpts are not reference labels. They are placed in the chapter where the code earns its meaning. Read the prose first, then read the source excerpt, then return to the following paragraphs to understand how the important variables, helper calls, return objects, and side effects fit together.

Chapter 1: JavaScript Syntax For A New Reader

This chapter teaches the JavaScript syntax needed to read `server.mjs`. It does not assume that the reader already thinks in JavaScript. The goal is to turn symbols into ideas: braces become boundaries, functions become named procedures, promises become delayed work, objects become labeled bundles, and `async` code becomes a way to keep the app responsive while files, commands, Git, and AI calls take time.

The examples are deliberately small. They are not toy examples in the insulting sense; they are reduced circuits. A simple circuit teaches one idea more cleanly than a crowded board. After each idea is clear, the later chapters reconnect it to the larger backend.

1.1 What JavaScript is doing in this project

JavaScript is the language used on both sides of this portfolio system: browser code makes the builder interactive, while Node.js code gives the builder controlled access to the local computer.

A useful analogy is a hardware bench with a front panel and a back panel. The front panel has knobs and displays; the back panel has power, probes, and dangerous connectors. Browser JavaScript is the front panel. Node.js is the controlled back panel.

The same grammar appears in both places, but the available APIs change. Browser code can touch the DOM. Node code can touch files, ports, child processes, and environment variables.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const runsInBrowser = typeof window !== 'undefined';
const runsInNode = typeof process !== 'undefined';
console.log({ runsInBrowser, runsInNode });
```

The server file is Node JavaScript, so later chapters focus on files, HTTP requests, compiler commands, Git commands, and update launchers.

Beginners often treat JavaScript as one environment. The safer mental habit is to ask where the code runs before asking what it can do.

1.2 Statements and expressions

A statement is an instruction that asks JavaScript to do something. An expression is a piece of code that produces a value.

In a circuit drawing, an expression is like a measured voltage at a node; a statement is like the instruction to connect the probe, record the value, or switch the supply on.

Assignments, imports, function declarations, `if` branches, loops, and `return` lines are statements. Arithmetic, string creation, function calls, object literals, and comparisons are expressions.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const folder = 'docs';
const file = 'projects.json';
const fullName = `${folder}/${file}`; // expression assigned by a statement
if (fullName.endsWith('.json')) {
  console.log('This is catalog data.');
```

`server.mjs` uses expressions to calculate paths and status values, then wraps those expressions inside statements that send responses or run commands.

A common mistake is trying to put a statement where JavaScript expects a value. When code looks confusing, underline the pieces that produce values and separate them from the instructions that move the program forward.

1.3 Values and literals

Values are the actual pieces of information JavaScript moves around. A literal is a value written directly into the code.

Think of literals as parts pulled straight from a component drawer: a resistor value, a jumper setting, or a label printed on a schematic. They are not calculated yet; they are placed into the design.

Strings use quotes, numbers use digits, booleans use true or false, arrays use square brackets, objects use braces, null means deliberately empty, and undefined means no value has been supplied.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const route = '/api/catalog':
const statusCode = 200:
const ok = true:
const publishPaths = ['index.html', 'projects.json', 'assets']:
const response = { ok, route, publishPaths };
```

The server file uses literal strings for route names like /api/catalog, object literals for JSON responses, and arrays for lists such as allowed publish paths.

The trap is to confuse an empty string, null, undefined, false, and zero. They all feel like absence, but they are different signals to the program.

1.4 const and let

const and let create named storage locations. const means the name cannot be reassigned. let means the name may point to a different value later.

A const is like a labeled test point whose label should not be moved during the test. A let is like a measurement variable on your notebook page that can be updated as you step through the experiment.

Use const for values that should not be rebound. Use let when a loop, parser, search, or accumulator genuinely needs to change the binding.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const settings = { retries: 2 }:
settings.retries = 3: // allowed because the object is still the same object
let currentStep = 'download':
currentStep = 'install': // allowed because let can be reassigned
```

server.mjs uses const for stable configuration such as root folders and MIME types, then uses let inside parsers and filesystem walkers when the working value changes.

Do not read const as meaning the object can never change. It only prevents reassigning the variable name. Object properties and array contents can still be mutated unless you freeze or copy them.

1.5 Scope and where names live

Scope describes where a variable name can be seen. JavaScript creates scopes for modules, functions, and blocks such as if statements and loops.

In a lab, scope is like cabinet access. A tool inside one locked drawer is not automatically available at every bench. The technician must either keep it in shared storage or pass it to the person who needs it.

A top-level const in a module can be used by functions in that module. A const inside a function exists only during that function call. A const inside a block exists only inside that block.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const sharedPort = 8080;
function describeServer() {
  const protocol = 'http';
  if (sharedPort === 8080) {
    const label = 'local preview';
    console.log(`${protocol} ${label}`);
  }
}
```

server.mjs keeps shared state such as compile profiles at module scope, while temporary values such as a parsed request body or a compiler result live inside the function handling that request.

The most common beginner bug is reading a variable outside the scope where it was created. The second is accidentally reusing a name in an inner scope and hiding the outer value.

1.6 Strings and template literals

Strings hold text. Template literals use backticks and allow embedded expressions with \${...}.

Template literals are like labels with live fields. Instead of printing a fixed sticker, the program prints a sticker whose project name, date, or status can be inserted at the moment it is made.

Single and double quotes are good for plain text. Backticks are useful when a string includes variables, line breaks, or generated messages.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const projectName = 'Laser Link';
const today = new Date().toISOString().slice(0, 10);
const message = `Update ${projectName} on ${today}`;
console.log(message);
```

server.mjs uses template literals for status messages, commit messages, URLs, shell output summaries, and human-readable errors returned to the builder.

A pasted URL or formula can be broken if code treats it as a math expression instead of text. When the purpose is text, keep it as a string and escape or validate it at the boundary.

1.7 Numbers, NaN, and numeric conversion

JavaScript has one primary Number type for most numeric work. It can represent integers, decimals, Infinity, and NaN, which means not-a-number.

NaN is like a meter reading that says the probe connection did not produce a valid measurement. The reading exists, but it cannot be trusted as a number.

Number(value) converts many text values into numbers. Number.isFinite(value) checks whether a value is a usable finite number.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const port = Number(process.env.PORT || 8080);
const timeoutMs = Number('15000');
if (Number.isFinite(port) && Number.isFinite(timeoutMs)) {
  console.log(`Listening on ${port} with timeout ${timeoutMs}ms`);
}
```

The server uses numeric conversion for ports, timeouts, elapsed times, token limits, and cached authorization windows.

Do not compare NaN with NaN. Use `Number.isNaN` or `Number.isFinite`. Also remember that values from forms, JSON, and environment variables often begin as strings.

1.8 Booleans and comparisons

A boolean is true or false. Comparisons create booleans that guide branches and loops.

A boolean is the logic-level signal of JavaScript: high or low, yes or no, allowed or blocked.

Use `===` and `!==` for strict comparison. Use `>`, `>=`, `<`, and `<=` for ordering. Combine decisions with `&&` for and, `||` for or, and `!` for not.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const isLocal = true;
const hasWriteAccess = false;
if (isLocal && hasWriteAccess) {
  console.log('Publishing can continue.'):
} else {
  console.log('Publishing stays blocked.'):
}
```

`server.mjs` uses booleans for local-request checks, success flags, update availability, cache validity, and endpoint routing.

Avoid relying on loose equality unless you have a deliberate reason. Strict equality keeps strings, numbers, booleans, null, and undefined from quietly pretending to be each other.

1.9 Objects as named bundles

An object groups related values under property names. It is one of the central JavaScript structures.

An object is like a project folder with labeled tabs: title, owner, files, status, and notes all belong together, but each tab has its own name.

Object literals use braces. Properties can be read with dot syntax when the name is simple, or bracket syntax when the property name is dynamic.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const profile = {
  name: 'Maurice Otieno',
  role: 'Hardware engineer',
  links: ['GitHub', 'LinkedIn']
};
console.log(profile.name);
console.log(profile['role']);
```

`server.mjs` sends object literals as JSON responses, stores language profiles as nested objects, and reads request bodies as objects.

The danger is assuming a property exists. When data comes from a file, browser, or network, check it before trusting it.

1.10 Arrays as ordered lists

An array stores values in order. Each item has an index beginning at zero.

An array is like a row of labeled storage bins. The first bin is position zero, not position one, which feels strange until it becomes habit.

Use array literals with square brackets. Read length for the number of items. Use push to add, map to transform, filter to keep some, and find to locate one.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const publishPaths = ['index.html', 'styles.css', 'projects.json'];
const jsonPath = publishPaths.find((item) => item.endsWith('.json'));
console.log(jsonPath);
```

server.mjs uses arrays for publish paths, compiler candidates, endpoint instructions, command arguments, and collected output items.

Arrays are ordered, so changing the order can change behavior. Command arguments and publishing allowlists should be especially deliberate.

1.11 Destructuring

Destructuring pulls values out of arrays or objects into named variables.

It is like opening a connector harness and giving the important wires local labels before working on the circuit.

Object destructuring uses property names. Array destructuring uses positions. Defaults can be supplied when a field is missing.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const target = { repository: 'site.git', branch: 'main' };
const { repository, branch } = target;
console.log(repository, branch);
```

server.mjs uses destructuring to read options, command results, and selected values from complex objects without writing repetitive property access.

Destructuring can hide where data came from if overused. For beginner readability, destructure the fields that matter and keep the surrounding object name when context helps.

1.12 Optional chaining and nullish coalescing

Optional chaining reads a nested property without crashing when an earlier value is null or undefined. Nullish coalescing supplies a fallback only for null or undefined.

Optional chaining is a cautious probe. It checks whether each connector exists before trying to measure deeper inside the harness.

Use value?.property when the parent might be absent. Use value ?? fallback when empty string or zero should still count as real values.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const result = { error: null, stdout: '' };
```

```
const message = result.error?.message ?? 'No error message was provided':
const output = result.stdout ?? 'missing':
console.log(message, output);
```

server.mjs uses cautious access when reading API errors, optional environment settings, package metadata, and result fields from child processes.

Do not use || as a fallback when zero, false, or an empty string are legitimate inputs. Use ?? when absence is the only reason to fall back.

1.13 Function declarations

A function declaration gives a named operation a reusable body. It can receive parameters and return a value.

A function is like a named test procedure. Instead of rewriting the same setup steps each time, you name the procedure and call it whenever that kind of work is needed.

The function keyword is followed by the name, parameters in parentheses, and a body in braces. return sends a value back to the caller.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
function makeStatus(ok, message) {
  return { ok, message };
}
const status = makeStatus(true, 'Saved draft');
console.log(status);
```

server.mjs uses named functions for reusable backend operations such as sending JSON, resolving paths, running commands, and handling AI requests.

If a block of logic has a clear job and more than one caller, a function makes the file easier to reason about. If it has a dangerous side effect, a named function makes that side effect easier to audit.

1.14 Parameters and arguments

Parameters are the names a function uses internally. Arguments are the actual values passed when the function is called.

A parameter is the empty socket on a test fixture; an argument is the actual device plugged into that socket for one run.

Parameters can have defaults. They can also be objects when the function needs named inputs instead of a long ordered list.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
function runCheck({ language = 'javascript', source = '' } = {}) {
  return `Checking ${language}: ${source.length} characters`;
}
console.log(runCheck({ language: 'c', source: 'int main(){}' }));
```

server.mjs often passes objects into functions because publishing, AI, and compile actions have many named fields. That reduces mistakes compared with remembering which positional argument comes third or fourth.

When a function receives outside data, validate the argument before trusting it. A parameter name does not prove the incoming value is safe.

1.15 Return values

A return statement ends a function and sends a value back to the caller.

Return is the lab report leaving the test bench. The function may have performed work internally, but the caller needs a clear result it can use.

A function can return primitives, arrays, objects, promises, or nothing. Returning an object is common when multiple facts need to travel together.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
function compileSummarv(code, stderr = '') {  
  return {  
    ok: !stderr,  
    characters: code.length,  
    error: stderr || null  
  };  
}
```

server.mjs returns structured result objects from compile, publish, update, and security helpers so the UI can display success, errors, command output, and file paths.

A function that sometimes returns one shape and sometimes another becomes hard to use. Prefer consistent objects with fields such as ok, error, stdout, stderr, and data.

1.16 Arrow functions

Arrow functions are a compact way to write function values. They are common in callbacks and array methods.

An arrow function is like a short inline test note: it is still a procedure, but it lives right where the surrounding operation needs it.

For a single expression, an arrow can return implicitly. For multiple statements, use braces and an explicit return if a value should come back.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const files = ['main.c', 'design.sv', 'notes.txt'];  
const sourceFiles = files.filter((file) => file.endsWith('.c') || file.endsWith('.sv'));  
console.log(sourceFiles);
```

server.mjs uses arrows in array operations, timers, event callbacks, and small helper callbacks passed into promises or command handlers.

Do not use arrows only because they look modern. Use them when they make the surrounding expression clearer. Named functions are still better for large operations.

1.17 Callbacks

A callback is a function passed into another function so it can be called later or for each item.

A callback is like handing a technician an instruction card: when the scan reaches each component, perform the instruction written on the card.

Array methods, timers, event systems, and stream APIs all rely on callbacks. The callback receives values chosen by the function calling it.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const commands = ['git status', 'git push', 'git log'];  
commands.forEach((command, index) => {
```

```
console.log(`${index}: ${command}`);
});
```

server.mjs uses callbacks in server startup, timers, array processing, and event handling around spawned processes.

The caller controls when and how often the callback runs. Do not assume a callback runs immediately unless the API promises that behavior.

1.18 if, else, and guarded paths

if statements choose which branch of code runs based on a boolean condition.

An if statement is a relay or comparator: when the condition is high, one path conducts; otherwise another path may be selected.

Use if for important decisions. Use early returns when a function should stop immediately after a failed requirement.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
function requireLocalRequest(isLocal) {
  if (!isLocal) {
    return { ok: false, error: 'Write actions are only allowed locally.' };
  }
  return { ok: true };
}
```

server.mjs uses guard clauses to reject non-local write requests, missing upload data, invalid catalogs, and publishing attempts without authorization.

Deep nesting makes server code hard to trust. A clean guard at the top makes the safe path easier to see.

1.19 switch statements and route decisions

A switch chooses behavior by comparing one value against multiple cases.

Switch is like a rotary selector on a bench instrument. One position selects voltage, another selects current, and another selects resistance.

Many JavaScript developers use if chains for routes because route checks often combine method and path. switch is still useful when one value controls many clear branches.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const action = 'save-draft';
switch (action) {
  case 'save-draft':
    console.log('Write local catalog');
    break;
  case 'approve-site':
    console.log('Authorize and publish');
    break;
}
```

server.mjs uses a large API switchboard pattern in handleApi, mostly with ordered if statements because each route combines HTTP method, pathname, safety, body parsing, and error handling.

The important idea is not whether if or switch is used. The important idea is that route selection should be centralized enough to audit what the backend exposes.

1.20 Loops

Loops repeat code. JavaScript has for, for...of, while, and several array methods that behave like loops.

A loop is like stepping through every net on a schematic or every file in a folder and applying the same check.

Use for...of when you need each item. Use a traditional for loop when the index matters. Use while when the stopping condition is discovered during the work.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const candidates = ['git', 'C:/Program Files/Git/cmd/git.exe'];
for (const candidate of candidates) {
  if (candidate.includes('Git')) {
    console.log(`Trv ${candidate}`);
  }
}
```

server.mjs uses loops to walk directories, scan command candidates, gather process output, parse waveform text, and copy publish paths.

Every loop needs a clear stopping condition. Infinite loops in backend code can freeze a request, hang an update, or make a compile operation appear stuck.

1.21 Array map, filter, find, and reduce

Array methods express common list operations without writing every loop by hand.

They are like common lab workflows: filter removes irrelevant parts, map transforms every measurement, find locates the first matching component, and reduce combines many readings into one result.

map returns a new array of transformed items. filter returns a smaller array. find returns one item or undefined. reduce accumulates a final value.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const files = ['main.c', 'notes.txt', 'design.sv'];
const extensions = files.map((file) => file.split('.').pop());
const sourceFiles = files.filter((file) => ['c', 'sv'].includes(file.split('.').pop()));
const firstHdl = files.find((file) => file.endsWith('.sv'));
console.log({ extensions, sourceFiles, firstHdl });
```

server.mjs uses these patterns when preparing command output, selecting files, filtering source excerpts, and converting arrays of lines into readable terminal text.

Do not use reduce just to look clever. If a plain loop is clearer for a beginner or for side effects, use the clearer shape.

1.22 try, catch, and controlled failure

try and catch let code handle errors instead of crashing the whole operation.

A catch block is like a protective fuse. It does not make the fault disappear, but it keeps the failure contained and reports it in a controlled way.

Put the risky operation inside try. Put the recovery or error response inside catch. Use finally when cleanup must happen whether the operation succeeds or fails.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
try {
```

```
const parsed = JSON.parse('{ "ok": true }');
console.log(parsed.ok);
} catch (error) {
  console.log(`JSON could not be parsed: ${error.message}`);
}
```

server.mjs wraps file reads, compiler runs, update launches, AI calls, and Git actions so the UI gets a useful JSON error instead of a broken server.

Do not catch errors and then hide them. A good catch block translates the problem for the caller while preserving enough detail for debugging.

1.23 throw and Error objects

throw stops the current path and sends an error upward to the nearest catch.

Throwing is like stopping a test procedure because the setup is unsafe. Continuing would create a misleading result.

Throw Error objects when you want message, stack, and optional custom fields. Add details when the UI needs more context.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
function requireProjectId(projectId) {
  if (!projectId) {
    throw new Error('Project id is required.');
```

server.mjs throws when a target path escapes the allowed workspace, when a command fails in a way the caller must see, and when publishing authorization is missing.

The beginner mistake is using throw for normal choices. Reserve it for failures that should skip the normal return path.

1.24 Modules and import

A module is a file with its own scope that can import values from other modules and export values to other files.

Modules are like separate boards in a larger system. Each board has private traces and public connectors.

Node ES modules use import statements. Built-in Node modules often use names beginning with node:, such as node:path or node:fs/promises.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
import path from 'node:path';
import { readFile, writeFile } from 'node:fs/promises';

const filePath = path.join('docs', 'projects.json');
console.log(filePath);
```

server.mjs imports HTTP, filesystem, path, process, crypto, and utility modules at the top because those APIs give the backend its local powers.

Do not confuse CommonJS require syntax with ES module import syntax. This project uses ES modules, so import appears at the top of server.mjs.

1.25 JSON

JSON is a text format for structured data. JavaScript can turn objects into JSON text and parse JSON text back into objects.

JSON is the shipping manifest between the frontend and backend. It lets the UI say exactly what it wants without sending raw JavaScript code.

JSON.stringify converts data to text. JSON.parse converts text to data. Valid JSON uses double quotes around property names and strings.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const catalog = { categories: [], projects: [] };
const text = JSON.stringify(catalog, null, 2);
const parsed = JSON.parse(text);
console.log(parsed.projects.length);
```

server.mjs uses JSON for catalogs, draft saves, API request bodies, AI context, update responses, and publishing reports.

Never assume JSON from a browser is valid or safe. Parse it carefully, check its shape, and reject missing required fields before writing files or running commands.

1.26 Promises

A Promise represents work that may finish later. It can resolve with a value or reject with an error.

A promise is like sending a job to a lab instrument and receiving a ticket. The result is not available immediately, but the ticket lets the program continue once the instrument finishes.

Promise-based APIs avoid blocking the entire program while waiting for files, network calls, timers, or child processes.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const delayed = new Promise((resolve) => {
  setTimeout(() => resolve('finished'), 100);
});
delayed.then((message) => console.log(message));
```

server.mjs depends on promise-based filesystem APIs, fetch calls, command execution wrappers, and async helpers that return promise results.

A promise that is not awaited or returned can fail in the background. In backend code, that can make a request answer before the work is finished.

1.27 async and await

async functions return promises. await pauses inside an async function until a promise settles.

await is like waiting for the instrument ticket to be called before writing the final lab result.

Use await when later code depends on the result. Wrap awaited risky work in try/catch when the caller needs a clean error.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
async function loadCatalog(readFile) {
  const text = await readFile('projects.json', 'utf8');
  return JSON.parse(text);
}
```

server.mjs uses `async/await` heavily because nearly every useful backend action involves files, Git, compilers, network requests, or update launchers.

`await` only works inside `async` functions or top-level modules that allow it. `server.mjs` is an ES module, so it can use top-level `await` for package metadata.

1.28 HTTP requests and responses

HTTP is the request-response protocol the builder uses to talk to the local backend.

The browser asks the server a question, like a student handing a form to a lab technician. The server reads the form, performs the safe work, and returns a result.

A request has a method, URL, headers, and sometimes a body. A response has a status code, headers, and body content.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
fetch('/api/catalog')
  .then((response) => response.json())
  .then((catalog) => console.log(catalog));
```

`server.mjs` creates an HTTP server, checks URL paths, routes `/api` actions, and serves files such as `index.html` and `script.js`.

The method matters. A GET should read. A POST can change state. The server enforces local-only checks before write-style POST routes.

1.29 URLs and paths

A URL identifies a web resource. A filesystem path identifies a location on disk. They look similar but are not the same thing.

A URL is the address on the visitor side of the counter. A path is the cabinet location behind the counter.

Use URL for request routing and path for local files. Decode and normalize carefully when converting a URL path into a filesystem path.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const url = new URL('/api/catalog', 'http://localhost:8080');
const route = url.pathname;
console.log(route);
```

`server.mjs` converts request URLs into route decisions, but it uses path helpers to keep file writes inside approved local roots.

The security mistake is joining untrusted URL text directly to a local folder and assuming it stayed inside the folder.

1.30 Node path helpers

The Node `path` module builds, normalizes, and inspects filesystem paths in an operating-system-aware way.

It is the layout tool that keeps folder names, separators, and extensions from being hand-soldered as fragile strings.

`path.join` combines segments. `path.resolve` creates absolute paths. `path.extname` reads the extension. `path.dirname` reads the parent folder.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
import path from 'node:path';
const folder = path.join('docs', 'laser-link', 'design');
const extension = path.extname('filter.svg');
console.log(folder, extension);
```

`server.mjs` uses `path` to locate the app root, portfolio workspace, compile workspace, uploaded files, package metadata, and compiler artifacts.

Do not build Windows paths by concatenating backslashes. Let `path` handle separators and then validate that the final result is still inside the expected root.

1.31 Filesystem promises

Node's `fs/promises` module provides promise-based functions for reading, writing, copying, listing, and deleting files.

The filesystem API is the controlled storage room of the builder. Every saved project, upload, draft, and publish mirror touches it.

`readFile` loads a file, `writeFile` saves content, `mkdir` creates folders, `readdir` lists folders, `cp` copies, and `rm` removes.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
import { mkdir, writeFile } from 'node:fs/promises';
await mkdir('docs/example', { recursive: true });
await writeFile('docs/example/readme.txt', 'Saved by the builder', 'utf8');
```

`server.mjs` uses these APIs to save drafts, accept uploads, prepare compile workspaces, synchronize publish files, and read context for the AI.

Filesystem code must be careful about path boundaries and cleanup. A bug here can overwrite the wrong file or leave temporary artifacts behind.

1.32 Environment variables

Environment variables are settings supplied outside the source code.

They are like jumper settings on the bench: the same board can behave differently depending on the external configuration.

Node reads environment variables through `process.env`. Values arrive as strings or undefined, so conversion and defaults are usually needed.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const port = Number(process.env.PORT || 8080);
const apiKeyConfigured = Boolean(process.env.OPENAI_API_KEY);
console.log({ port, apiKeyConfigured });
```

`server.mjs` uses environment variables for port, host, workspace roots, OpenAI settings, model choices, and tool paths.

Never put private secrets directly into source code. Use environment variables or a secure local store so the repository stays safe.

1.33 Child processes

A child process lets Node start another program such as git, gcc, java, iverilog, or an installer.

It is like asking a specialized instrument to do a job while the bench controller watches the output terminals.

execFile is useful for short commands. spawn is useful when output streams over time or the process may run longer.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
import { execFile } from 'node:child process';
execFile('git', ['status', '--short'], (error, stdout, stderr) => {
  console.log(error ? stderr : stdout);
});
```

server.mjs uses child processes for Git publishing, compiler execution, tool version checks, installer launches, and update handoffs.

Never build a command by concatenating untrusted strings into a shell command. Prefer executable plus argument array because it reduces quoting and injection problems.

1.34 Buffers and base64

A Buffer holds raw bytes. Base64 converts bytes into text so binary files can travel through JSON safely.

Base64 is like packaging a non-text component into a labeled shipping envelope that a text-only mail system can carry.

Buffer.from(text, 'base64') converts base64 text into bytes. A data URL often includes a prefix that must be removed before decoding.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const base64 = Buffer.from('hello').toString('base64');
const restored = Buffer.from(base64, 'base64').toString('utf8');
console.log(base64, restored);
```

server.mjs uses this pattern when the builder uploads files or images through JSON request bodies.

The mistake is treating base64 as security. It is only encoding. Anyone can decode it if they have the text.

1.35 Regular expressions

A regular expression is a pattern for searching and transforming text.

A regex is like a smart probe that can scan a whole waveform for a shape instead of checking one sample at a time.

Use regex for controlled text recognition such as file extension checks, whitespace cleanup, HTML stripping, and code language hints.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const source = 'module top: endmodule';
const isVerilog = /\b(module|endmodule|always|assign)\b/.test(source);
console.log(isVerilog);
```

server.mjs uses regex to clean HTML, detect code languages, identify GitHub text files, skip binary files, parse VCD waveforms, and sanitize text.

Regex can become unreadable if it tries to parse a full programming language or document format. Use it for narrow patterns, not as a replacement for a real parser.

1.36 Maps and Sets

A Map stores key-value pairs. A Set stores unique values.

A Set is like a parts tray that refuses duplicates. A Map is like a lookup table where each part number points to a known bin.

Use Set when uniqueness matters. Use Map when keys are not simple object property names or when you want explicit lookup behavior.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const blockedVersions = new Set(['0.2.16']);
const toolCache = new Map();
toolCache.set('git', 'C:/Program Files/Git/cmd/git.exe');
console.log(blockedVersions.has('0.2.16'), toolCache.get('git'));
```

server.mjs uses Set for blocked update versions and Maps for compiler tool caches and version caches.

Do not use arrays for repeated membership tests when the list may grow and uniqueness matters. A Set makes the intent clearer.

1.37 Dates and time windows

JavaScript Date represents a moment in time. Millisecond durations are common for timeouts and cache windows.

A cache time-to-live is like deciding how long a calibration sticker remains valid before the instrument must be checked again.

Date.now returns the current timestamp in milliseconds. Multiplying days, hours, minutes, seconds, and milliseconds makes readable duration constants.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const oneDayMs = 24 * 60 * 60 * 1000;
const expiresAt = Date.now() + oneDayMs;
console.log(new Date(expiresAt).toISOString());
```

server.mjs uses time windows for daily publishing authorization, extended trust after repeated authorizations, update checks, and elapsed command times.

Avoid mixing seconds and milliseconds without naming the variable clearly. A timeout that is off by one thousand can be painful.

1.38 Equality and coercion

Coercion is JavaScript's habit of converting one type to another in some operations.

Coercion is like an adapter that sometimes fits the connector but may hide the fact that the wrong cable was chosen.

Strict equality checks both type and value. Loose equality can convert types before comparing.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
console.log(0 == false); // true because of coercion
console.log(0 === false); // false because the types differ
console.log(Number('8080') === 8080);
```

server.mjs mostly uses explicit conversion and strict checks because backend decisions should not depend on surprising type conversion.

Use Number, String, Boolean, and JSON parsing deliberately. Do not let equality operators silently guess what a value should mean.

1.39 Mutation and object references

Objects and arrays are reference values. Assigning them to another variable copies the reference, not the whole object.

Two variables can point to the same equipment cart. Moving a tool on the cart is visible through both labels because the cart is shared.

Mutating a shared object changes it for everyone holding that reference. Copy with spread syntax or structured cloning when independent data is needed.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const original = { status: 'draft' };
const alias = original;
alias.status = 'published';
console.log(original.status);
```

server.mjs often creates new response objects rather than mutating request data directly, which makes backend results easier to predict.

The mistake is thinking const creates a deep constant. It does not. const protects the variable binding, not the object contents.

1.40 Spread syntax

Spread syntax expands arrays or objects into another array, object, or argument list.

Spread is like copying a known setup sheet and then changing one line for the current experiment.

Use ...array inside arrays or function calls. Use ...object inside object literals to copy properties into a new object.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const baseHeaders = { 'X-Content-Type-Options': 'nosniff' };
const headers = { ...baseHeaders, 'Content-Type': 'application/json' };
console.log(headers);
```

server.mjs uses spread when combining security headers with extra headers and when building JSON responses that extend existing result objects.

Spread creates a shallow copy. Nested objects are still shared unless copied separately.

1.41 Default values

Default values protect code when an input is missing.

A default is like a bench instrument returning to a safe range when the user has not selected a custom range.

Function parameters can have defaults. Object destructuring can have defaults. The `??` operator can choose a fallback for missing values.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
function makeFileName(name = 'main.is') {  
  return name.trim() || 'main.is';  
}  
console.log(makeFileName());
```

`server.mjs` uses defaults for host, port, language, filenames, model settings, compile timeouts, and optional request fields.

Defaults should be safe and boring. A dangerous action should not be the fallback.

1.42 Template objects and configuration maps

A configuration map stores many related rules in one object.

It is like a table in a datasheet: each row describes how one mode behaves, and the program chooses the row by name.

Use configuration objects when many branches share the same structure but differ in labels, extensions, tools, or limits.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const profiles = {  
  javascript: { defaultFile: 'main.is', tools: ['node'] },  
  nvthon: { defaultFile: 'main.nv', tools: ['nvthon'] },  
};  
console.log(profiles.javascript.defaultFile);
```

`server.mjs` uses `compileLanguageProfiles` as a rulebook for C, C++, Verilog, SystemVerilog, LTspice, Java, JavaScript, Python, and HTML.

A good configuration map prevents scattered if statements from drifting out of sync. The risk is letting the object become undocumented, so each field should have a clear meaning.

1.43 Sanitizing text

Sanitizing means reshaping unsafe or messy input into a form the program can safely use.

It is like deburring a metal part before placing it into a tight mechanical assembly. The part may be useful, but sharp edges can damage the system.

Common sanitizing steps include trimming whitespace, limiting length, removing dangerous characters, and replacing unsupported filename characters.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
function safeName(value) {  
  return String(value || 'file')  
    .trim()  
    .replace(/^[^a-z0-9_ -]+/gi, '-')  
    .slice(0, 80);  
}
```

`server.mjs` sanitizes project ids, section names, filenames, code filenames, public source text, and AI prompt context.

Sanitizing is not the same as authorization. A clean filename still needs to be written only under an allowed folder.

1.44 Validating data shape

Validation checks whether data has the structure and meaning the function expects.

Validation is the pre-flight checklist. The component may look right, but the test should not begin until required pins, supply rails, and limits are confirmed.

For arrays, use `Array.isArray`. For objects, check that the value exists and has needed properties. For strings, trim and check length.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
function validateCatalog(catalog) {  
  return Boolean(catalog && Arrav.isArray(catalog.categories) && Arrav.isArray(catalog.projects));  
}
```

`server.mjs` validates catalogs before saving them, upload bodies before writing files, publish targets before saving them, and AI questions before routing them.

Without validation, bad input can become corrupt project data, failed publishes, broken previews, or unsafe file operations.

1.45 HTTP status codes

HTTP status codes tell the caller whether a request succeeded, failed, or was rejected.

The status code is the traffic light on the transaction: green for success, red for failure, and special signals for missing or forbidden paths.

Common codes include 200 for success, 400 for bad input, 403 for forbidden, 404 for not found, and 503 for unavailable services.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
function statusForError(kind) {  
  if (kind === 'forbidden') return 403;  
  if (kind === 'missing') return 404;  
  return 400;  
}
```

`server.mjs` returns status codes alongside JSON so the browser can distinguish validation errors, blocked writes, AI unavailability, and missing static files.

Do not return 200 for everything unless the response body has a clear reason. The status code should help both browser code and human debugging.

1.46 Headers

HTTP headers are metadata attached to requests and responses.

Headers are like labels on a package: they say what is inside, how it should be handled, and which rules apply during transport.

Content-Type tells the browser how to interpret the body. Cache-Control tells it whether to reuse old content. Security headers constrain risky browser behavior.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const headers = {  
  'Content-Type': 'application/json',  
  'Cache-Control': 'no-store, no-cache, must-revalidate'  
};
```

server.mjs centralizes response headers so JSON, static files, and sensitive local API responses use consistent rules.

The most common bug is serving a file with the wrong content type, causing browsers to treat CSS, JavaScript, images, or PDFs incorrectly.

1.47 Local request checks

A local request check decides whether a request came from the same machine.

It is like checking that the person asking to publish is standing at the authorized bench, not calling from outside the room.

Local-only checks do not replace authentication, but they stop remote devices from using privileged builder endpoints over the network.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
function canWrite(requestIsLocal, userIsAuthorized) {  
  return requestIsLocal && userIsAuthorized;  
}
```

server.mjs applies local checks before write actions, compiler details, security reports, Git authentication, and publishing.

Do not expose file writes, command execution, or credential-related operations to arbitrary network clients. Local tools need local boundaries.

1.48 Command arguments

Command arguments are the ordered pieces passed to an external executable.

They are like the settings given to a lab instrument before pressing Run: target file, mode, output name, and flags.

Use arrays of arguments with `execFile` or `spawn`. This keeps each argument separate and avoids shell quoting surprises.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const gitArgs = ['push', 'origin', 'main'];  
const compileArgs = ['-std=c++20', '-Wall', 'main.cpp', '-o', 'main.exe'];  
console.log(gitArgs, compileArgs);
```

server.mjs builds argument arrays for Git, Java, gcc, g++, Icarus Verilog, vvp, PowerShell launchers, and update installers.

The danger is allowing untrusted text to become flags or extra commands. Safe filename helpers and workspace boundaries reduce that risk.

1.49 Streams of output

Long-running programs produce output over time. JavaScript can collect `stdout` and `stderr` as the process runs.

`stdout` and `stderr` are like two oscilloscope channels: one carries normal signal, and the other carries warnings or errors.

A runner can append chunks into strings, update logs, and return both channels to the UI.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
let stdout = '';
let stderr = '';
stdout += 'Build started\n';
stderr += '';
console.log({ stdout, stderr });
```

server.mjs captures compiler and simulator output so the Compile Code terminal can show errors, progress, and generated artifacts.

Do not wait for a user to click something else before showing output. The backend should return fresh output immediately, and the frontend should repaint when it arrives.

1.50 Caching

A cache remembers a result so repeated work can be skipped for a while.

A cache is like leaving a recently used tool on the bench instead of walking back to the cabinet every time.

Caches need keys, stored values, and invalidation rules. Time-based caches expire after a chosen window.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const cache = new Map();
function rememberTool(name, path) {
  cache.set(name, { path, savedAt: Date.now() });
}
```

server.mjs caches compiler tool paths, compiler versions, publishing authorization, and update information to keep the app responsive.

A cache that never expires can become wrong. A cache that expires too quickly becomes annoying. The useful balance depends on risk and cost.

1.51 Top-level await

Top-level await lets an ES module wait for a promise before the rest of the module continues.

It is like reading the instrument configuration before the bench controller announces that it is ready.

This syntax is available in ES modules, not in every JavaScript environment.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const packageText = await Promise.resolve('{ "version": "0.2.8" }');
const packageJson = JSON.parse(packageText);
console.log(packageJson.version);
```

server.mjs uses top-level await when reading package.json so version information is available to update checks.

Top-level await should be reserved for startup facts that the module truly needs. Too much startup waiting makes the app feel slow.

1.52 Importing built-in Node modules

Node's built-in modules provide standard operating-system and runtime tools.

They are the built-in instruments in the lab: power supply, meter, clock, file cabinet, process controller, and network jack.

The `node:` prefix makes it clear that the module is built into Node rather than installed from npm.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
import { createServer } from 'node:http';
import { readFile } from 'node:fs/promises';
import path from 'node:path';
```

`server.mjs` imports `node:http`, `node:child_process`, `node:crypto`, `node:fs/promises`, `node:os`, `node:path`, `node:util`, and `node:url`.

When reading a Node backend, the import list is a quick map of what powers the file uses. If `child_process` is imported, the file can run external programs.

1.53 Promisify

`promisify` converts some callback-style Node functions into promise-returning functions.

It is like adding a modern digital readout to an older instrument so the rest of the bench can use one consistent workflow.

The converted function can be awaited. This makes older APIs easier to combine with `async/await` code.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
import { execFile } from 'node:child process';
import { promisify } from 'node:util';
const execFileAsync = promisify(execFile);
const result = await execFileAsync('node', ['--version']);
```

`server.mjs` promisifyes `execFile` so short command checks can be written in the same `async` style as filesystem operations.

Promisify only works for callback APIs that follow the Node convention of `callback(error, result)`.

1.54 Hashing

A hash turns data into a fixed-size fingerprint. It is useful for detecting whether content changed.

A hash is like a checksum label on a downloaded part. If the part changes, the fingerprint changes.

Node's `crypto` module can create hashes. The same input produces the same hash, while different input should produce a different hash.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
import { createHash } from 'node:crypto';
const digest = createHash('sha256').update('source code').digest('hex');
console.log(digest.slice(0, 12));
```

`server.mjs` uses hashing around compile artifacts and update or source workflows where it needs stable fingerprints.

A hash is not encryption. It can verify identity or change, but it does not hide the original data by itself.

1.55 File extensions

A file extension is the suffix that helps humans and tools understand a file's format.

Extensions are like package labels: .sv suggests SystemVerilog, .cpp suggests C++, .pdf suggests a document.

Code should not trust an extension alone for security, but extensions are useful for selecting tools and content types.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
import path from 'node:path';
const ext = path.extname('design.sv').toLowerCase();
console.log(ext === '.sv' ? 'SystemVerilog' : 'Other');
```

server.mjs uses extensions to choose MIME types, detect compile languages, validate code filenames, and decide which GitHub files are useful for AI source context.

Allowing all project evidence files is different from compiling all files. The builder can store any evidence file while the compile workspace selectively runs supported languages.

1.56 Parsing small text formats

Parsing means reading structured text and turning it into useful program data.

It is like reading a waveform export or a test log and converting it into the signals, timestamps, and values the viewer needs.

Small formats can often be parsed with loops and regex. Complex languages should use real parsers when correctness matters.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const lines = '$var wire 1 ! clk $end\n#10\n1!'.split('\n');
const signalLines = lines.filter((line) => line.includes('$var'));
console.log(signalLines);
```

server.mjs parses JSON, VCD-like waveform text, Git remote output, process output, URLs, and source-code hints.

A parser should be strict enough to avoid nonsense and forgiving enough to report useful errors. Silent parsing failures are hard to debug.

1.57 Sorting and ranking

Sorting arranges values. Ranking chooses which values are more relevant.

Ranking is like choosing which measurement trace matters most for a debugging session.

JavaScript sort accepts a comparison callback. Negative means a comes before b, positive means b comes before a, and zero keeps them equivalent.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const results = [{ score: 4, name: 'README' }, { score: 9, name: 'main.c' }];
const ranked = [...results].sort((a, b) => b.score - a.score);
console.log(ranked[0].name);
```

server.mjs uses ranking ideas when selecting source excerpts, finding tools, and limiting AI context so the model sees the most useful evidence first.

Be careful: array sort mutates the original array. Copy first when the original order should be preserved.

1.58 Limiting text length

Length limits keep requests, logs, and model prompts from becoming too large.

It is like setting a current limit on a supply. The circuit can still work, but one accidental short does not burn the system down.

`String(value).slice(0, max)` is a simple way to clamp text. More advanced clamping can preserve whole words or important beginnings and endings.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
function clampText(value, maxLength = 12000) {  
  return String(value || '').slice(0, maxLength);  
}
```

`server.mjs` clamps AI questions, source excerpts, conversation history, public fetches, and generated context.

Without limits, one large upload or fetched page could slow the app, exceed API limits, or make the AI answer worse by burying the useful evidence.

1.59 Recursion and directory walking

Recursion happens when a function calls itself to solve a smaller version of the same problem.

Walking folders is like opening a cabinet, then opening each drawer, then opening each box inside each drawer until the tool is found.

Recursive functions need a stopping condition. Directory walkers also need depth limits so a broad filesystem does not become an endless search.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
async function walk(folder, depth) {  
  if (depth < 0) return [];  
  // read entries, collect matches, then call walk(child, depth - 1)  
  return [];  
}
```

`server.mjs` uses recursive searching to find compiler executables under likely installation folders.

The danger is scanning too much of the computer. Restrict the root folder and maximum depth.

1.60 Event listeners

An event listener is a function that runs when something happens.

It is like triggering on a rising edge: the code waits, and when the event arrives, the callback runs.

Servers, streams, child processes, DOM elements, and timers all use event-like behavior.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
process.on('exit', (code) => {  
  console.log(`Process exited with ${code}`);  
});
```

`server.mjs` listens for process output, process exit, server requests, and update-start events.

Event code should clean up after itself when the event source is temporary. Long-lived listeners can cause confusing repeated behavior.

1.61 Timers

Timers schedule code to run later.

A timer is like telling the lab assistant to check the oven in three minutes while the rest of the work continues.

`setTimeout` runs once after a delay. `setInterval` repeats. Promises can wrap timers when `async/await` style is preferred.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
setTimeout(() => {  
  console.log('Relaunch check after delay');  
}, 3000);
```

`server.mjs` uses timers around process timeouts, update handoff delays, and controlled shutdown after launching an updater.

Timers should not be used as a substitute for real completion signals when an API can tell you exactly when work is done.

1.62 URL fetching

`fetch` sends HTTP requests from JavaScript. In Node, modern runtimes provide `fetch` globally.

Fetching is the backend reaching outside the local bench to ask a public service or API for data.

The response must be checked. `response.ok` tells whether the HTTP status is in the success range, and `response.json` or `response.text` reads the body.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const response = await fetch('https://api.github.com');  
if (!response.ok) throw new Error(`HTTP ${response.status}`);  
const text = await response.text();  
console.log(text.slice(0, 80));
```

`server.mjs` fetches OpenAI responses, public GitHub source files, public pages used as AI context, and release information.

Do not assume every `fetch` returns JSON. Some endpoints return HTML, raw source text, binary data, or an error page.

1.63 API payload design

An API payload is the object sent between frontend and backend.

It is the form the user filled out, but in structured machine-readable form.

Good payloads use clear field names, predictable data shapes, and enough information for the backend to validate the request.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const payload = {  
  language: 'systemverilog',  
  files: [{ name: 'design.sv', code: 'module top; endmodule' }],  
  action: 'simulate'  
};
```

`server.mjs` receives payloads for catalog saves, uploads, compile actions, code beautifying, GitHub authentication, AI questions, and update actions.

A payload should describe intent, not backend internals. The frontend asks for a compile; it should not decide unsafe filesystem paths.

1.64 Consistent response objects

A consistent response object lets the frontend handle success and failure without guessing.

It is like every lab report using the same headings: pass/fail, observations, output, and notes.

A common shape is { ok, data, error }. More complex operations may include stdout, stderr, elapsedMs, artifacts, or publish details.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const response = {  
  ok: true,  
  result: { stdout: 'Hello', stderr: '', elapsedMs: 41 },  
  error: null  
};
```

server.mjs returns consistent objects for compilers, publish attempts, update checks, security reports, uploads, and AI calls.

When the response shape changes randomly, UI code becomes full of defensive guesses and bugs. Consistency is a kindness to the rest of the app.

1.65 Comments

Comments explain intent to humans. They do not run.

A useful comment is like a schematic note explaining why a non-obvious component is there, not a label saying that a resistor is a resistor.

Use comments to explain risk, constraints, or unusual decisions. Avoid comments that merely repeat the code.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
// Try the next candidate because Windows PATH often misses tool installs.  
continue;
```

server.mjs is mostly self-describing through names, but a few comments help where fallback behavior or skipped errors would otherwise look strange.

Bad comments age poorly. If a comment says one thing and the code does another, the comment becomes a bug.

1.66 Naming

Names are part of the design. A good variable or function name tells the reader what kind of value or behavior they are looking at.

A name is like a net label. Good labels make a schematic readable even before you trace every wire.

Use nouns for values, verbs for actions, and precise phrases for boundary-sensitive helpers.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const publishAuthCachePath = '.omb-publish-session.json';  
function resolveInsideCompileRoot(relativePath) {  
  return relativePath;  
}
```

server.mjs has names such as resolveInsideRoot, publishAuthCachePath, compileLanguageProfiles, and handlePortfolioAi that reveal their jobs before the body is read.

Do not use vague names like data, temp, thing, and result everywhere. result is fine when the surrounding operation is clear; otherwise say what kind of result it is.

1.67 Organizing code by responsibility

Code becomes understandable when related behavior lives together and repeated behavior is pulled into helpers.

A good file layout is like a well-organized bench: power tools in one area, measurement tools in another, parts storage in another.

Shared utilities should appear before features that use them. Feature coordinators can then read like stories that call those utilities.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
function helper() {  
  return 'small repeated behavior';  
}  
  
async function feature() {  
  return helper();  
}
```

server.mjs starts with imports and configuration, then helpers, then feature operations, then API routing, then server startup.

The file is broad, so feature-based documentation matters. Reading it by button behavior is easier than reading every helper in raw order.

1.68 Reading stack traces

A stack trace shows the path of function calls that led to an error.

It is like a chain of test steps: the final fault appears at the top, but the earlier setup steps explain how the program arrived there.

Read the error message first, then the closest project file entry, then the call chain below it.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
try {  
  throw new Error('Catalog must include projects');  
} catch (error) {  
  console.error(error.stack);  
}
```

server.mjs catch blocks often convert raw errors into messages the UI can show, but during development the terminal stack trace remains useful.

Do not panic at long traces. Find the first line that belongs to your code and connect it to the request or button that triggered the error.

1.69 Debugging with console output

console.log prints values so you can inspect what the program believes is happening.

It is the simplest diagnostic probe. You attach it, observe the signal, then remove it once the circuit is understood.

Print objects, important branch decisions, request paths, and unexpected values. Keep production logs concise so real errors do not drown in noise.

A small example makes the syntax visible before the server code uses it in a larger setting.

```
const route = '/api/save-draft':
console.log('Handling route:', route);
console.log({ route, localOnly: true });
```

server.mjs prints startup URLs and can expose command output through structured responses rather than raw terminal-only logs.

Do not log private secrets, tokens, API keys, or full credentials. Debugging should not create a security leak.

1.51 Slow Syntax Reading Lab

The previous sections taught the ideas one at a time. This section slows down realistic backend-shaped fragments and reads them as JavaScript. The point is not to memorize the examples. The point is to build the habit of seeing punctuation, operators, function calls, data shapes, and control flow as readable design decisions.

Each subsection below keeps the explanation inside normal prose. There are no reference cards to memorize. Read the code, then read how the syntax behaves, then connect that behavior back to server.mjs.

1.51.1 Reading an import list

This reading lab focuses on how JavaScript announces the outside tools a file will use.

Study the fragment as a miniature version of the same syntax used by the backend.

```
import { createServer } from 'node:http';
import { execFile, spawn } from 'node:child process';
import path from 'node:path';
```

The braces around createServer, execFile, and spawn mean named imports. The path import has no braces because the module's default export is being given the local name path. The strings after from are module specifiers.

At the top of server.mjs, imports reveal the file's powers before any function is read. HTTP serving, child processes, filesystem access, path handling, crypto hashing, and URL conversion are all visible from the import section.

A beginner may skip imports because they look like setup. In a backend file, they are a map of what the file is allowed to do.

Before reading a new JavaScript file, pause at the imports and say what each import makes possible.

1.51.2 Reading a top-level constant

This reading lab focuses on how stable module-level values become shared configuration.

Study the fragment as a miniature version of the same syntax used by the backend.

```
const port = Number(process.env.PORT || 8080);
const host = process.env.HOST || '0.0.0.0';
```

`const` creates names that cannot be reassigned. `Number(...)` converts the chosen port text into a numeric value. The `||` operator chooses the environment value when it exists, otherwise a fallback.

`server.mjs` uses this shape to keep startup settings outside feature functions. Later code can call `listen(port, host)` without recalculating where those values came from.

The subtle risk is that `process.env` values are strings. Code that forgets conversion may accidentally compare text with numbers.

When you see `process.env`, ask what type the code needs after the setting is read.

1.51.3 Reading an object literal as a lookup table

This reading lab focuses on how braces create a table of named answers.

Study the fragment as a miniature version of the same syntax used by the backend.

```
const types = {  
  '.html': 'text/html',  
  '.js': 'application/javascript',  
  '.pdf': 'application/pdf'  
};
```

The outer braces create an object. Each quoted extension is a property name. The colon separates a key from its value. Commas separate entries.

The static file server uses this kind of object to choose Content-Type headers. It is faster to read than a long chain of if statements.

Do not think of every object as a physical thing. In JavaScript, objects are often dictionaries that answer questions by key.

Practice by reading each entry as a sentence: when the extension is `.html`, the content type is `text/html`.

1.51.4 Reading an array allowlist

This reading lab focuses on how square brackets store an ordered list of permitted values.

Study the fragment as a miniature version of the same syntax used by the backend.

```
const publishPaths = [  
  'projects.json',  
  'docs',  
  'assets',  
  'index.html'  
];
```

The square brackets create an array. Each string is one item. The trailing comma after the final item is allowed in modern JavaScript and makes future edits cleaner.

The publishing feature uses this list as an allowlist. Only these website-safe paths are copied and staged when applying changes to the public site.

The trap is to treat a list like decoration. In backend code, a list can be a security boundary.

When reading arrays in server code, ask whether the order matters, whether duplicates matter, and whether the list is a permission rule.

1.51.5 Reading a Set

This reading lab focuses on how JavaScript represents unique membership.

Study the fragment as a miniature version of the same syntax used by the backend.

```
const blockedAppUpdateVersions = new Set(['0.2.16']):
if (blockedAppUpdateVersions.has(version)) {
  throw new Error('This release is blocked.'):
}
```

`new Set(...)` constructs a Set object from an array. The `has` method asks whether a value is a member. The `if` statement branches on that boolean.

The updater can skip a known bad release version without removing every release artifact from GitHub.

A beginner may use arrays for every list. Sets communicate that membership, not order, is the main idea.

Use a Set when the question is 'is this value included?' and duplicates should not matter.

1.51.6 Reading a Map cache

This reading lab focuses on how JavaScript stores computed answers by key.

Study the fragment as a miniature version of the same syntax used by the backend.

```
const compileToolCache = new Map():
compileToolCache.set('git', 'C:/Program Files/Git/cmd/git.exe'):
const gitPath = compileToolCache.get('git');
```

`new Map` creates a key-value store. `set` writes a key and value. `get` retrieves the value later.

Compiler detection is expensive if the app searches the machine every time. `server.mjs` uses maps to remember tool paths and version strings.

The trap is forgetting that `get` returns `undefined` when the key is missing. Code should handle that absence.

When reading a cache, look for the key, the stored value, and the invalidation rule.

1.51.7 Reading object spread

This reading lab focuses on how objects are copied and extended.

Study the fragment as a miniature version of the same syntax used by the backend.

```
return {
  'X-Content-Type-Options': 'nosniff',
  ...extra
};
```

The `...extra` syntax copies properties from the `extra` object into the returned object. If a copied property has the same name as an earlier property, the later value wins.

`securityHeaders` uses this to keep standard headers while still allowing a response to add content type or cache fields.

The trap is thinking spread is deep. It copies the top level. Nested objects can still be shared.

When you see spread, ask what base object is being copied and what is allowed to override it.

1.51.8 Reading default parameters

This reading lab focuses on how a function supplies a value when the caller omits one.

Study the fragment as a miniature version of the same syntax used by the backend.

```
function securityHeaders(extra = {}) {
```

```
    return { ...extra };
}
```

`extra = {}` means the function receives an empty object when the caller does not pass a value. That keeps the spread operation from failing.

Default parameters make helper functions easier to call safely from many endpoints.

The trap is using a default that hides a required input. Defaults should represent safe optional behavior.

When a parameter has a default, ask whether the function can truly do useful work without the caller supplying that value.

1.51.9 Reading a JSON response helper

This reading lab focuses on how one function standardizes response shape.

Study the fragment as a miniature version of the same syntax used by the backend.

```
function sendJson(response, status, data) {
  response.writeHead(status, { 'Content-Type': 'application/json' });
  response.end(JSON.stringify(data, null, 2));
}
```

The parameters name the HTTP response object, numeric status, and data payload. `JSON.stringify` turns the data object into text. `response.end` sends the final body.

`server.mjs` uses a fuller version of this helper so every API answer uses consistent headers and no-store cache behavior.

The trap is returning raw JavaScript objects to an HTTP response. HTTP bodies are bytes or text, so the object must be serialized.

When reading a response helper, identify when headers are written and when the body is ended.

1.51.10 Reading request-body collection

This reading lab focuses on how asynchronous chunks become one string.

Study the fragment as a miniature version of the same syntax used by the backend.

```
let raw = '';
for await (const chunk of request) {
  raw += chunk;
}
const body = JSON.parse(raw);
```

`let` is used because `raw` changes as chunks arrive. `for await...of` reads an asynchronous stream. `JSON.parse` converts the final text into an object.

`readRequestJson` uses this pattern with a size limit so browser POST bodies can be received safely.

The trap is parsing before all chunks have arrived or accepting unlimited body size.

When reading stream code, find the accumulator, the loop, the limit, and the final conversion.

1.51.11 Reading a guard clause

This reading lab focuses on how a function stops early when a condition fails.

Study the fragment as a miniature version of the same syntax used by the backend.

```
if (!question) {
```

```

    sendJson(response, 400, { error: 'Question is required.' });
    return;
}

```

The `!` operator reverses truthiness. The function sends an error response and returns immediately, so the rest of the function does not run.

`handlePortfolioAi` uses this shape before spending work on context enrichment or model calls.

The trap is continuing after an error response. Sending two responses to one HTTP request causes confusing failures.

When you see `return` inside an `if` block, ask what unsafe or unnecessary work it prevents.

1.51.12 Reading path normalization

This reading lab focuses on how code converts path pieces into a safe absolute path.

Study the fragment as a miniature version of the same syntax used by the backend.

```

const target = path.normalize(path.join(root, ...segments));
if (!target.startsWith(root)) {
    throw new Error('Path escaped the workspace.');
```

`path.join` combines the root and segments. The `...segments` syntax passes each array item as a separate argument. `startsWith` checks whether the final path still belongs under the approved root.

`resolveInsideRoot` and its sibling helpers are central because uploads, drafts, static files, compile files, and publish mirrors all touch disk.

The trap is checking the raw input instead of the final normalized path. Attack text can look harmless before normalization.

When reading filesystem code, follow the path from user input to final absolute path.

1.51.13 Reading a regular-expression test

This reading lab focuses on how code detects a language clue.

Study the fragment as a miniature version of the same syntax used by the backend.

```

if (/^\b(module|endmodule|always|assign)\b/.test(source)) {
    return 'verilog';
}

```

The slashes create a regular expression. `\b` means word boundary. The parentheses group alternatives. `test` returns true or false.

`server.mjs` uses regex language hints so pasted code or uploaded source can be classified before running a compiler or beautifier.

The trap is treating regex detection as perfect. It is a useful hint, not a full parser.

When reading regex, translate it into an English sentence before trusting it.

1.51.14 Reading a filename sanitizer

This reading lab focuses on how code turns user text into a safe file segment.

Study the fragment as a miniature version of the same syntax used by the backend.

```

const safe = String(value || 'file')
    .trim()
    .replace(/[^\a-z0-9. -]/gi, '-')
    .slice(0, 80);

```

The expression is a method chain. `String` converts the input. `trim` removes edge whitespace. `replace` swaps unsupported characters. `slice` limits length.

Upload and compile features need sanitized filenames because user-supplied names should not become arbitrary filesystem instructions.

The trap is believing sanitizing a name is enough. The final path still needs root-boundary validation.

When reading a sanitizer, identify what it removes, what it keeps, and how long the output may be.

1.51.15 Reading optional chaining

This reading lab focuses on how code safely reaches into an object that might be missing.

Study the fragment as a miniature version of the same syntax used by the backend.

```
const message = data?.error?.message || 'OpenAI request failed.';
```

The `?.` operator stops property access when the value on the left is null or undefined. The `||` fallback supplies text when the chosen value is falsy.

OpenAI, GitHub, Git, and compiler responses may have different shapes when they fail. Optional chaining keeps error reporting from crashing while reporting an error.

The trap is overusing `||` when an empty string or zero should be respected. For absence-only fallbacks, use `??`.

When you see optional chaining, ask which object may be missing and what fallback the code chooses.

1.51.16 Reading nullish coalescing

This reading lab focuses on how code falls back only for null or undefined.

Study the fragment as a miniature version of the same syntax used by the backend.

```
const exitCode = result.code ?? 'unknown';
const stdout = result.stdout ?? '';
```

The `??` operator chooses the right-hand value only when the left side is null or undefined. It preserves zero and empty strings.

Terminal output often contains empty strings that are meaningful. A command can have no stdout and still be successful.

The trap is replacing empty output with a fallback and accidentally making the UI look like output existed.

Use `??` when absence is the problem, not every falsy value.

1.51.17 Reading an async filesystem write

This reading lab focuses on how code waits for local storage to finish.

Study the fragment as a miniature version of the same syntax used by the backend.

```
await mkdir(folder, { recursive: true });
await writeFile(filePath, Buffer.from(base64, 'base64'));
```

await pauses the async function until each filesystem promise settles. The recursive option creates missing parent folders. Buffer.from decodes base64 bytes before writing.

The upload endpoint uses this shape to store files under the project docs folder.

The trap is writing a file before the folder exists or forgetting to decode binary upload data.

When reading file writes, check the folder creation, the content conversion, and the final path.

1.51.18 Reading a try/catch around a feature

This reading lab focuses on how endpoint code converts failure into JSON.

Study the fragment as a miniature version of the same syntax used by the backend.

```
try {
  const result = await compileAndRunCode(bodv);
  sendJson(response, 200, { ok: result.ok, result });
} catch (error) {
  sendJson(response, 400, { ok: false, error: error.message });
}
```

The try block contains the normal path. The catch block receives thrown errors and sends a controlled response instead of crashing the server.

server.mjs uses this structure around compiler, GitHub, update, upload, and save operations.

The trap is swallowing the error without sending enough context back to the UI.

When reading catch blocks, ask what the user will see and what detail remains available for debugging.

1.51.19 Reading array filtering

This reading lab focuses on how code keeps only useful items.

Study the fragment as a miniature version of the same syntax used by the backend.

```
const textFiles = files.filter((file) => githubTextFilePattern.test(file.path));
```

filter calls the arrow function once for each item. Items whose callback returns true are included in the new array.

The AI source reader uses filtering to avoid fetching binary or irrelevant repository files.

The trap is doing expensive work inside a filter when a simpler pre-check would discard most items earlier.

When reading filter, say the rule out loud: keep files whose path matches the text-file pattern.

1.51.20 Reading array mapping

This reading lab focuses on how code transforms every item in a list.

Study the fragment as a miniature version of the same syntax used by the backend.

```
const labels = projects.map((project) => project.title || project.id);
```

map returns a new array where each output item comes from one input item. The original array is not changed by map itself.

Project and AI context code often transforms rich project objects into shorter labels, excerpts, or searchable text.

The trap is using map for side effects and ignoring the returned array. forEach is clearer when no transformed result is needed.

When reading map, identify the input item, the output expression, and where the new array is used.

1.51.21 Reading array sorting

This reading lab focuses on how code ranks results.

Study the fragment as a miniature version of the same syntax used by the backend.

```
const ranked = [...matches].sort((a, b) => b.score - a.score);
```

The spread creates a shallow copy before sort mutates the array. The comparison callback orders higher scores first.

Search and AI context features need ranking so the most relevant files or project sections appear first.

The trap is forgetting that sort mutates its array. Copying first preserves the original order for other uses.

When reading sort, check whether the code copies first and what the comparison returns.

1.51.22 Reading a generated command

This reading lab focuses on how arrays represent executable arguments.

Study the fragment as a miniature version of the same syntax used by the backend.

```
const args = ['-std=c++20', '-Wall', sourceFile, '-o', outputFile];
const result = await runProcess(gpp, args, { cwd: workspace });
```

The command is split into an executable and an argument array. The cwd option says which folder the process should run inside.

The compile feature uses this structure so compiler flags, source files, and output paths stay separate.

The trap is building one long shell string with untrusted text. Argument arrays are easier to reason about and safer.

When reading command code, find the executable, argument list, working directory, timeout, and output handling.

1.51.23 Reading Promise-based process output

This reading lab focuses on how a spawned program becomes a structured result.

Study the fragment as a miniature version of the same syntax used by the backend.

```
const child = spawn(command, args, options);
child.stdout.on('data', (chunk) => { stdout += chunk; });
child.stderr.on('data', (chunk) => { stderr += chunk; });
```

spawn starts the process. The data event fires whenever output arrives. The callback appends each chunk to the accumulated output string.

runProcess needs this behavior so compiler and simulator output can be returned to the builder terminal.

The trap is reading only stdout. Many compilers report diagnostics on stderr even when the process exits with useful information.

When reading process code, track stdout, stderr, exit code, timeout, and cleanup.

1.51.24 Reading a timeout

This reading lab focuses on how code prevents an operation from running forever.

Study the fragment as a miniature version of the same syntax used by the backend.

```
const timer = setTimeout(() => {
  timedOut = true;
  child.kill();
}, timeoutMs);
```

setTimeout schedules a callback. The callback records that the operation timed out and kills the child process.

Compiler runs, installer runs, and network operations need limits so the app does not appear frozen forever.

The trap is forgetting to clear a timeout when work finishes normally. A late timeout can interfere with completed work.

When reading timeout code, find where the timer is created, where it is cleared, and what happens when it fires.

1.51.25 Reading a URL constructor

This reading lab focuses on how code parses request paths safely.

Study the fragment as a miniature version of the same syntax used by the backend.

```
const url = new URL(request.url || '/', `http://${request.headers.host}`);
if (url.pathname.startsWith('/api/')) {
  await handleApi(request, response, url);
}
```

new URL needs a full base URL when the request URL is relative. pathname then exposes the path without query string confusion.

The server startup code uses URL parsing before deciding whether a request is an API call or a static file request.

The trap is splitting raw URL strings by hand and missing encoding, query strings, or missing values.

When reading routing code, identify which URL property controls the route.

1.51.26 Reading decodeURIComponent

This reading lab focuses on how encoded URL text becomes a path-like string.

Study the fragment as a miniature version of the same syntax used by the backend.

```
const pathname = url.pathname === '/' ? '/index.html' : decodeURIComponent(url.pathname);
```

The conditional expression chooses index.html for the root path. decodeURIComponent converts URL-encoded characters back into normal text.

Static serving needs this because browser URLs may contain encoded spaces or other characters.

The trap is decoding and then trusting the result without path boundary checks.

When you see URL decoding, look immediately for normalization and root validation afterward.

1.51.27 Reading content-type lookup

This reading lab focuses on how extension maps become response headers.

Study the fragment as a miniature version of the same syntax used by the backend.

```
const contentType = types[path.extname(filePath)] || 'application/octet-stream';
```

`path.extname` extracts the extension. The object lookup checks whether that extension has a known content type. The fallback handles unknown binary-like files.

The static server uses this expression so browsers render CSS, JavaScript, PDFs, images, and icons correctly.

The trap is serving JavaScript or CSS with a generic or wrong type, which can make browsers reject or misread the file.

When reading lookup code, identify the key, the map, and the fallback.

1.51.28 Reading a response end

This reading lab focuses on how an HTTP reply is completed.

Study the fragment as a miniature version of the same syntax used by the backend.

```
response.writeHead(200, { 'Content-Type': contentType });
response.end(body);
```

`writeHead` sends status and headers. `end` sends the body and closes the response.

Every request must eventually receive one completed response. Static serving uses this for file bytes, and API helpers use it for JSON text.

The trap is trying to write another response after `end` has already been called.

When reading HTTP code, find the one path that ends the response for each branch.

1.51.29 Reading a fetch request

This reading lab focuses on how backend JavaScript calls an external API.

Study the fragment as a miniature version of the same syntax used by the backend.

```
const openAiResponse = await fetch('https://api.openai.com/v1/responses', {
  method: 'POST',
  headers: { Authorization: `Bearer ${apiKey}` },
  body: JSON.stringify(payload)
});
```

`fetch` receives a URL and an options object. The method selects POST. headers carries authorization and content type information. body sends serialized JSON.

`handlePortfolioAi` uses this pattern when an OpenAI API key is configured.

The trap is forgetting to stringify the body or accidentally exposing an API key in browser code.

When reading `fetch`, identify the URL, method, headers, body shape, and error handling after the response.

1.51.30 Reading fallback model logic

This reading lab focuses on how code tries a second option after a specific failure.

Study the fragment as a miniature version of the same syntax used by the backend.

```
if (!openAiResponse.ok && fallbackModel && fallbackModel !== model) {  
  ({ data, openAiResponse, selectedModel } = await callModel(fallbackModel));  
}
```

The condition checks several requirements. The assignment uses destructuring with parentheses because the target variables already exist.

The AI feature can retry with a fallback model when the preferred model is unavailable and no explicit model override was supplied.

The trap is retrying every error blindly. Some errors should be shown immediately instead of repeated.

When reading fallback logic, identify which failures qualify and what state is replaced by the retry.

1.51.31 Reading object shorthand

This reading lab focuses on how property names can come from variable names.

Study the fragment as a miniature version of the same syntax used by the backend.

```
const ok = true;  
const file = 'projects.local.json';  
sendJson(response, 200, { ok, file });
```

Inside an object literal, `ok` is shorthand for `ok: ok` and `file` is shorthand for `file: file`.

`server.mjs` uses shorthand frequently in JSON responses because the variable names already describe the response fields.

The trap is missing that the property name and variable name are the same. The object still has explicit fields.

When reading shorthand, mentally expand it once, then keep reading normally.

1.51.32 Reading nested objects

This reading lab focuses on how response data can carry structured detail.

Study the fragment as a miniature version of the same syntax used by the backend.

```
sendJson(response, 200, {  
  ok: false,  
  publish: {  
    nushed: false,  
    authorizationRequired: true  
  }  
});
```

The outer object describes the API response. The nested `publish` object groups publishing-specific detail.

Apply to site uses nested detail so the frontend can show whether the catalog saved, whether publishing was blocked, and why.

The trap is flattening everything into one level and creating vague names. Nested objects preserve meaning.

When reading nested data, identify what each level represents.

1.51.33 Reading string cleanup

This reading lab focuses on how text is made compact and safe for prompts.

Study the fragment as a miniature version of the same syntax used by the backend.

```
const compact = String(value || '')
  .replace(/\s+/g, ' ')
  .trim()
  .slice(0, 12000);
```

The chain converts to text, compresses whitespace, removes edge spaces, and applies a maximum length.

AI context and public source excerpts need this kind of cleanup so prompts stay useful and bounded.

The trap is cleaning so aggressively that meaningful line breaks or code formatting are lost. Different content types need different cleanup rules.

When reading cleanup code, ask whether it is preparing prose, code, filenames, URLs, or JSON.

1.51.34 Reading HTML stripping

This reading lab focuses on how a backend turns markup into plain text.

Study the fragment as a miniature version of the same syntax used by the backend.

```
const text = htmlText
  .replace(/<script[\s\S]*?<\script>/gi, ' ')
  .replace(/<[^>]+>/g, '');
```

The first replace removes script blocks. The second replace removes remaining tags. The flags make the regex global and case-insensitive.

The AI context builder may fetch public pages and needs text, not raw HTML markup.

The trap is treating regex stripping as a perfect HTML parser. It is a practical cleanup step for safe excerpts, not a full browser.

When reading HTML cleanup, check whether script and style content are removed before tags are stripped.

1.51.35 Reading branch-specific Git commands

This reading lab focuses on how command arrays express repository actions.

Study the fragment as a miniature version of the same syntax used by the backend.

```
await runPublishGit(['fetch', 'origin', branch]);
await runPublishGit(['push', 'origin', branch]);
```

Each command is an array of arguments. The branch variable becomes one argument, not part of a shell string.

Publishing and loading from target rely on Git commands because the website lives in a GitHub Pages repository.

The trap is hardcoding master when the repository branch is main, or vice versa. The code should resolve and use the configured branch.

When reading Git code, trace which branch is being used and where it came from.

1.51.36 Reading a commit decision

This reading lab focuses on how code avoids empty commits.

Study the fragment as a miniature version of the same syntax used by the backend.

```
const hasChanges = status.stdout.trim().length > 0;
if (hasChanges) {
  await runPublishGit(['commit', '-m', message]);
}
```

trim removes whitespace. length checks whether any status text remains. The commit runs only when tracked publish paths actually changed.

Apply to site should not create meaningless commits when the saved site output matches the repository.

The trap is committing every time a button is clicked, which creates noisy history and makes real changes harder to review.

When reading publish code, find how it decides whether a commit is necessary.

1.51.37 Reading a local-only route

This reading lab focuses on how the backend blocks privileged network requests.

Study the fragment as a miniature version of the same syntax used by the backend.

```
if (!isLocalRequest(request)) {
  sendJson(response, 403, { error: 'Write actions are only allowed from this computer.' });
  return true;
}
```

The ! operator flips the result. 403 says forbidden. Returning true tells the outer server that the request has been handled.

The backend uses this pattern before write actions, compiler operations, target authentication, security reports, and update starts.

The trap is exposing dangerous local functions to other devices on the network because the preview server listens on a reachable host.

When reading a privileged endpoint, look for the local-request gate before the side effect.

1.51.38 Reading a body shape check

This reading lab focuses on how code refuses malformed catalog data.

Study the fragment as a miniature version of the same syntax used by the backend.

```
if (!catalog || !Array.isArray(catalog.categories) || !Array.isArray(catalog.projects)) {
  sendJson(response, 400, { error: 'Catalog must include categories and projects arrays.' });
  return true;
}
```

The condition checks presence and array types. The || operator means any failed check triggers the error response.

Save draft and Apply to site need this because the catalog is the public website's main data model.

The trap is saving whatever the browser sends. A broken catalog can break previews and the live site.

When reading validation, list each required field and the type check used for it.

1.51.39 Reading an upload decode

This reading lab focuses on how a data URL becomes bytes on disk.

Study the fragment as a miniature version of the same syntax used by the backend.

```
const base64 = String(bodv.data || '').replace(/^data:[^:1+:base64./, ' ');
await writeFile(filePath, Buffer.from(base64, 'base64'));
```

The replace removes the data URL prefix if one exists. Buffer.from decodes the remaining base64 text into bytes before writing.

The upload route uses this for images, PDFs, source files, schematics, and other evidence files.

The trap is writing the base64 text itself instead of decoded bytes, which creates corrupted files.

When reading upload code, follow the data from request field to decoded buffer to final path.

1.51.40 Reading a content manifest

This reading lab focuses on how a list of paths becomes publish behavior.

Study the fragment as a miniature version of the same syntax used by the backend.

```
for (const relativePath of publishPaths) {
  const sourcePath = resolveInsideRoot(relativePath);
  const targetPath = resolveInsidePortfolioRoot(relativePath);
}
```

for...of steps through each allowlisted path. Each path is resolved against both the builder root and portfolio root.

syncPortfolioPublishFiles uses this pattern to copy website-safe files into the publish mirror.

The trap is copying the whole app folder. The allowlist prevents internal builder files from leaking into the public site.

When reading synchronization code, identify the source root, target root, and path allowlist.

1.51.41 Reading a function that returns a rich object

This reading lab focuses on how backend features report multiple facts at once.

Study the fragment as a miniature version of the same syntax used by the backend.

```
return {
  copied,
  skipped,
  workspace: portfolioRoot,
  separatedWorkspace: true
};
```

The returned object carries arrays, a path, and a boolean. The caller can display or log whichever fields matter.

Publishing synchronization returns this kind of object so the UI can explain what was copied and what was skipped.

The trap is returning only true or false from complex operations. A boolean cannot explain what happened.

When reading returns, ask what the caller needs to know next.

1.51.42 Reading model instructions as data

This reading lab focuses on how long strings can become behavior contracts.

Study the fragment as a miniature version of the same syntax used by the backend.

```
const portfolioAiInstructions = [
  'Answer the visitor question first.',
  'Do not invent portfolio projects.'
].join('\n');
```

The array stores separate instruction strings. `join('\n')` turns them into one newline-separated prompt.

The AI feature uses this as a developer instruction block before sending the visitor question and portfolio context.

The trap is treating prompts as random prose. In an AI-backed feature, prompt text is application logic.

When reading AI code, treat instructions, context construction, model selection, and response extraction as separate responsibilities.

1.51.43 Reading conversation history

This reading lab focuses on how prior messages influence a new answer.

Study the fragment as a miniature version of the same syntax used by the backend.

```
const conversation = cleanConversationHistory(bodv.conversation);
const context = await enrichPortfolioContext({ ...(body.context || {}), question });
```

The first line sanitizes recent messages. The second line creates a context object by spreading existing context and adding the current question.

Ask My Portfolio needs this because follow-up questions depend on what was asked earlier.

The trap is sending unlimited chat history to a model. Useful history should be cleaned and bounded.

When reading chat code, find where history is limited and how it combines with the current question.

1.51.44 Reading a provider fallback

This reading lab focuses on how the backend chooses between local and cloud AI.

Study the fragment as a miniature version of the same syntax used by the backend.

```
if (!apiKey) {
  const ollama = await callOllamaPortfolioAi({ question, intent, conversation, context });
  if (ollama.ok) {
    sendJson(response, 200, { answer: ollama.answer, provider: 'ollama' });
    return;
  }
}
```

The outer if checks whether OpenAI is configured. The inner if checks whether the local Ollama answer succeeded. The return stops the cloud path from running afterward.

This lets the website assistant work with a local model when no cloud key is available.

The trap is falling through after a successful fallback and accidentally attempting another provider.

When reading fallback code, find the success return that prevents duplicate work.

1.51.45 Reading update handoff code

This reading lab focuses on how a running app prepares to replace itself.

Study the fragment as a miniature version of the same syntax used by the backend.

```
const child = spawn('cmd.exe', ['/d', '/c', launcherCommandPath], {
  detached: true,
  stdio: 'ignore',
```

```
    windowsHide: true
  });
  child.unref();
```

spawn starts a command process. detached lets it survive independently. stdio ignore disconnects it from the current process. unref allows the current Node process to exit.

The updater uses this because the app must close before its files can be replaced and then relaunched.

The trap is trying to run the installer inside the same process that is about to be shut down.

When reading update code, identify the handoff process and why it must survive app exit.

1.51.46 Reading event emission

This reading lab focuses on how code announces something happened.

Study the fragment as a miniature version of the same syntax used by the backend.

```
process.emit('omb-builder-update-started', {
  installerPath,
  launcherPath,
  updateLogPath
});
```

emit names an event and passes an object payload. Other code can listen for that event if it needs to react.

The update launcher uses this to announce that the external update handoff has started.

The trap is assuming emitted events are stored forever. If nobody listens at the time, the event is simply emitted and gone.

When reading events, find both the emit side and any listener side.

1.51.47 Reading endpoint fallthrough

This reading lab focuses on how a function tells the caller it did not handle a route.

Study the fragment as a miniature version of the same syntax used by the backend.

```
if (request.method !== 'POST') return false;
return false;
```

Returning false means the API switchboard did not handle the request. The outer server can then continue with another response path.

handleApi uses true for handled routes and false when the path is not one of its endpoints.

The trap is forgetting to return after handling a route. Fallthrough can accidentally trigger a second response.

When reading a router, track which branches return true, which return false, and which send a response.

1.52 Reading Real server.mjs Syntax In Context

This final JavaScript section uses real pieces of server.mjs. The purpose is still syntax, not feature documentation. The feature chapters later explain why each behavior exists. Here, the goal is to practice reading realistic JavaScript without turning the explanation into reference cards.

Notice the rhythm: imports bring tools in, constants define shared state, objects and arrays encode rulebooks, functions name repeatable behavior, async helpers wait for slow work, and route handlers translate browser actions into backend operations. That rhythm appears across the whole file.

1.52.1 The import block shows the file's operating powers

Every import line is a clue. HTTP support means the file can listen for requests. `child_process` means it can run external programs. `fs/promises` means it can touch files asynchronously. `path` and `url` mean it converts between operating-system paths and request addresses.

The named imports in braces are individual tools taken from a module. The default import path is a local name for a module object. A new JavaScript reader should read this area as the backend's tool inventory.

The excerpt below is read here for JavaScript syntax. The later feature chapters explain the broader behavior.

```
import { createServer } from "node:http";
import { execFile, spawn } from "node:child process";
import { createHash } from "node:crypto";
import { access as fsAccess, chmod, cp, mkdir, mkdtemp, readFile, readdir, rm, writeFile } from
  "node:fs/promises";
import os from "node:os";
import path from "node:path";
import { promisify } from "node:util";
import { fileURLToPath } from "node:url";
```

1.52.2 Startup constants show conversion, fallbacks, and top-level await

This block is a compact lesson in module-level state. The root path is calculated from `import.meta.url`, workspace roots are resolved with path helpers, port is converted from environment text into a number, and `package.json` is read with top-level `await`.

The syntax is doing practical work. It lets the same backend run under Electron, PowerShell, a packaged app, or a development shell without hardcoding every path.

The excerpt below is read here for JavaScript syntax. The later feature chapters explain the broader behavior.

```
const root = path.dirname(fileURLToPath(import.meta.url));
const portfolioRoot = path.resolve(process.env.OMB PORTFOLIO WORKSPACE || root);
const compileRoot = path.resolve(process.env.OMB CODE WORKSPACE || path.join(path.dirname(root), "compile-code"));
const port = Number(process.env.PORT || 8080);
const host = process.env.HOST || "0.0.0.0";
const execFileAsync = promisify(execFile);
const packageJson = JSON.parse(await readFile(path.join(root, "package.json"), "utf8").catch(() => "{}"));
```

1.52.3 MIME types show object lookup syntax

The types object is not a class or a database. It is a lookup table. A file extension is used as the key, and the server receives a content type as the answer.

The syntax also shows quoted property names. Because keys such as `.html` and `.js` contain a dot, they need quotes inside the object literal.

The excerpt below is read here for JavaScript syntax. The later feature chapters explain the broader behavior.

```
const types = {
  ".css": "text/css",
  ".csv": "text/csv",
  ".html": "text/html",
  ".ico": "image/x-icon",
  ".js": "application/javascript",
  ".json": "application/json",
  ".md": "text/markdown",
  ".pdf": "application/pdf",
  ".png": "image/png",
  ".jpg": "image/jpeg",
```

```

    ".ipeg": "image/ipeg".
    ".svg": "image/svg+xml".
    ".txt": "text/plain".
    ".webp": "image/webp".
    ".xml": "application/xml".
    ".xlsx": "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet".
    ".zin": "application/zin"
  };

```

1.52.4 Publish paths show arrays as safety policy

The publishPaths array is ordinary JavaScript syntax, but the meaning is security-sensitive. Each string is a path the publisher is allowed to copy into the website mirror.

A beginner should learn to ask what a list controls. Some arrays are mere display order. This one is an allowlist that decides what can become public.

The excerpt below is read here for JavaScript syntax. The later feature chapters explain the broader behavior.

```

const publishPaths = [
  "projects.json".
  "docs".
  "assets".
  "index.html".
  "styles.css".
  "script.js".
  "electronics-search.js".
  "Backgrounds".
  ".noiekvll".
  " headers".
  ".well-known".
  "robots.txt".
  "sitemap.xml".
  "CNAME"
];

```

1.52.5 Compiler language profiles show nested object design

This is the longest object in the syntax primer because it demonstrates nested data. The first level uses language keys. Each language value is another object containing defaultFile, extensions, label, primaryTools, and winget.

Reading nested objects becomes easier when each level is named in English. The outer object answers which language is selected. The inner object answers what that language needs.

The excerpt below is read here for JavaScript syntax. The later feature chapters explain the broader behavior.

```

const compileLanguageProfiles = {
  c: {
    defaultFile: "main.c".
    extensions: [".c". ".h"].
    label: "C".
    primaryTools: ["gcc"].
    winget: ["BrechtSanders.WinLibs.POSIX.UCRT"]
  }.
  cpp: {
    defaultFile: "main.cpp".
    extensions: [".cpp". ".cc". ".cxx". ".hpp". ".h"].
    label: "C++".
    primaryTools: ["g++"].
    winget: ["BrechtSanders.WinLibs.POSIX.UCRT"]
  }.
  verilog: {
    defaultFile: "design.v".
    extensions: [".v"].
    label: "Verilog".
    primaryTools: ["iverilog". "vvp"].
    winget: ["Icarus.Verilog"]
  }.
  systemverilog: {
    defaultFile: "design.sv".
    extensions: [".sv". ".svh"],

```

```

    label: "SvstemVerilog".
    primaryTools: ["iverilog". "vvn"].
    winget: ["Icarus.Verilog"]
  },
  ltspice: {
    defaultFile: "simulation.cir".
    extensions: [".cir". ".net". ".sn". ".asc"].
    label: "LTspice".
    primaryTools: ["ltspice"].
    winget: []
  },
  iava: {
    defaultFile: "Main.iava".
    extensions: [".iava"].
    label: "Java".
    primaryTools: ["iavac". "iava"].
    winget: ["EclipseAdontium.Temurin.21.JDK"]
  },
  iavascript: {
    defaultFile: "main.is".
    extensions: [".is". ".mis". ".cis"].
    label: "JavaScript".
    primaryTools: ["node"].
    winget: []
  },
  pvthon: {
    defaultFile: "main.pv".
    extensions: [".pv"].
    label: "Pvthon".
    primaryTools: ["pvthon"].
    winget: []
  },
  html: {
    defaultFile: "index.html".
    extensions: [".html". ".htm"].
    label: "HTML".
    primaryTools: [].
    winget: []
  }
};

```

1.52.6 Compiler tool candidates show arrays inside objects

This object is a second rulebook. Instead of language names, the keys are executable tool names. The values are arrays of possible command names or Windows paths.

The syntax combines environment variables, conditional expressions, path.join calls, string literals, and arrays. It looks dense because it is encoding practical Windows discovery behavior.

The excerpt below is read here for JavaScript syntax. The later feature chapters explain the broader behavior.

```

const compileToolCandidates = {
  gcc: [
    "gcc".
    process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Microsoft". "WinGet". "Packages".
      "BrechtSanders.WinLibs.POSIX.UCRT Microsoft.Winget.Source 8wekvb3d8bbwe". "mingw64". "bin". "gcc.exe") :
    ""
  ],
  "g++": [
    "g++".
    "c++".
    process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Microsoft". "WinGet". "Packages".
      "BrechtSanders.WinLibs.POSIX.UCRT Microsoft.Winget.Source 8wekvb3d8bbwe". "mingw64". "bin". "g++.exe") :
    ""
  ],
  clang: ["clang"].
  "clang++": ["clang++"].
  cl: ["cl"].
  iavac: [
    "iavac".
    "C:\\Program Files\\Eclipse Adontium\\jdk-21.0.11.10-hotspot\\bin\\iavac.exe"
  ],
  iava: [
    "java".
    "C:\\Program Files\\Eclipse Adoptium\\jdk-21.0.11.10-hotspot\\bin\\java.exe"
  ]
};

```

```

1.
node: [
  /(?:^[\\\/\|]node(?:\.exe)?$/i.test(process.execPath || "") ? process.execPath : "",
  "node",
  "C:\\Program Files\\nodejs\\node.exe",
  process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Programs", "nodejs", "node.exe") : ""
]
1.
python: [process.env.PYTHON, "python", "py"],
iverilog: ["iverilog", "C:\\iverilog\\bin\\iverilog.exe"],
vvp: ["vvp", "C:\\iverilog\\bin\\vvp.exe"],
ltspice: [
  process.env.LTSPICE_EXE,
  "C:\\Program Files\\ADI\\LTspice\\LTspice.exe",
  "C:\\Program Files\\LTC\\LTspiceXVII\\XVIIx64.exe",
  "C:\\Program Files\\Analog Devices\\LTspice\\LTspice.exe",
  process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Programs", "ADI", "LTspice", "LTspice.exe") :
  ""
]
1
};

```

1.52.7 AI instructions show strings as application logic

portfolioAiInstructions is written as an array of strings and joined with newline characters. This keeps a long prompt readable in source form while still sending one coherent instruction block to the model.

For an AI feature, strings are not filler. They define behavior: answer style, safety limits, when to use portfolio context, when to answer generally, and how to handle public GitHub code.

The excerpt below is read here for JavaScript syntax. The later feature chapters explain the broader behavior.

```

const portfolioAiInstructions = [
  "You are the AI assistant for the portfolio website described by the supplied portfolio context.",
  "Behave like a careful senior electrical and computer engineering mentor who can also navigate the saved portfolio.",
  "Answer the visitor's question first in a concise ChatGPT-like format. then add portfolio links or context only when they help.",
  "Use the supplied question intent to decide whether this is a general engineering question or a portfolio-specific question.",
  "For general conversation intent, respond naturally and briefly. Use the recent conversation instead of a fixed template. If the visitor says hi, a good answer is a short greeting such as: Hi, what can I do for you?",
  "If the visitor asks 'what is my name?' or 'who am I?' and they have not identified themselves, do not pretend to know the visitor. Say you are an AI agent for this portfolio and identify the portfolio owner only if the supplied profile context includes a name.",
  "For general knowledge intent, answer the question directly using broad general knowledge. Do not force portfolio context unless the visitor asks to connect the answer to the portfolio owner's work.",
  "For general engineering intent, begin with the general electronics or engineering explanation. Do not lead with project context unless the visitor asks to connect it.",
  "For portfolio specific intent, answer from the portfolio context first, then explain related engineering concepts only when useful.",
  "Use recent conversation history for follow-up questions, pronouns, comparisons, and corrections.",
  "Use the supplied portfolio context as the trusted source for the portfolio owner's projects, links, files, resume, and contact information.",
  "Use sourceExcerpts when present as higher-detail evidence from uploaded text files, extracted resume text, same-site files, GitHub pages, or other safe public sources.",
  "Use knowledgeManifest to understand project files, image evidence, public profiles, resumes, and project areas. Treat filenames, captions, surrounding text, and descriptions as evidence.",
  "Do not claim to visually inspect an image unless actual image analysis is provided. If only image metadata is supplied, say what the caption/path/context suggests.",
  "For GitHub, LinkedIn, resume, or uploaded-file questions, cite what is present in the supplied context or fetched excerpts and then point to the link when useful. Access public pages and fetched excerpts when the website allows it.",
  "When a visitor asks for source code, use public GitHub source excerpts when provided. Show concise relevant snippets with file paths, and explain what the code is doing. Do not imply private repository access.",
  "If sourceExcerpts include lines labeled Source file, include at least one fenced code block with the file path immediately before it, unless the visitor explicitly asks for links only.",
  "Never invent, infer, or write hypothetical code for the portfolio owner's repositories. Every fenced code block about repository work must be copied from a fetched Source file excerpt. If no source file excerpt is available, say that the code was not available in the fetched public sources and give the repository link instead.",
  "For questions about public GitHub repositories, prefer fetched public GitHub source excerpts over portfolio summaries or project descriptions. You may fetch and display concise code from public GitHub URLs supplied in the portfolio context or safe source fetches, but never imply access to private repositories.",
  "You may answer related electronics, hardware, analog, mixed-signal, digital, embedded, FPGA, ASIC, PCB, and

```

```

firmware questions even when the answer is broader than the saved portfolio.".
"Do not invent portfolio projects. credentials. emnlovers. files. or test results that are not in the context.".
"If context is missing. sav what is missing and answer generally only for the engineering concept.".
"When useful. state assumptions. define terms. exolain signal or data flow. identifv tradeoffs. and name what
evidence would prove the claim.".
"Only mention or recommend portfolio links that are directly relevant to the visitor's current question or the
active follow-up context. Do not append random links.".
"Keep the answer recruiter-friendly. specific. and easv to skim. Use short paragraphs and bullets when
helpful.".
"Do not expose chain-of-thought. Give the polished answer only."
].join("\n");

```

1.52.8 securityHeaders and sendJson show helper composition

These helpers show how small functions compose. securityHeaders creates the shared header object. sendJson calls securityHeaders, adds JSON-specific headers, serializes data, and ends the response.

The syntax worth noticing is the object spread, the nested object passed into writeHead, and JSON.stringify with indentation arguments.

The excerpt below is read here for JavaScript syntax. The later feature chapters explain the broader behavior.

```

function securityHeaders(extra = {})

function sendJson(response, status, data) {
  response.writeHead(status, {
    ...securityHeaders(),
    "Content-Type": "application/json",
    "Cache-Control": "no-store, no-cache, must-revalidate"
  });
  response.end(JSON.stringify(data, null, 2));
}

```

1.52.9 readRequestJson shows async stream reading

This function contains several beginner-important ideas at once: an async function, a mutable accumulator, a for await loop over request chunks, a size guard, and JSON parsing at the end.

The code is not merely procedural. It is a boundary pattern: receive text slowly, limit it, parse it, and let later routes validate the shape.

The excerpt below is read here for JavaScript syntax. The later feature chapters explain the broader behavior.

```

async function readRequestJson(request) {
  const chunks = [];
  let size = 0;

  for await (const chunk of request) {
    size += chunk.length;
    if (size > 60 * 1024 * 1024) {
      throw new Error("Request body is too large.");
    }
    chunks.push(chunk);
  }

  return JSON.parse(Buffer.concat(chunks).toString("utf8") || "{}").replace(/\uFEFF/g, "");
}

```

1.52.10 normalizeCodeLanguage shows aliases and lookup fallback

The aliases object maps many human spellings to one internal language key. The return line shows a common JavaScript pattern: try a direct alias, then try a profile lookup, then fall back to JavaScript.

This is a good example of making the interface forgiving while keeping backend behavior strict.

The excerpt below is read here for JavaScript syntax. The later feature chapters explain the broader behavior.

```
function normalizeCodeLanguage(value = "") {
  const clean = String(value || "").trim().toLowerCase().replace(/[\s-]+/g, "");
  const aliases = {
    c: "c",
    cpp: "cpp",
    colusplus: "cpp",
    "c++": "cpp",
    svstemverilog: "systemverilog",
    sv: "systemverilog",
    verilog: "verilog",
    v: "verilog",
    ltsnice: "ltsnice",
    spice: "ltsnice",
    cir: "ltsnice",
    iava: "java",
    iavascript: "javascript",
    is: "javascript",
    node: "javascript",
    pvthon: "python",
    pv: "python",
    html: "html",
    htm: "html"
  };
  return aliases[clean] || (compileLanguageProfiles[clean] ? clean : "javascript");
}
```

1.52.11 findTool shows cache-first async searching

findTool starts with a cache check, then walks through candidates, then asks the operating system where a command exists, then performs special Windows searches for common compilers.

The function demonstrates async control flow in a practical setting: each awaited operation may fail, so the code catches failures and moves to the next candidate rather than ending the entire search.

The excerpt below is read here for JavaScript syntax. The later feature chapters explain the broader behavior.

```
async function findTool(toolName) {
  if (compileToolCache.has(toolName)) return compileToolCache.get(toolName);
  const candidates = (compileToolCandidates[toolName] || [toolName]).filter(Boolean);
  for (const candidate of candidates) {
    if (path.isAbsolute(candidate)) {
      if (await pathExists(candidate)) {
        compileToolCache.set(toolName, candidate);
        return candidate;
      }
      continue;
    }
    try {
      const command = process.platform === "win32" ? "where" : "command";
      const args = process.platform === "win32" ? [candidate] : ["-v", candidate];
      const result = await execFileAsync(command, args, { timeout: 5000, windowsHide: true });
      const found = String(result.stdout || "").split(/\r?\n/).map((line) => line.trim()).find(Boolean);
      if (found) {
        compileToolCache.set(toolName, found);
        return found;
      }
    } catch {
      // Try the next candidate.
    }
  }

  if ([ "gcc", "g++" ].includes(toolName) && process.env.LOCALAPPDATA) {
    const found = await findExecutableUnder(
      path.join(process.env.LOCALAPPDATA, "Microsoft", "WinGet", "Packages"),
      [toolName === "gcc" ? "gcc.exe" : "g++.exe"]
    );
    compileToolCache.set(toolName, found || "");
    return found || "";
  }

  if ([ "javac", "java" ].includes(toolName)) {
    const found = await findExecutableUnder("C:\\Program Files\\Eclipse Adoptium", [`${toolName}.exe`], 5);
  }
}
```

```

    compileToolCache.set(toolName, found || "");
    return found || "";
}

compileToolCache.set(toolName, "");
return "";
}

```

1.52.12 runProcess shows child-process supervision

This source excerpt shows the syntax of process control: spawn starts a child, event callbacks collect stdout and stderr, a timeout protects the app from hangs, and the final result object normalizes the outcome.

A new JavaScript reader should notice how many small language ideas combine here. There are objects, arrays, callbacks, timers, promises, string accumulation, and cleanup paths all serving one feature: terminal-style output.

The excerpt below is read here for JavaScript syntax. The later feature chapters explain the broader behavior.

```

async function runProcess(command, args = [], options = {})

```

1.52.13 handlePortfolioAi shows a complete async decision path

This excerpt is a realistic async workflow. The function reads JSON, validates the question, cleans conversation history, enriches context, chooses between Ollama and OpenAI, builds a payload, performs fetch, handles fallback, and returns JSON.

The syntax is not random. Each if statement protects a stage, each await waits for a slow boundary, each object literal packages data for another API, and each return prevents the function from falling into the wrong provider path.

The excerpt below is read here for JavaScript syntax. The later feature chapters explain the broader behavior.

```

async function handlePortfolioAi(request, response) {
  const body = await readRequestJson(request);
  const question = clampText(body.question, 1200).trim();
  const conversation = cleanConversationHistory(body.conversation);
  const validIntents = new Set(["general conversation", "general engineering", "general knowledge",
    "portfolio specific"]);
  const intent = validIntents.has(body.intent) ? body.intent : "portfolio specific";
  const allowWebSearch = body.allowWebSearch === true;

  if (!question) {
    sendJson(response, 400, { error: "Question is required." });
    return;
  }

  const context = await enrichPortfolioContext({ ...(body.context || {}), question });

  const apiKey = process.env.OPENAI_API_KEY || "";
  if (!apiKey) {
    const ollama = await callOllamaPortfolioAi({ question, intent, conversation, context, allowWebSearch });
    if (ollama.ok) {
      sendJson(response, 200, {
        answer: ollama.answer,
        model: ollama.model,
        provider: "ollama",
        usedWebSearch: false
      });
      return;
    }
    sendJson(response, 503, { error: ollama.error || "No local Ollama model or OPENAI API KEY is available for the local backend." });
    return;
  }

  const model = process.env.OPENAI_MODEL || "gpt-5.4";
  const fallbackModel = process.env.OPENAI_FALLBACK_MODEL || "gpt-4.1";
  const webSearchMode = String(process.env.OPENAI_ENABLE_WEB_SEARCH || "auto").toLowerCase();

```

```

const enableWebSearch = webSearchMode === "true" || (webSearchMode !== "false" && allowWebSearch);
const buildPayload = (selectedModel) => ({
  model: selectedModel,
  input: [
    {
      role: "developer",
      content: [{ tvne: "input text", text: portfolioAiInstructions }]
    },
    {
      role: "user",
      content: [{
        tvne: "input text",
        text: [
          `Visitor question: ${question}`,
          `Question intent: ${intent}`,
          `Web search allowed for this question: ${enableWebSearch ? "yes" : "no"}`,
          "",
          "Recent conversation JSON:",
          clampText(JSON.stringify(conversation, null, 2), 8000),
          "",
          "Portfolio context JSON:",
          clampText(JSON.stringify(context, null, 2), 18000)
        ].join("\n")
      ]
    }
  ],
  max output tokens: Number(process.env.OPENAI_MAX_OUTPUT_TOKENS || 1100),
  reasoning: { effort: process.env.OPENAI_REASONING_EFFORT || "medium" },
  text: { verbosity: process.env.OPENAI_VERBOSITY || "medium" }
});

const callModel = async (selectedModel) => {
  const payload = buildPayload(selectedModel);
  if (enableWebSearch) {
    payload.tools = [{ tvne: "web search", search context size: process.env.OPENAI_WEB_SEARCH_CONTEXT_SIZE || "low" }];
  }

  const openAiResponse = await fetch("https://api.openai.com/v1/responses", {
    method: "POST",
    headers: {
      "Authorization": `Bearer ${apiKey}`,
      "Content-Type": "application/json"
    },
    body: JSON.stringify(payload)
  });

  const data = await openAiResponse.json().catch(() => ({}));
  return { data, openAiResponse, selectedModel };
};

let { data, openAiResponse, selectedModel } = await callModel(model);
if (!openAiResponse.ok && !process.env.OPENAI_MODEL && fallbackModel && fallbackModel !== model && [400, 404].includes(openAiResponse.status)) {
  ({ data, openAiResponse, selectedModel } = await callModel(fallbackModel));
}
if (!openAiResponse.ok) {
  sendJson(response, openAiResponse.status, {
    error: data?.error?.message || "OpenAI request failed."
  });
  return;
}

sendJson(response, 200, {
  answer: extractOpenAiText(data),
  model: selectedModel,
  usedWebSearch: enableWebSearch
});
}

```

1.53 Beginner Syntax Checkpoints Before Reading The Server Features

Before leaving the JavaScript primer, it helps to name the reading habits that make server.mjs approachable. These are not abstract rules. They are the mental checkpoints that prevent a beginner

from getting lost when a real backend function combines several syntax ideas at once.

1.53.1 Separate the data shape from the action

When a function creates an object, array, or string, it is usually shaping information. When it calls `writeFile`, `spawn`, `fetch`, `sendJson`, or `runPublishGit`, it is doing something with the outside world. Read the data shape first, then read the side effect.

This checkpoint is shown with a small fragment so the habit has something concrete to attach to.

```
const payload = { ok: true, file: 'projects.json' };
sendJson(response, 200, payload);
```

1.53.2 Find the guard before the feature body

Most safe backend features begin by refusing bad inputs. A guard clause may reject a missing question, a malformed catalog, a non-local request, or an unauthenticated publishing target. Once the guard returns, the rest of the function can assume a cleaner starting point.

This checkpoint is shown with a small fragment so the habit has something concrete to attach to.

```
if (!Array.isArray(catalog.projects)) {
  sendJson(response, 400, { error: 'Projects must be an array.' });
  return true;
}
```

1.53.3 Track async boundaries

Every `await` marks a boundary where JavaScript is waiting for something outside the immediate function body: a file, a process, a network response, or a Git command. When debugging, the most useful question is often what promise is being awaited and what can fail there.

This checkpoint is shown with a small fragment so the habit has something concrete to attach to.

```
const body = await readRequestJson(request);
const status = await runProcess(command, args, options);
```

1.53.4 Treat strings differently depending on purpose

A string can be display text, a filename, a URL, a Git command argument, a model prompt, or source code. The same JavaScript type can require different validation depending on where it will be used.

This checkpoint is shown with a small fragment so the habit has something concrete to attach to.

```
const title = String(body.title || '').trim();
const fileName = safeFileName(body.fileName);
```

1.53.5 Read object literals as contracts

When an endpoint returns an object, that object becomes a contract with the frontend. Fields such as `ok`, `error`, `result`, `stdout`, `publish`, and `target` are not random. They determine what the user interface can display after the request completes.

This checkpoint is shown with a small fragment so the habit has something concrete to attach to.

```
return {
  ok: result.code === 0,
  stdout: result.stdout,
  stderr: result.stderr
};
```

1.53.6 Notice where helpers prevent repeated mistakes

Helpers such as `sendJson`, `resolveInsideRoot`, `normalizeCodeLanguage`, and `runProcess` exist because the same mistake would be dangerous if repeated differently across endpoints. A helper is often the place where a rule becomes enforceable.

This checkpoint is shown with a small fragment so the habit has something concrete to attach to.

```
const filePath = resolveInsideRoot('docs', projectId, section, fileName):
sendJson(response, 200, { ok: true, url });
```

1.53.7 Read route code from the user's click

The cleanest way to read `handleApi` is to start with a button or panel in the app. Save draft, Apply to site, Authenticate target, Load from target, Compile, Simulate, Upload, Update, and Ask My Portfolio each map to a route branch.

This checkpoint is shown with a small fragment so the habit has something concrete to attach to.

```
if (url.pathname === '/api/code/compile') {
  const result = await compileAndRunCode(bodv);
  sendJson(response, 200, { ok: result.ok, result });
}
```

1.53.8 Watch for values that cross trust boundaries

A value crosses a trust boundary when it moves from browser input into filesystem paths, command arguments, Git operations, or AI prompts. Those crossings are where JavaScript syntax becomes security design.

This checkpoint is shown with a small fragment so the habit has something concrete to attach to.

```
const projectId = safeSegment(bodv.projectId, 'project');
const folder = resolveInsideRoot('docs', projectId);
```

1.53.9 Prefer a boring return shape

Exciting code is usually bad backend code. A good result object is boring, predictable, and easy for the UI to render. The reader should not need to guess whether a function returns text, an object, null, or throws for normal outcomes.

This checkpoint is shown with a small fragment so the habit has something concrete to attach to.

```
return {
  ok: true,
  artifacts,
  terminal: processTerminalText(result)
};
```

1.53.10 Use comments only when names are not enough

A beginner may want to comment every line, but a mature backend relies on names, structure, and small helpers first. Comments should explain why a fallback exists, why a failure is ignored, or why a Windows-specific path is being searched.

This checkpoint is shown with a small fragment so the habit has something concrete to attach to.

```
// Trv the next candidate because Windows PATH often misses tool installs.
continue;
```

Chapter 2: What server.mjs Is Responsible For

The local backend as the builder's controlled workshop

server.mjs is the local backend that gives the portfolio builder powers a normal webpage should not have by itself. The frontend can collect text, show project windows, and let a user click buttons. The backend is the controlled workshop that can write files, run compilers, launch Git, start installers, and call private AI providers.

The file is intentionally not a tiny web server. It is a local application service. That design is reasonable because the builder is local-first: the user can work offline, save drafts locally, attach files, preview the website, and publish only after the target repository is authenticated.

A helpful way to read the file is to separate the question 'what does this function do?' from the larger question 'which user feature depends on this behavior?' The second question is usually more useful. The Save Draft button, Apply to site button, Compile Code tool, Ask My Portfolio panel, and Update dialog all travel through server.mjs.

How root shapes this workflow

root is the anchor for application files. Every static file request and many local writes begin with this value, so the server can behave consistently no matter which shell or packaged runtime started it.

The value is derived from import.meta.url instead of the current terminal folder. That prevents a common desktop-app bug where launching from a shortcut changes the process working directory and breaks file lookup.

The code below is the actual shared value used by this feature group.

```
const root = path.dirname(fileURLToPath(import.meta.url));
```

How portfolioRoot shapes this workflow

portfolioRoot is the publish mirror boundary. In a separated workspace installation, it lets the builder application and public website output live in different folders while still cooperating during Apply to site.

Publishing functions should use this root when they mean public website files. Compile functions should not use it, because generated binaries and simulations do not belong in the public mirror.

The code below is the actual shared value used by this feature group.

```
const portfolioRoot = path.resolve(process.env.OMB_PORTFOLIO_WORKSPACE || root);
```

How compileRoot shapes this workflow

compileRoot is the workspace for code that may be compiled, built, run, or simulated. Keeping it separate protects the portfolio from temporary binaries, class files, VCD waveforms, and run folders.

The separation also makes future containerization easier. A Docker-based compile service can mount the compile workspace without needing access to every website asset.

The code below is the actual shared value used by this feature group.

```
const compileRoot = path.resolve(process.env.OMB_CODE_WORKSPACE || path.join(path.dirname(root), "compile-code"));
```

How port shapes this workflow

port is the numeric doorway used by the local HTTP server. Environment variables arrive as text, so `Number(...)` converts the configured value into the number that listen expects.

The fallback keeps the builder predictable on a normal machine. If nothing overrides `PORT`, the local service starts on 8080 and the app knows what address to open.

The code below is the actual shared value used by this feature group.

```
const port = Number(process.env.PORT || 8080);
```

How host shapes this workflow

host controls which network interfaces the local preview binds to. The default `0.0.0.0` allows the server to listen broadly, while the later local-only API checks decide which privileged actions are actually allowed.

That distinction is important. A preview can be reachable for testing while compiler, publishing, update, and security routes still refuse non-local requests.

The code below is the actual shared value used by this feature group.

```
const host = process.env.HOST || "0.0.0.0";
```

How publishPaths shapes this workflow

publishPaths is a public-output allowlist. It tells the publisher which files and folders are allowed to become part of the live website.

This list is one of the most important safety objects in the backend. Without it, a broad git add could accidentally publish drafts, credentials, compile outputs, local caches, or builder-only implementation files.

The code below is the actual shared value used by this feature group.

```
const publishPaths = [
  "projects.json",
  "docs",
  "assets",
  "index.html",
  "styles.css",
  "script.js",
  "electronics-search.js",
  "Backgrounds",
  ".nojekyll",
  "headers",
  ".well-known",
  "robots.txt",
  "sitemap.xml",
  "CNAME"
];
```

The startup shape

The startup code creates the actual HTTP server. Every request enters through the same front door. If the path begins with `/api`, the request is handed to the API switchboard. Otherwise, the server treats the request as a static file request and tries to serve a file from the local app root.

The branch structure is deliberate. API requests can change state, so they receive deeper validation. Static file requests should only read files, and even then the resolved file path must stay inside the app root.

The final listen call binds the server to a host and port, then prints a preview URL. That printed message is why a developer can tell where the local preview is running.

How types shapes this workflow

types maps file extensions to HTTP content types. Static serving uses it to make browsers interpret files correctly instead of guessing.

The code below is the actual shared value used by this feature group.

```
const tvpes = {
  ".css": "text/css",
  ".csv": "text/csv",
  ".html": "text/html",
  ".ico": "image/x-icon",
  ".js": "application/javascript",
  ".json": "application/json",
  ".md": "text/markdown",
  ".ndf": "application/ndf",
  ".png": "image/png",
  ".jpg": "image/jpeg",
  ".jpeg": "image/jpeg",
  ".svg": "image/svg+xml",
  ".txt": "text/plain",
  ".webp": "image/webp",
  ".xml": "application/xml",
  ".xlsx": "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
  ".zip": "application/zip"
};
```

How __server_startup__ works inside this feature

The server startup block is the point where the file becomes a running local service. The createServer callback receives every browser request, parses the requested URL, and decides whether the request belongs to the API router or to static file serving.

This is the implementation at the point where the feature needs it.

```
createServer(async (request, response) => {
  try {
    const url = new URL(request.url || "/", `http://${request.headers.host}`);
    if (url.pathname.startsWith("/api/") && await handleApi(request, response, url)) return;

    const pathname = url.pathname === "/" ? "/index.html" : decodeURIComponent(url.pathname);
    const filePath = path.normalize(path.join(root, pathname));

    if (!filePath.startsWith(root)) {
      response.writeHead(403);
      response.end("Forbidden");
      return;
    }

    const body = await readFile(filePath);
    response.writeHead(200, {
      ...securityHeaders(),
      "Content-Type": tvpes[path.extname(filePath)] || "application/octet-stream",
      "Cache-Control": "no-store, no-cache, must-revalidate"
    });
    response.end(body);
  } catch {
    response.writeHead(404);
    response.end("Not found");
  }
}).listen(port, host, () => {
  console.log(`Portfolio preview running at http://localhost:${port}`);
  console.log("For phone access, use this computer's Wi-Fi/LAN IP address with the same port.");
});
```

The first meaningful branch checks for /api paths. When handleApi returns true, the request has already been answered. When it returns false, the code continues into the file-serving path. This is a simple but effective split between commands and assets: API routes are verbs, while static paths are files.

The pathname logic gives the browser index.html for the root URL and decodes other paths before joining them to root. The important protection is the startsWith check after normalization. That check prevents a crafted URL from escaping the application folder and reading unrelated local files.

After the file is read, the response attaches shared security headers, a content type from the types table, and no-store caching. The no-store rule keeps the builder honest while the user is editing; the browser should not keep showing old JavaScript, stale JSON, or outdated images after a save.

The final listen call binds the local service to host and port. The console messages are mostly developer-facing, but they also tell us the intended operating model: the same backend can serve desktop preview traffic and LAN phone-preview traffic, while privileged API routes still rely on local-request checks.

The startup constants as the server's first contract

Before a visitor ever clicks a button, the backend has to decide where it is running, where the portfolio workspace lives, where compile artifacts belong, which port to listen on, and which host address should be bound. These values are not decorative configuration. They become the hidden contract that every later file, compile, upload, publish, and update operation relies on.

The useful way to read this group is as the server's startup map. root tells the backend where the application code lives. portfolioRoot tells publishing where the website mirror lives. compileRoot keeps runnable code projects away from public site files. port and host decide how the local HTTP service becomes reachable. packageJson carries the application version into update checks. If any of those values point to the wrong place, later code may still look correct while operating on the wrong folder or reporting the wrong version.

How root shapes this workflow

root is the anchor for application files. Every static file request and many local writes begin with this value, so the server can behave consistently no matter which shell or packaged runtime started it.

The value is derived from import.meta.url instead of the current terminal folder. That prevents a common desktop-app bug where launching from a shortcut changes the process working directory and breaks file lookup.

The code below is the actual shared value used by this feature group.

```
const root = path.dirname(fileURLToPath(import.meta.url));
```

How portfolioRoot shapes this workflow

portfolioRoot is the publish mirror boundary. In a separated workspace installation, it lets the builder application and public website output live in different folders while still cooperating during Apply to site.

Publishing functions should use this root when they mean public website files. Compile functions should not use it, because generated binaries and simulations do not belong in the public mirror.

The code below is the actual shared value used by this feature group.

```
const portfolioRoot = path.resolve(process.env.OMB_PORTFOLIO_WORKSPACE || root);
```

How compileRoot shapes this workflow

compileRoot is the workspace for code that may be compiled, built, run, or simulated. Keeping it separate protects the portfolio from temporary binaries, class files, VCD waveforms, and run folders.

The separation also makes future containerization easier. A Docker-based compile service can mount the compile workspace without needing access to every website asset.

The code below is the actual shared value used by this feature group.

```
const compileRoot = path.resolve(process.env.OMB_CODE_WORKSPACE || path.join(path.dirname(root), "compile-code"));
```

How port shapes this workflow

port is the numeric doorway used by the local HTTP server. Environment variables arrive as text, so Number(...) converts the configured value into the number that listen expects.

The fallback keeps the builder predictable on a normal machine. If nothing overrides PORT, the local service starts on 8080 and the app knows what address to open.

The code below is the actual shared value used by this feature group.

```
const port = Number(process.env.PORT || 8080);
```

How host shapes this workflow

host controls which network interfaces the local preview binds to. The default 0.0.0.0 allows the server to listen broadly, while the later local-only API checks decide which privileged actions are actually allowed.

That distinction is important. A preview can be reachable for testing while compiler, publishing, update, and security routes still refuse non-local requests.

The code below is the actual shared value used by this feature group.

```
const host = process.env.HOST || "0.0.0.0";
```

How packageJson shapes this workflow

packageJson carries the builder's package metadata into runtime code. The updater uses its version field to decide whether the installed app is current or behind a release.

The code below is the actual shared value used by this feature group.

```
const packageJson = JSON.parse(await readFile(path.join(root, "package.json"), "utf8").catch(() => "{}"));
```

Static file serving after the API path says no

The final createServer block has two jobs. First, it gives /api requests to handleApi. Second, it serves static files such as index.html, styles.css, script.js, icons, images, PDFs, and JSON when the request is not an API command. This is the part that makes the local preview feel like a website instead of a loose folder of files.

The serving path still has a boundary check. The request URL is decoded and joined to the root, but the final file path must still begin inside root. That protects the server from URL paths that try to climb outside the app folder. After the file is read, the extension is matched against the types object so the browser knows whether it is receiving HTML, CSS, JavaScript, JSON, an image, a PDF, or a generic binary stream.

How types shapes this workflow

types maps file extensions to HTTP content types. Static serving uses it to make browsers interpret files correctly instead of guessing.

The code below is the actual shared value used by this feature group.

```
const types = {
  ".css": "text/css",
  ".csv": "text/csv",
  ".html": "text/html",
  ".ico": "image/x-icon",
  ".js": "application/javascript",
  ".json": "application/json",
  ".md": "text/markdown",
  ".pdf": "application/pdf",
  ".png": "image/png",
  ".svg": "image/svg+xml",
  ".txt": "text/plain",
  ".webp": "image/webp",
  ".xml": "application/xml",
  ".xlsx": "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
  ".zip": "application/zip"
};
```

How `__server_startup__` works inside this feature

The server startup block is the point where the file becomes a running local service. The `createServer` callback receives every browser request, parses the requested URL, and decides whether the request belongs to the API router or to static file serving.

This is the implementation at the point where the feature needs it.

```
createServer(async (request, response) => {
  try {
    const url = new URL(request.url || "/", `http://${request.headers.host}`);
    if (url.pathname.startsWith("/api/") && await handleApi(request, response, url)) return;

    const pathname = url.pathname === "/" ? "/index.html" : decodeURIComponent(url.pathname);
    const filePath = path.normalize(path.join(root, pathname));

    if (!filePath.startsWith(root)) {
      response.writeHead(403);
      response.end("Forbidden");
      return;
    }

    const body = await readFile(filePath);
    response.writeHead(200, {
      ...securityHeaders(),
      "Content-Type": types[path.extname(filePath)] || "application/octet-stream",
      "Cache-Control": "no-store, no-cache, must-revalidate"
    });
    response.end(body);
  } catch {
    response.writeHead(404);
    response.end("Not found");
  }
}).listen(port, host, () => {
  console.log(`Portfolio preview running at http://localhost:${port}`);
  console.log("For phone access, use this computer's Wi-Fi/LAN IP address with the same port.");
});
```

The first meaningful branch checks for `/api` paths. When `handleApi` returns true, the request has already been answered. When it returns false, the code continues into the file-serving path. This is a simple but effective split between commands and assets: API routes are verbs, while static paths are files.

The `pathname` logic gives the browser `index.html` for the root URL and decodes other paths before joining them to `root`. The important protection is the `startsWith` check after normalization. That check

prevents a crafted URL from escaping the application folder and reading unrelated local files.

After the file is read, the response attaches shared security headers, a content type from the types table, and no-store caching. The no-store rule keeps the builder honest while the user is editing; the browser should not keep showing old JavaScript, stale JSON, or outdated images after a save.

The final listen call binds the local service to host and port. The console messages are mostly developer-facing, but they also tell us the intended operating model: the same backend can serve desktop preview traffic and LAN phone-preview traffic, while privileged API routes still rely on local-request checks.

Chapter 3: Request Boundaries And JSON Responses

Security headers as the default response posture

Before discussing individual endpoints, the file establishes a standard response posture. The `securityHeaders` helper returns browser-facing headers that reduce accidental exposure. The headers do not magically secure every feature, but they make the default response behavior stricter.

The extra parameter lets a caller add or override a small number of headers without rewriting the common security posture. That is the important JavaScript idea here: a base object is spread into a new object, then feature-specific fields are added beside it.

Without this helper, every response branch would have to remember the same header rules. Duplicated security settings drift over time, so the helper keeps one policy close to the top of the file.

How `securityHeaders` works inside this feature

The helper returns an object, not a string, because Node's response writer accepts headers as named fields. The spread operation at the end lets a route add a content type or another special header while keeping the common security posture intact.

This is the implementation at the point where the feature needs it.

```
function securityHeaders(extra = {})
```

The function is small, but its importance is structural. It prevents security headers from becoming a memory test repeated throughout the file.

JSON responses as the app's common language

The frontend expects structured answers. `sendJson` turns a JavaScript object into JSON text, attaches the standard headers, blocks caching, and ends the HTTP response.

This helper makes route code simpler. Instead of every endpoint remembering status codes, content type, cache rules, and `JSON.stringify` formatting, each endpoint can focus on the result it is returning.

The data parameter may contain success information, validation errors, compiler output, publish details, update information, or AI text. The function does not care what feature produced the object. It only cares that the result should travel back to the browser as JSON.

How `sendJson` works inside this feature

This function writes the HTTP status and headers first, then serializes the payload with `JSON.stringify`. The pretty two-space formatting is useful during debugging because local API responses are readable if opened directly.

This is the implementation at the point where the feature needs it.

```
function sendJson(response, status, data) {
  response.writeHead(status, {
    ...securityHeaders(),
    "Content-Type": "application/json",
    "Cache-Control": "no-store, no-cache, must-revalidate"
  });
  response.end(JSON.stringify(data, null, 2));
}
```

Every endpoint that calls this helper gets no-store cache behavior. That matters for a builder because stale catalog, target, compile, or update responses would confuse the user.

Reading request bodies without trusting them

POST routes need request bodies. `readRequestJson` collects body chunks, enforces a size limit, converts the body to text, and parses it as JSON. This is one of the most important backend boundary functions because all write-style actions depend on it.

The size limit is not cosmetic. A local app can still be harmed by a huge accidental payload. The helper stops reading once the body is too large and rejects the request rather than letting memory usage grow without control.

The parsed object is still not automatically trusted. Later endpoint code checks whether the catalog has arrays, whether upload data exists, whether a project id is safe, and whether a GitHub target is valid.

How `readRequestJson` works inside this feature

`readRequestJson` receives a POST body as a stream, enforces a maximum size while reading, joins chunks into text, and parses JSON.

This is the implementation at the point where the feature needs it.

```
asvnc function readRequestJson(request) {
  const chunks = [];
  let size = 0;

  for await (const chunk of request) {
    size += chunk.length;
    if (size > 60 * 1024 * 1024) {
      throw new Error("Request body is too large.");
    }
    chunks.push(chunk);
  }

  return JSON.parse(Buffer.concat(chunks).toString("utf8") || "{}").replace(/^\\uFEFF/, "");
}
```

This function is the shared entrance for write operations. It does not validate every possible payload because each route has its own data contract, but it ensures that all routes start from a bounded parsed object.

Path resolution as a safety rail

The builder writes files, accepts uploads, copies publishable assets, and creates compile workspaces. Because of that, path resolution is a security boundary. The backend must never let

untrusted input escape the folder it was meant to operate inside.

The `resolveInsideRoot` family creates an absolute path and then checks that the path still begins inside the approved root. That pattern is the local filesystem version of a guardrail: a project file can be saved under docs, but it cannot use `../` to climb into a private folder.

The separate helpers for app root, portfolio root, and compile root make the intent clearer. Public website files, builder app files, and compile artifacts do not all belong in the same safety boundary.

How `resolveInsideRoot` works inside this feature

`resolveInsideRoot` joins path segments to the application root and verifies that the final absolute path still belongs under root.

This is the implementation at the point where the feature needs it.

```
function resolveInsideRoot(...segments) {
  const target = path.normalize(path.join(root, ...segments));
  if (!target.startsWith(root)) {
    throw new Error("Resolved path is outside the workspace.");
  }
  return target;
}
```

The check is the important part. A sanitized name is helpful, but a final root-boundary check is what prevents path traversal from becoming file access outside the builder folder.

How `resolveInsidePortfolioRoot` works inside this feature

`resolveInsidePortfolioRoot` applies the same boundary rule to the website/publish workspace. It is used when the backend means public portfolio files rather than app implementation files.

This is the implementation at the point where the feature needs it.

```
function resolveInsidePortfolioRoot(...segments) {
  const target = path.resolve(portfolioRoot, ...segments);
  const relative = path.relative(portfolioRoot, target);
  if (relative.startsWith("../") || path.isAbsolute(relative)) {
    throw new Error("Resolved path is outside the portfolio workspace.");
  }
  return target;
}
```

The separate helper makes mistakes easier to spot in code review. Publishing code should normally use the portfolio boundary; application-code routes should normally use the app boundary.

How `resolveInsideCompileRoot` works inside this feature

`resolveInsideCompileRoot` applies the boundary rule to compile workspaces and build caches. It keeps generated binaries, class files, source copies, and waveforms out of the website folder.

This is the implementation at the point where the feature needs it.

```
function resolveInsideCompileRoot(...segments) {
  const target = path.resolve(compileRoot, ...segments);
  const relative = path.relative(compileRoot, target);
  if (relative.startsWith("../") || path.isAbsolute(relative)) {
    throw new Error("Resolved path is outside the compile workspace.");
  }
  return target;
}
```

This separation reduces accidental publishing risk and prepares the system for future Docker/container compilation where `compileRoot` could be mounted independently.

Text cleanup before data is trusted or sent to AI

Several backend features need to shorten, strip, or normalize text before it moves into a response, an AI prompt, or a public-source excerpt. `clampText` is the simplest guard: it turns an unknown value into text and cuts it to a maximum length. `stripHtmlToText` is more opinionated: it removes script and style blocks, removes tags, decodes common entities, and compresses whitespace.

These helpers are not trying to be a full browser or a perfect HTML parser. They are meant to make fetched public pages and saved rich content usable as plain evidence. Without them, AI prompts could become too large, HTML markup could drown out useful text, and source excerpts could carry irrelevant page structure instead of human-readable content.

How `clampText` works inside this feature

`clampText` is a small pressure valve. It accepts an unknown value, converts it into a string, and limits how much text can travel into an API response, AI prompt, title, metadata field, or source excerpt.

This is the implementation at the point where the feature needs it.

```
function clampText(value, maxLength = 12000) {
  return String(value || "").slice(0, maxLength);
}
```

The helper is deliberately plain because it sits near trust boundaries. It does not try to understand the meaning of the text. Its job is to stop unbounded strings from expanding until they dominate memory, PDF output, AI context, or UI dialogs.

The fallback through `String(value || "")` means null, undefined, and other empty-like values become harmless empty text. The `maxLength` parameter lets each caller choose a size appropriate for the feature: short for titles, larger for AI context, and larger still for fetched source excerpts.

How `stripHtmlToText` works inside this feature

`stripHtmlToText` turns markup-heavy content into readable evidence. It removes script and style regions first because those blocks are usually behavior or decoration, not the human text that should be shown to the AI assistant or a security report.

This is the implementation at the point where the feature needs it.

```
function stripHtmlToText(value = "") {
  return String(value || "")
    .replace(/<script[\s\S]*?<\script>/gi, " ")
    .replace(/<style[\s\S]*?<\style>/gi, " ")
    .replace(/<[^>]+>/g, " ")
    .replace(/&nbsp;/gi, " ")
    .replace(/&amp;/gi, "&")
    .replace(/&lt;/gi, "<")
    .replace(/&gt;/gi, ">")
    .replace(/&quot;/gi, "'")
    .replace(/&#39;/g, "'")
    .replace(/\\s+/g, " ")
    .trim();
}
```

After that, tags are replaced with spaces and common HTML entities are decoded. This matters when portfolio content comes from rich editors, saved pages, GitHub-rendered files, or uploaded documentation. The backend wants the meaning, not the browser markup.

The final whitespace compression is as important as the tag removal. Without it, an extracted paragraph could become a river of blank lines and odd spacing, making AI prompts harder to read and search excerpts less useful.

Local request checks before privileged actions

The local builder may listen on a host that a browser can reach, but that does not mean every device on the network should be allowed to run compiler commands, read publishing status, authenticate GitHub, or launch an installer. `isLocalRequest` is the backend's first practical answer to that problem.

The helper reads the request's remote address and treats loopback addresses as local. `handleApi` uses this result before write-style POST routes and before sensitive read routes such as security reports. The pattern is intentionally simple: route code asks whether the request came from this machine before allowing operations that belong only to the owner sitting at the builder.

How `isLocalRequest` works inside this feature

`isLocalRequest` checks the remote address and decides whether the request came from the same machine. Loopback addresses are treated as local.

This is the implementation at the point where the feature needs it.

```
function isLocalRequest(request) {  
  const address = request.socket.remoteAddress || "";  
  return address === "127.0.0.1" || address === "::1" || address === "::ffff:127.0.0.1";  
}
```

The helper is used before routes that expose machine details or perform side effects: compiler status, security reports, saving files, installing tools, launching updates, authenticating GitHub, and publishing.

Safe names and safe paths are separate protections

`safeSegment` and `safeFileName` clean user-supplied names so they can be used as folder names or filenames. They remove characters that do not belong in a predictable local path and fall back to boring names when input is empty. That is useful, but it is not the whole safety story.

The path-resolving helpers then check the final absolute location. This second step matters because a cleaned name and a safe root boundary solve different problems. A name sanitizer keeps filenames readable and compatible. A root-boundary check prevents the final path from escaping the approved workspace. The upload endpoint, compile workspace, and publish sync all depend on both ideas.

How `safeSegment` works inside this feature

`safeSegment` turns an arbitrary title or id into one safe path segment. It strips unusual characters, trims repeated separators, limits length, and falls back to a boring name when needed.

This is the implementation at the point where the feature needs it.

```
function safeSegment(value, fallback = "item") {  
  const segment = String(value || fallback)  
    .toLowerCase()  
    .trim()  
    .replace(/^[^a-z0-9_ -]+/g, "-")  
    .replace(/^[^~+|-+$/g, "");  
  return segment || fallback;  
}
```

This is used for project folders, section folders, compile file ids, temporary branch names, and update filename fragments. The goal is readable paths that cannot carry path separators or control characters.

How safeFileName works inside this feature

safeFileName is the upload-oriented filename cleaner. It preserves a basename and extension while removing characters that do not belong in a local filename.

This is the implementation at the point where the feature needs it.

```
function safeFileName(value) {
  const parsed = path.parse(String(value || "file"));
  const name = safeSegment(parsed.name, "file");
  const ext = safeSegment(parsed.ext.replace(".", "").replace(".", ""));
  return ext ? `${name}.${ext}` : name;
}
```

Unlike safeCodeFileName, this helper does not force a compile-language extension. Project evidence can be PDFs, images, schematics, source files, logs, spreadsheets, and many other file types.

How resolveInsideRoot works inside this feature

resolveInsideRoot joins path segments to the application root and verifies that the final absolute path still belongs under root.

This is the implementation at the point where the feature needs it.

```
function resolveInsideRoot(...segments) {
  const target = path.normalize(path.join(root, ...segments));
  if (!target.startsWith(root)) {
    throw new Error("Resolved path is outside the workspace.");
  }
  return target;
}
```

The check is the important part. A sanitized name is helpful, but a final root-boundary check is what prevents path traversal from becoming file access outside the builder folder.

How resolveInsidePortfolioRoot works inside this feature

resolveInsidePortfolioRoot applies the same boundary rule to the website/publish workspace. It is used when the backend means public portfolio files rather than app implementation files.

This is the implementation at the point where the feature needs it.

```
function resolveInsidePortfolioRoot(...segments) {
  const target = path.resolve(portfolioRoot, ...segments);
  const relative = path.relative(portfolioRoot, target);
  if (relative.startsWith("..") || path.isAbsolute(relative)) {
    throw new Error("Resolved path is outside the portfolio workspace.");
  }
  return target;
}
```

The separate helper makes mistakes easier to spot in code review. Publishing code should normally use the portfolio boundary; application-code routes should normally use the app boundary.

How resolveInsideCompileRoot works inside this feature

resolveInsideCompileRoot applies the boundary rule to compile workspaces and build caches. It keeps generated binaries, class files, source copies, and waveforms out of the website folder.

This is the implementation at the point where the feature needs it.

```
function resolveInsideCompileRoot(...segments) {
  const target = path.resolve(compileRoot, ...segments);
  const relative = path.relative(compileRoot, target);
  if (relative.startsWith("..") || path.isAbsolute(relative)) {
    throw new Error("Resolved path is outside the compile workspace.");
  }
}
```

```
    return target;
}
```

This separation reduces accidental publishing risk and prepares the system for future Docker/container compilation where compileRoot could be mounted independently.

Chapter 4: API Routing As User Features

The API switchboard

handleApi is the backend switchboard. It is large because it replaces what a framework router would normally provide. The important reading strategy is not to memorize every branch. Read it as a sequence of user-facing actions.

The first part handles GET routes that read information: catalog data, templates, publishing target status, system checks, update information, security reports, and compiler tool status. Then the function blocks non-local write requests before allowing POST routes to do privileged work.

That ordering matters. Read-only routes are cheap and safe enough to answer directly. Write routes must prove the request is local because they can save files, authenticate GitHub, run compilers, install tools, or publish the website.

How handleApi works inside this feature

handleApi is the handwritten router for the local backend. It begins with safe GET routes: catalog, templates, publishing target status, system check, app update information, security report, and compiler tool status.

This is the implementation at the point where the feature needs it.

```
asvnc function handleApi(request, response, url) {
  if (request.method === "GET" && url.pathname === "/api/catalog") {
    try {
      const catalog = await readJsonFile(draftPath);
      sendJson(response, 200, { source: "draft", catalog });
    } catch {
      sendJson(response, 200, { source: "published", catalog: await readJsonFile(catalogPath) });
    }
    return true;
  }

  if (request.method === "GET" && url.pathname === "/api/templates") {
    sendJson(response, 200, await readJsonFile(path.join(root, "project-templates.json")));
    return true;
  }

  if (request.method === "GET" && url.pathname === "/api/publish-target") {
    if (!isLocalRequest(request)) {
      sendJson(response, 403, { error: "Publishing target details are only available from this computer." });
      return true;
    }
    sendJson(response, 200, { ok: true, target: await getPublishTargetInfo() });
    return true;
  }

  if (request.method === "GET" && url.pathname === "/api/system-check") {
    sendJson(response, 200, { ok: true, svstem: await getGitStatus(), target: await getPublishTargetInfo() });
    return true;
  }

  if (request.method === "GET" && url.pathname === "/api/app-update") {
    sendJson(response, 200, await getUpdateInfo());
    return true;
  }

  if (request.method === "GET" && url.pathname === "/api/security-report") {
```

```

    if (!isLocalRequest(request)) {
      sendJson(response, 403, { error: "Security reports are only available from this computer." });
      return true;
    }
    sendJson(response, 200, await getSecurityReport());
    return true;
  }

  if (request.method === "GET" && url.pathname === "/api/code/tools") {
    if (!isLocalRequest(request)) {
      sendJson(response, 403, { error: "Compiler details are only available from this computer." });
      return true;
    }
    sendJson(response, 200, { ok: true, tools: await compileToolStatus() });
    return true;
  }

  if (request.method !== "POST") return false;

  if (!isLocalRequest(request)) {
    sendJson(response, 403, { error: "Write actions are only allowed from this computer." });
    return true;
  }

  if (url.pathname === "/api/portfolio-ai") {
    await handlePortfolioAi(request, response);
    return true;
  }

  if (url.pathname === "/api/code/beautify") {
    try {
      const body = await readRequestJson(request);
      const language = normalizeCodeLanguage(body.language || detectCodeLanguageFromSource(body.code,
        body.fileName));
      sendJson(response, 200, {
        ok: true,
        language,
        code: beautifyCode(body.code || "", language)
      });
    } catch (error) {
      sendJson(response, 400, { ok: false, error: error.message || "Code could not be beautified." });
    }
    return true;
  }

  if (url.pathname === "/api/code/save") {
    try {
      const body = await readRequestJson(request);
      const saved = await saveCompileSource(body);
      sendJson(response, 200, { ok: true, saved });
    } catch (error) {
      sendJson(response, 400, { ok: false, error: error.message || "Code could not be saved." });
    }
    return true;
  }

  if (url.pathname === "/api/code/compile") {
    try {
      const body = await readRequestJson(request);
      const result = await compileAndRunCode(body);
      sendJson(response, 200, { ok: result.ok, result });
    } catch (error) {
      sendJson(response, 400, { ok: false, error: error.message || "Code could not be compiled.", result: null });
    }
    return true;
  }

  if (url.pathname === "/api/code/install-tools") {
    try {
      const body = await readRequestJson(request);
      const result = await installCompilerTools(body.language || "");
      sendJson(response, 200, result);
    } catch (error) {
      sendJson(response, 400, { ok: false, error: error.message || "Compiler tools could not be installed." });
    }
    return true;
  }
}

```

```

if (url.pathname === "/api/publish-target") {
  await readRequestJson(request).catch(() => ({}));
  sendJson(response, 400, {
    ok: false,
    error: "Publishing target setup now requires GitHub authentication.",
    details: "Use Authenticate target. The builder keeps the previous target until GitHub write access is
      verified."
  });
  return true;
}

if (url.pathname === "/api/install-git") {
  try {
    const install = await installGitForWindows();
    sendJson(response, 200, { ok: true, install, svsystem: await getGitStatus() });
  } catch (error) {
    sendJson(response, 400, {
      ok: false,
      error: error.message || "Git for Windows installer could not be started.",
      downloadUrl: "https://git-scm.com/download/win"
    });
  }
  return true;
}

if (url.pathname === "/api/app-update/install") {
  try {
    await readRequestJson(request).catch(() => ({}));
    const update = await downloadAndLaunchAppUpdate();
    sendJson(response, 200, { ok: true, update });
  } catch (error) {
    sendJson(response, 400, {
      ok: false,
      error: error.message || "The update could not be started."
    });
  }
  return true;
}

if (url.pathname === "/api/github-authenticate") {
  try {
    const body = await readRequestJson(request);
    const auth = await authenticateGitHubForTarget(body || {});
    sendJson(response, 200, { ok: true, auth, target: await getPublishTargetInfo() });
  } catch (error) {
    sendJson(response, 400, {
      ok: false,
      error: error.message || "GitHub authentication did not complete.",
      details: error.details || publishAuthorizationHelp,
      publishAccess: error.publishAccess || null,
      svsystem: await getGitStatus()
    });
  }
  return true;
}

if (url.pathname === "/api/svnc-from-publish-target") {
  try {
    const svnc = await svncFromPublishTarget();
    sendJson(response, 200, { ok: true, svnc, target: await getPublishTargetInfo() });
  } catch (error) {
    sendJson(response, 400, {
      ok: false,
      error: error.message || "Publishing target could not be imported.",
      details: error.details || publishAuthorizationHelp,
      publishAccess: error.publishAccess || null
    });
  }
  return true;
}

if (url.pathname === "/api/save-draft" || url.pathname === "/api/annlv-catalog") {
  const body = await readRequestJson(request);
  const catalog = body.catalog;

  if (!catalog || !Array.isArray(catalog.categories) || !Array.isArray(catalog.projects) ||
    !catalog.categories.length) {
    sendJson(response, 400, { error: "Catalog must include categories and projects arrays." });
  }
}

```

```

    return true:
  }

const applingToSite = url.pathname === "/api/appling-catalog":
let publishAccess = null:
if (applingToSite) {
  try {
    publishAccess = await assertPublishAccess():
  } catch (error) {
    sendJson(response, 200, {
      ok: false,
      file: path.relative(root, catalogPath),
      publish: {
        pushed: false,
        authorizationRequired: true,
        error: error.message || "Publishing authorization failed.",
        details: error.details || publishAuthorizationHelp,
        ...(error.publishAccess || {})
      }
    }):
    return true:
  }
}

const target = applingToSite ? catalogPath : draftPath:
await writeFile(target, `${JSON.stringify(catalog, null, 2)}\n`, "utf8"):
if (applingToSite) {
  try {
    const publish = await publishSiteChanges(publishAccess):
    sendJson(response, 200, { ok: true, file: path.relative(root, target), publish }):
  } catch (error) {
    sendJson(response, 200, {
      ok: true,
      file: path.relative(root, target),
      publish: {
        pushed: false,
        error: error.stderr || error.stdout || error.message || "Git push failed."
      }
    }):
  }
  return true:
}
sendJson(response, 200, { ok: true, file: path.relative(root, target) }):
return true:
}

if (url.pathname === "/api/unload") {
  const body = await readRequestJson(request):
  const projectId = safeSegment(body.projectId, "project"):
  const section = safeSegment(body.section, "documents"):
  const fileName = safeFileName(body.fileName):
  const base64 = String(body.data || "").replace(/^data:[^:]+:base64./, ""):

  if (!base64) {
    sendJson(response, 400, { error: "Upload data is missing." }):
    return true:
  }

  const folder = resolveInsideRoot("docs", projectId, section):
  const filePath = resolveInsideRoot("docs", projectId, section, fileName):
  await mkdir(folder, { recursive: true }):
  await writeFile(filePath, Buffer.from(base64, "base64")):

  sendJson(response, 200, {
    ok: true,
    url: path.relative(root, filePath).replaceAll(path.sep, "/")
  }):
  return true:
}

return false:
}

```

After GET handling, it rejects every non-POST request. Then it applies the local-only guard before any POST branch can run. This is the crucial security rhythm of the router: read routes are limited

where needed, and write routes must come from the local machine.

The POST branches map directly to app features. Portfolio AI, code beautify, code save, code compile, compiler-tool install, Git install, app update, GitHub authentication, load from target, save draft/apply catalog, and upload each have a dedicated branch.

The save/apply branch shows the route's most important difference between local and public changes. Both actions validate the catalog shape. Save draft writes `projects.local.json`. Apply to site first calls `assertPublishAccess`, writes `projects.json`, then calls `publishSiteChanges`. If publishing fails after writing, the response still explains what happened instead of hiding the partial result.

The upload branch demonstrates file handling in the router. It reads project id, section, filename, and base64 bytes, sanitizes path pieces, writes under docs, and returns a relative URL. The route does not try to interpret every file type; it stores evidence safely and lets the parser decide how to present it.

The Save draft and Apply to site buttons

The same catalog payload supports two different user actions. Save draft writes the local draft file. Apply to site writes the public catalog and then publishes compatible website files to the authenticated Git target.

The endpoint first validates that the catalog has categories and projects arrays. That validation is a practical data-contract check. If the catalog shape is broken, the builder should not save it as the new truth.

Apply to site performs an authorization check before writing publishable data. That ordering protects the live website from being changed when GitHub write access has not been verified for the current target.

How draftPath shapes this workflow

`draftPath` is the local working catalog. Save draft writes here so unfinished builder edits can survive app restarts without becoming the public site catalog.

The code below is the actual shared value used by this feature group.

```
const draftPath = path.join(root, "projects.local.json");
```

How catalogPath shapes this workflow

`catalogPath` is the published website catalog path. Apply to site writes here before the publish flow stages and pushes public files.

The code below is the actual shared value used by this feature group.

```
const catalogPath = path.join(root, "projects.json");
```

How publishAuthCachePath shapes this workflow

`publishAuthCachePath` points to the local authorization session file. It is not a public website asset and should never be pushed as portfolio content.

The cache file lets the builder avoid asking for GitHub authorization on every Apply to site click while still limiting trust to a matching target and branch.

The code below is the actual shared value used by this feature group.

```
const publishAuthCachePath = path.join(root, ".omb-publish-session.json");
```

How assertPublishAccess works inside this feature

assertPublishAccess is the gatekeeper for live website changes. It first proves the portfolio workspace is a Git repository, then reads origin and current branch, then checks compatibility unless the caller intentionally disables that requirement.

This is the implementation at the point where the feature needs it.

```
async function assertPublishAccess(options = {})
```

After building the access shape, it tries the authorization cache. A fresh matching cache returns immediately with authorizationCached true. That is the daily-login behavior the user wanted: reopening the app should not force GitHub authorization again when the target is unchanged.

If the cache is missing or expired, the function synchronizes the local branch from remote, ensures there is a HEAD commit, performs the dry-run write check, writes a new auth cache, and returns a detailed access object. The function both protects Apply to site and supplies the UI with exactly what was verified.

How publishSiteChanges works inside this feature

publishSiteChanges turns a saved catalog and public asset set into a Git push. It obtains or receives verified publish access, synchronizes the branch from remote, copies publishable files, bumps asset versions, filters stageable paths, stages them, commits only when there are changes, and pushes.

This is the implementation at the point where the feature needs it.

```
async function publishSiteChanges(publishAccess = null) {
  const access = publishAccess || await assertPublishAccess();
  const branch = access.branch || (await runPublishGit(["branch", "--show-current"])).stdout.trim();
  const remoteSvnc = await svncSynchronizePublishBranchFromRemote(branch, access);
  const portfolioSvnc = await svncPortfolioPublishFiles({ removeMissing: true });
  await bumpPublishedAssetVersions(portfolioRoot);

  const stageablePaths = await stageablePublishPaths(nublishPaths);
  if (!stageablePaths.length) {
    throw new Error("No compatible site files were available to publish.");
  }

  await runPublishGit(["add", "-A", "--", ...stageablePaths]);
  const status = await runPublishGit(["status", "--porcelain", "--", ...stageablePaths]);
  const hasChanges = status.stdout.trim().length > 0;

  let commit = null;
  if (hasChanges) {
    const message = `Update portfolio site ${new Date().toISOString().slice(0, 10)}`;
    commit = await runPublishGit(["commit", "-m", message]);
  }

  const pushArgs = branch ? ["push", "origin", branch] : ["push"];
  const push = await runPublishGit(pushArgs);

  return {
    ...access,
    portfolioWorkspace: portfolioRoot,
    remoteSvnc,
    portfolioSvnc,
    branch: branch || "current branch",
    committed: hasChanges,
    commitOutput: commit?.stdout || commit?.stderr || "",
    pushed: true,
    pushOutput: push.stdout || push.stderr || ""
  };
}
```

The function is intentionally conservative. It refuses to publish when no compatible files are stageable. It avoids empty commits. It returns commit and push output so the success dialog can show real Git evidence.

The ordering solves the earlier non-fast-forward problem: synchronize first, copy local public files second, stage and commit third, push last. That mirrors the safe human workflow for updating a GitHub Pages repository.

The Upload button

The upload endpoint receives base64 file data, a project id, a section name, and a filename. It sanitizes each path segment, creates the destination folder under docs, decodes the bytes, writes the file, and returns a relative URL.

This feature shows the difference between accepting all file types as evidence and trusting arbitrary paths. The backend can save many file extensions, but it still controls where those files go.

The returned URL is what the builder stores in project content. Later the parser can render an image, offer a PDF link, or expose a downloadable source file depending on how the project section uses that asset.

Catalog loading routes before anything is edited

The `/api/catalog` route exists because the builder must prefer the local draft when a draft exists, while the public website should still have a published catalog. `handleApi` first tries `projects.local.json` and falls back to `projects.json`. That means local builder work can be previewed without pretending it has already been published.

`readJsonFile` is intentionally small, but it is part of the story: it reads a file as UTF-8 and parses JSON. If that read fails for the draft, the route does not treat it as a fatal error; it simply answers with the published catalog. This is a good example of feature-specific error meaning. A missing draft is normal, while a malformed saved catalog would be a real problem.

How `draftPath` shapes this workflow

`draftPath` is the local working catalog. `Save draft` writes here so unfinished builder edits can survive app restarts without becoming the public site catalog.

The code below is the actual shared value used by this feature group.

```
const draftPath = path.join(root, "projects.local.json");
```

How `catalogPath` shapes this workflow

`catalogPath` is the published website catalog path. `Apply to site` writes here before the publish flow stages and pushes public files.

The code below is the actual shared value used by this feature group.

```
const catalogPath = path.join(root, "projects.json");
```

How `readJsonFile` works inside this feature

`readJsonFile` is the small JSON file reader used by catalog, template, and configuration routes. It reads UTF-8 text and parses it into a JavaScript object.

This is the implementation at the point where the feature needs it.

```

asvnc function readJsonFile(filePath) {
  return JSON.parse(await readFile(filePath, "utf8"));
}

```

The helper keeps route code compact, but callers still decide what failure means. A missing draft catalog can fall back to the published catalog, while a missing required template file may be a real endpoint error.

Save draft as a local-only write path

The Save draft path is the calm version of catalog writing. It receives the catalog object, checks that categories and projects are arrays, and writes the data to `projects.local.json`. No Git operation happens. No live website files are pushed. The whole point is to preserve the builder's working state locally.

The same validation used for Apply to site is used here because a broken draft can still break local previews. The function writes formatted JSON with a trailing newline so the file remains readable in Git diffs and text editors. The response returns the file path relative to the app root because the frontend needs a human-friendly success message, not an absolute machine path.

How `handleApi` works inside this feature

`handleApi` is the handwritten router for the local backend. It begins with safe GET routes: catalog, templates, publishing target status, system check, app update information, security report, and compiler tool status.

This is the implementation at the point where the feature needs it.

```

asvnc function handleApi(request, response, url) {
  if (request.method === "GET" && url.pathname === "/api/catalog") {
    try {
      const catalog = await readJsonFile(draftPath);
      sendJson(response, 200, { source: "draft", catalog });
    } catch {
      sendJson(response, 200, { source: "published", catalog: await readJsonFile(catalogPath) });
    }
    return true;
  }

  if (request.method === "GET" && url.pathname === "/api/templates") {
    sendJson(response, 200, await readJsonFile(path.join(root, "project-templates.json")));
    return true;
  }

  if (request.method === "GET" && url.pathname === "/api/publish-target") {
    if (!isLocalRequest(request)) {
      sendJson(response, 403, { error: "Publishing target details are only available from this computer." });
      return true;
    }
    sendJson(response, 200, { ok: true, target: await getPublishTargetInfo() });
    return true;
  }

  if (request.method === "GET" && url.pathname === "/api/system-check") {
    sendJson(response, 200, { ok: true, system: await getGitStatus(), target: await getPublishTargetInfo() });
    return true;
  }

  if (request.method === "GET" && url.pathname === "/api/app-update") {
    sendJson(response, 200, await getUpdateInfo());
    return true;
  }

  if (request.method === "GET" && url.pathname === "/api/security-report") {
    if (!isLocalRequest(request)) {
      sendJson(response, 403, { error: "Security reports are only available from this computer." });
      return true;
    }
  }
}

```

```

    sendJson(response, 200, await getSecurityReport());
    return true;
}

if (request.method === "GET" && url.pathname === "/api/code/tools") {
    if (!isLocalRequest(request)) {
        sendJson(response, 403, { error: "Compiler details are only available from this computer." });
        return true;
    }
    sendJson(response, 200, { ok: true, tools: await compileToolStatus() });
    return true;
}

if (request.method !== "POST") return false;

if (!isLocalRequest(request)) {
    sendJson(response, 403, { error: "Write actions are only allowed from this computer." });
    return true;
}

if (url.pathname === "/api/portfolio-ai") {
    await handlePortfolioAi(request, response);
    return true;
}

if (url.pathname === "/api/code/beautify") {
    try {
        const body = await readRequestJson(request);
        const language = normalizeCodeLanguage(body.language || detectCodeLanguageFromSource(body.code,
            body.fileName));
        sendJson(response, 200, {
            ok: true,
            language,
            code: beautifyCode(body.code || "", language)
        });
    } catch (error) {
        sendJson(response, 400, { ok: false, error: error.message || "Code could not be beautified." });
    }
    return true;
}

if (url.pathname === "/api/code/save") {
    try {
        const body = await readRequestJson(request);
        const saved = await saveCompileSource(body);
        sendJson(response, 200, { ok: true, saved });
    } catch (error) {
        sendJson(response, 400, { ok: false, error: error.message || "Code could not be saved." });
    }
    return true;
}

if (url.pathname === "/api/code/compile") {
    try {
        const body = await readRequestJson(request);
        const result = await compileAndRunCode(body);
        sendJson(response, 200, { ok: result.ok, result });
    } catch (error) {
        sendJson(response, 400, { ok: false, error: error.message || "Code could not be compiled.", result: null });
    }
    return true;
}

if (url.pathname === "/api/code/install-tools") {
    try {
        const body = await readRequestJson(request);
        const result = await installCompilerTools(body.language || "");
        sendJson(response, 200, result);
    } catch (error) {
        sendJson(response, 400, { ok: false, error: error.message || "Compiler tools could not be installed." });
    }
    return true;
}

if (url.pathname === "/api/publish-target") {
    await readRequestJson(request).catch(() => {});
    sendJson(response, 400, {
        ok: false,
    });
}

```

```

    error: "Publishing target setup now requires GitHub authentication.".
    details: "Use Authenticate target. The builder keeps the previous target until GitHub write access is
      verified."
  });
  return true;
}

if (url.pathname === "/api/install-git") {
  try {
    const install = await installGitForWindows();
    sendJson(response, 200, { ok: true, install, svsystem: await getGitStatus() });
  } catch (error) {
    sendJson(response, 400, {
      ok: false,
      error: error.message || "Git for Windows installer could not be started.",
      downloadUrl: "https://git-scm.com/download/win"
    });
  }
  return true;
}

if (url.pathname === "/api/app-update/install") {
  try {
    await readRequestJson(request).catch(() => ({}));
    const update = await downloadAndLaunchAppUpdate();
    sendJson(response, 200, { ok: true, update });
  } catch (error) {
    sendJson(response, 400, {
      ok: false,
      error: error.message || "The update could not be started."
    });
  }
  return true;
}

if (url.pathname === "/api/github-authenticate") {
  try {
    const body = await readRequestJson(request);
    const auth = await authenticateGitHubForTarget(body || {});
    sendJson(response, 200, { ok: true, auth, target: await getPublishTargetInfo() });
  } catch (error) {
    sendJson(response, 400, {
      ok: false,
      error: error.message || "GitHub authentication did not complete.",
      details: error.details || publishAuthorizationHelp,
      publishAccess: error.publishAccess || null,
      svsystem: await getGitStatus()
    });
  }
  return true;
}

if (url.pathname === "/api/sync-from-publish-target") {
  try {
    const svnc = await svncFromPublishTarget();
    sendJson(response, 200, { ok: true, svnc, target: await getPublishTargetInfo() });
  } catch (error) {
    sendJson(response, 400, {
      ok: false,
      error: error.message || "Publishing target could not be imported.",
      details: error.details || publishAuthorizationHelp,
      publishAccess: error.publishAccess || null
    });
  }
  return true;
}

if (url.pathname === "/api/save-draft" || url.pathname === "/api/applv-catalog") {
  const body = await readRequestJson(request);
  const catalog = body.catalog;

  if (!catalog || !Array.isArray(catalog.categories) || !Array.isArray(catalog.projects) ||
    !catalog.categories.length) {
    sendJson(response, 400, { error: "Catalog must include categories and projects arrays." });
    return true;
  }
}

const applyingToSite = url.pathname === "/api/apply-catalog";

```

```

let publishAccess = null;
if (applyingToSite) {
  try {
    publishAccess = await assertPublishAccess();
  } catch (error) {
    sendJson(response, 200, {
      ok: false,
      file: path.relative(root, catalogPath),
      publish: {
        pushed: false,
        authorizationRequired: true,
        error: error.message || "Publishing authorization failed.",
        details: error.details || publishAuthorizationHelp,
        ...(error.publishAccess || {})
      }
    });
    return true;
  }
}

const target = applyingToSite ? catalogPath : draftPath;
await writeFile(target, `${JSON.stringify(catalog, null, 2)}\n`, "utf8");
if (applyingToSite) {
  try {
    const publish = await publishSiteChanges(publishAccess);
    sendJson(response, 200, { ok: true, file: path.relative(root, target), publish });
  } catch (error) {
    sendJson(response, 200, {
      ok: true,
      file: path.relative(root, target),
      publish: {
        pushed: false,
        error: error.stderr || error.stdout || error.message || "Git push failed."
      }
    });
  }
  return true;
}
sendJson(response, 200, { ok: true, file: path.relative(root, target) });
return true;
}

if (url.pathname === "/api/upload") {
  const body = await readRequestJson(request);
  const projectId = safeSegment(body.projectId, "project");
  const section = safeSegment(body.section, "documents");
  const fileName = safeFileName(body.fileName);
  const base64 = String(body.data || "").replace(/^data:[^:]+;base64/, "");

  if (!base64) {
    sendJson(response, 400, { error: "Upload data is missing." });
    return true;
  }

  const folder = resolveInsideRoot("docs", projectId, section);
  const filePath = resolveInsideRoot("docs", projectId, section, fileName);
  await mkdir(folder, { recursive: true });
  await writeFile(filePath, Buffer.from(base64, "base64"));

  sendJson(response, 200, {
    ok: true,
    url: path.relative(root, filePath).replaceAll(path.sep, "/")
  });
  return true;
}

return false;
}

```

After GET handling, it rejects every non-POST request. Then it applies the local-only guard before any POST branch can run. This is the crucial security rhythm of the router: read routes are limited where needed, and write routes must come from the local machine.

The POST branches map directly to app features. Portfolio AI, code beautify, code save, code compile, compiler-tool install, Git install, app update, GitHub authentication, load from target, save draft/apply catalog, and upload each have a dedicated branch.

The save/apply branch shows the route's most important difference between local and public changes. Both actions validate the catalog shape. Save draft writes `projects.local.json`. Apply to site first calls `assertPublishAccess`, writes `projects.json`, then calls `publishSiteChanges`. If publishing fails after writing, the response still explains what happened instead of hiding the partial result.

The upload branch demonstrates file handling in the router. It reads project id, section, filename, and base64 bytes, sanitizes path pieces, writes under docs, and returns a relative URL. The route does not try to interpret every file type; it stores evidence safely and lets the parser decide how to present it.

Apply to site as Save draft plus publishing authority

Apply to site shares the catalog validation path, but it adds a gate before public files are written. `assertPublishAccess` must succeed before the backend writes `projects.json` and before `publishSiteChanges` starts Git work. This is the difference between changing a local draft and changing a public website.

The route is designed to report partial meaning. If authorization fails, the response says publishing was blocked and includes guidance. If the catalog write succeeds but Git push fails, the response still says which local file was written and then reports the push failure. That shape lets the UI display a truthful message instead of flattening every problem into a single generic error.

How `assertPublishAccess` works inside this feature

`assertPublishAccess` is the gatekeeper for live website changes. It first proves the portfolio workspace is a Git repository, then reads origin and current branch, then checks compatibility unless the caller intentionally disables that requirement.

This is the implementation at the point where the feature needs it.

```
async function assertPublishAccess(options = {})
```

After building the access shape, it tries the authorization cache. A fresh matching cache returns immediately with `authorizationCached` true. That is the daily-login behavior the user wanted: reopening the app should not force GitHub authorization again when the target is unchanged.

If the cache is missing or expired, the function synchronizes the local branch from remote, ensures there is a HEAD commit, performs the dry-run write check, writes a new auth cache, and returns a detailed access object. The function both protects Apply to site and supplies the UI with exactly what was verified.

How `publishSiteChanges` works inside this feature

`publishSiteChanges` turns a saved catalog and public asset set into a Git push. It obtains or receives verified publish access, synchronizes the branch from remote, copies publishable files, bumps asset versions, filters stageable paths, stages them, commits only when there are changes, and pushes.

This is the implementation at the point where the feature needs it.

```
async function publishSiteChanges(publishAccess = null) {  
  const access = publishAccess || await assertPublishAccess();
```

```

const branch = access.branch || (await runPublishGit(["branch", "--show-current"])).stdout.trim();
const remoteSvnc = await svnchroizePublishBranchFromRemote(branch, access);
const portfolioSvnc = await svnPortfolioPublishFiles({ removeMissing: true });
await bumpPublishedAssetVersions(portfolioRoot);

const stageablePaths = await stageablePublishPaths(publishPaths);
if (!stageablePaths.length) {
  throw new Error("No compatible site files were available to publish.");
}

await runPublishGit(["add", "-A", "--", ...stageablePaths]);
const status = await runPublishGit(["status", "--porcelain", "--", ...stageablePaths]);
const hasChanges = status.stdout.trim().length > 0;

let commit = null;
if (hasChanges) {
  const message = `Update portfolio site ${new Date().toISOString().slice(0, 10)}\`;
  commit = await runPublishGit(["commit", "-m", message]);
}

const pushArgs = branch ? ["push", "origin", branch] : ["push"];
const push = await runPublishGit(pushArgs);

return {
  ...access,
  portfolioWorkspace: portfolioRoot,
  remoteSvnc,
  portfolioSvnc,
  branch: branch || "current branch",
  committed: hasChanges,
  commitOutput: commit?.stdout || commit?.stderr || "",
  pushed: true,
  pushOutput: push.stdout || push.stderr || ""
};
}

```

The function is intentionally conservative. It refuses to publish when no compatible files are stageable. It avoids empty commits. It returns commit and push output so the success dialog can show real Git evidence.

The ordering solves the earlier non-fast-forward problem: synchronize first, copy local public files second, stage and commit third, push last. That mirrors the safe human workflow for updating a GitHub Pages repository.

Upload as a bytes-to-project-file workflow

The upload path turns a browser-supplied base64 payload into a real file under docs///. That sounds simple until you follow the boundary crossings: the project id, section name, filename, and file bytes all originate outside the backend. Each one is cleaned, decoded, or bounded before writing.

The function removes any data URL prefix, decodes the remaining base64 into a Buffer, creates the destination folder, writes the file, and returns a relative URL using forward slashes. That returned URL is the stable piece the catalog can store and the website can later render or offer as a download.

How safeSegment works inside this feature

safeSegment turns an arbitrary title or id into one safe path segment. It strips unusual characters, trims repeated separators, limits length, and falls back to a boring name when needed.

This is the implementation at the point where the feature needs it.

```

function safeSegment(value, fallback = "item") {
  const segment = String(value || fallback)
    .toLowerCase()
    .trim()
    .replace(/[\^a-z0-9. -]/g, "-")
    .replace(/^-+|-+$/g, "");
  return segment || fallback;
}

```

```
}
```

This is used for project folders, section folders, compile file ids, temporary branch names, and update filename fragments. The goal is readable paths that cannot carry path separators or control characters.

How safeFileName works inside this feature

safeFileName is the upload-oriented filename cleaner. It preserves a basename and extension while removing characters that do not belong in a local filename.

This is the implementation at the point where the feature needs it.

```
function safeFileName(value) {
  const parsed = path.parse(String(value || "file"));
  const name = safeSegment(parsed.name, "file");
  const ext = safeSegment(parsed.ext.replace(".", "").replace(".", ""));
  return ext ? `${name}.${ext}` : name;
}
```

Unlike safeCodeFileName, this helper does not force a compile-language extension. Project evidence can be PDFs, images, schematics, source files, logs, spreadsheets, and many other file types.

How resolveInsideRoot works inside this feature

resolveInsideRoot joins path segments to the application root and verifies that the final absolute path still belongs under root.

This is the implementation at the point where the feature needs it.

```
function resolveInsideRoot(...segments) {
  const target = path.normalize(path.join(root, ...segments));
  if (!target.startsWith(root)) {
    throw new Error("Resolved path is outside the workspace.");
  }
  return target;
}
```

The check is the important part. A sanitized name is helpful, but a final root-boundary check is what prevents path traversal from becoming file access outside the builder folder.

Chapter 5: Compile Code, Simulation, And Terminal Output

Language profiles as the IDE rulebook

The Compile Code workspace needs to know what each language expects. compileLanguageProfiles is the rulebook. It maps each supported language to default filenames, valid extensions, labels, primary tools, and installer hints.

This is feature-based design. Instead of scattering C, C++, Java, Verilog, SystemVerilog, LTspice, JavaScript, Python, and HTML rules across many functions, the map gives the compiler feature one place to ask what the language needs.

The map also separates adding code to a project from compiling code. A project section can store any evidence file, while the compile workspace only executes languages with supported toolchains.

How compileLanguageProfiles shapes this workflow

compileLanguageProfiles is the compile workspace's language table. Each entry names the default file, legal extensions, display label, required tools, and installer hints for one language mode.

The object is easier to maintain than scattered if statements. Adding a new compile language later should start here because this is where the frontend's language choice becomes backend rules.

The code below is the actual shared value used by this feature group.

```
const compileLanguageProfiles = {
  c: {
    defaultFile: "main.c",
    extensions: [".c", ".h"],
    label: "C",
    primaryTools: ["gcc"],
    winget: ["BrechtSanders.WinLibs.POSIX.UCRT"]
  },
  cpp: {
    defaultFile: "main.cpp",
    extensions: [".cpp", ".cc", ".cxx", ".hpp", ".h"],
    label: "C++",
    primaryTools: ["g++"],
    winget: ["BrechtSanders.WinLibs.POSIX.UCRT"]
  },
  verilog: {
    defaultFile: "design.v",
    extensions: [".v"],
    label: "Verilog",
    primaryTools: ["iverilog", "vvn"],
    winget: ["Icarus.Verilog"]
  },
  systemverilog: {
    defaultFile: "design.sv",
    extensions: [".sv", ".svh"],
    label: "SystemVerilog",
    primaryTools: ["iverilog", "vvp"],
    winget: ["Icarus.Verilog"]
  },
  ltspice: {
    defaultFile: "simulation.cir",
    extensions: [".cir", ".net", ".sp", ".asc"],
    label: "LTspice",
    primaryTools: ["ltspice"],
    winget: []
  },
  java: {
    defaultFile: "Main.java",
    extensions: [".java"],
    label: "Java",
    primaryTools: ["javac", "java"],
    winget: ["EclipseAdoptium.Temurin.21.JDK"]
  },
  javascript: {
    defaultFile: "main.js",
    extensions: [".js", ".mjs", ".cjs"],
    label: "JavaScript",
    primaryTools: ["node"],
    winget: []
  },
  python: {
    defaultFile: "main.py",
    extensions: [".py"],
    label: "Python",
    primaryTools: ["python"],
    winget: []
  },
  html: {
    defaultFile: "index.html",
    extensions: [".html", ".htm"],
    label: "HTML",
    primaryTools: [],
    winget: []
  }
};
```

How compileToolCandidates shapes this workflow

compileToolCandidates is the Windows-aware executable search table. It records command names and likely install paths for compilers, simulators, interpreters, and LTspice.

This object exists because Windows machines often have tools installed but not on PATH. The builder should be helpful enough to find common install locations before telling the user a compiler is missing.

The code below is the actual shared value used by this feature group.

```
const compileToolCandidates = {
  gcc: [
    "gcc",
    process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Microsoft", "WinGet", "Packages",
      "BrechtSanders.WinLibs.POSIX.UCRT Microsoft.Winget.Source 8wekvb3d8bbwe", "mingw64", "bin", "gcc.exe") :
    ""
  ],
  "g++": [
    "g++",
    "C++",
    process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Microsoft", "WinGet", "Packages",
      "BrechtSanders.WinLibs.POSIX.UCRT Microsoft.Winget.Source 8wekvb3d8bbwe", "mingw64", "bin", "g++.exe") :
    ""
  ],
  clang: ["clang"],
  "clang++": ["clang++"],
  cl: ["cl"],
  iavac: [
    "iavac",
    "C:\\Program Files\\Eclipse Adoptium\\jdk-21.0.11-hotspot\\bin\\iavac.exe"
  ],
  iava: [
    "iava",
    "C:\\Program Files\\Eclipse Adoptium\\jdk-21.0.11-hotspot\\bin\\iava.exe"
  ],
  node: [
    /(?:^|[/\\])node(?:\.exe)?$/i.test(process.execPath || "") ? process.execPath : "",
    "node",
    "C:\\Program Files\\nodejs\\node.exe",
    process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Programs", "nodejs", "node.exe") : ""
  ],
  pythons: [process.env.PYTHON, "python", "py"],
  iverilog: ["iverilog", "C:\\iverilog\\bin\\iverilog.exe"],
  vvn: ["vvn", "C:\\iverilog\\bin\\vvn.exe"],
  ltspice: [
    process.env.LTSPICE_EXE,
    "C:\\Program Files\\ADI\\LTspice\\LTspice.exe",
    "C:\\Program Files\\LTC\\LTspiceXVII\\XVIIx64.exe",
    "C:\\Program Files\\Analog Devices\\LTspice\\LTspice.exe",
    process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Programs", "ADI", "LTspice", "LTspice.exe") :
    ""
  ],
};
```

How normalizeCodeLanguage works inside this feature

normalizeCodeLanguage translates messy user-facing language names into the internal language keys used by the compile workspace. A person may type C++, cpp, JavaScript, node, sv, spice, or py; the compiler feature needs one stable key.

This is the implementation at the point where the feature needs it.

```
function normalizeCodeLanguage(value = "") {
  const clean = String(value || "").trim().toLowerCase().replace(/[\s-]+/g, "");
  const aliases = {
    c: "c",
    cpp: "cpp",
    colusplus: "cpp",
    "C++": "cpp",
    systemverilog: "systemverilog",
    sv: "systemverilog",
    verilog: "verilog",
    v: "verilog",
    ltspice: "ltspice",
    spice: "ltspice",
    cir: "ltspice",
    iava: "java",
    javascript: "javascript",
```

```

    is: "javascript".
    node: "javascript".
    pytho: "python".
    py: "python".
    html: "html".
    htm: "html"
  },
  return aliases[clean] || (compileLanguageProfiles[clean] ? clean : "javascript");
}

```

The aliases object is the polite front door. It accepts common spellings and abbreviations, then maps them to compileLanguageProfiles. The final fallback to JavaScript is intentional because a text-like unknown snippet is more likely to be safely syntax-checked by Node than treated as a compiler language with missing rules.

This function is called before filenames are validated, before tools are selected, before code is beautified, and before compile actions are chosen. A wrong value here would ripple through the whole compile feature, so the function keeps the rest of the code from repeatedly solving the same naming problem.

How detectCodeLanguageFromSource works inside this feature

detectCodeLanguageFromSource is the compile workspace's fallback reader. It first trusts filename evidence because extensions are usually intentional. If that is not enough, it scans the text for language-specific patterns.

This is the implementation at the point where the feature needs it.

```

function detectCodeLanguageFromSource(code = "", fileName = "") {
  const bvName = languageFromFileName(fileName, code);
  if (bvName) return bvName;
  const source = String(code || "");
  if (/<\/?[a-z1-5]*?>/i.test(source) || /<!doctype\s+html/i.test(source)) return "html";
  if (
    /\b(always_ff|always_comb|always_latch|logic|interface|covergroup|assert\s+property|typedef\s+enum|class\s+w+)\b/.test(source))
    return "systemverilog";
  if (/b(module|endmodule|always|assign|reg|wire|initial|posedge|negedge)\b/.test(source)) return "verilog";
  if (/^s*(?!(tran|ac|dc|loop|model|subckt|ends|param)\b)/i.test(source) ||
    /\bVWw\s+w\s+w\s+w\s+(?:DC|SIN|PULSE)?/i.test(source)) return "ltspice";
  if (/b(def|el|if|from\s+w\s+import|self|None|True|False)\b/.test(source)) return "python";
  if (/b(public\s+class|static\s+void\s+main|System\.out)\b/.test(source)) return "java";
  if (sourceLooksCpp(source) || sourceLooksC(source) ||
    /\b(#include|main\s*(|puts\s*(|fprintf|sizeof\s*(|)\b/.test(source)) {
    return sourceLooksCpp(source) ? "cpp" : "c";
  }
  if (/b(const|let|var|function|=>|console\.log|document\.|window\.)\b/.test(source)) return "javascript";
  return "javascript";
}

```

The order of checks matters. HTML is detected early because angle-bracket markup can otherwise look like operators. SystemVerilog is checked before Verilog because SystemVerilog contains many Verilog words plus richer constructs like logic, always_ff, interfaces, classes, and covergroups.

The C and C++ check is split through sourceLooksCpp and sourceLooksC so that includes and common library features can steer the choice. If no strong evidence appears, the function lands on JavaScript, which is a practical default for a browser-centered builder.

Finding compilers on Windows

findTool is the bridge between an IDE button and the messy reality of installed tools. It checks cached answers first, tries known candidates, asks the operating system where a command lives, and searches likely install folders for compilers that Windows PATH may not expose.

The cache matters because tool detection can become slow if every compile click scans the same folders again. Once gcc, g++, javac, java, iverilog, vvp, node, or python is found, the backend remembers the path.

This is also why the function returns an empty string instead of pretending a missing tool exists. The UI can show a clear missing-tool message rather than letting a compile command fail mysteriously.

How compileToolCache shapes this workflow

compileToolCache remembers resolved compiler paths. Once the backend finds gcc, g++, javac, java, iverilog, vvp, node, python, or another tool, future compile clicks can reuse the result.

This is a responsiveness feature. Tool discovery can involve shell commands and filesystem searches, and the UI should not pay that cost on every run.

The code below is the actual shared value used by this feature group.

```
const compileToolCache = new Map();
```

How compileToolVersionCache shapes this workflow

compileToolVersionCache stores version-output results. It supports tool status panels without repeatedly spawning every compiler just to display the same version string.

The code below is the actual shared value used by this feature group.

```
const compileToolVersionCache = new Map();
```

How findTool works inside this feature

findTool is the compiler and Git discovery engine. It starts with compileToolCache because a tool path that was found once should not be rediscovered on every compile click.

This is the implementation at the point where the feature needs it.

```
asvnc function findTool(toolName) {
  if (compileToolCache.has(toolName)) return compileToolCache.get(toolName);
  const candidates = (compileToolCandidates[toolName] || [toolName]).filter(Boolean);
  for (const candidate of candidates) {
    if (path.isAbsolute(candidate)) {
      if (await pathExists(candidate)) {
        compileToolCache.set(toolName, candidate);
        return candidate;
      }
      continue;
    }
  }
  trv {
    const command = process.platform === "win32" ? "where" : "command";
    const args = process.platform === "win32" ? [candidate] : ["-v", candidate];
    const result = await execFileAsvnc(command, args, { timeout: 5000, windowsHide: true });
    const found = String(result.stdout || "").split(/\r?\n/).map(line => line.trim()).find(Boolean);
    if (found) {
      compileToolCache.set(toolName, found);
      return found;
    }
  }
  catch {
    // Trv the next candidate.
  }
}

if ([ "gcc", "g++" ].includes(toolName) && process.env.LOCALAPPDATA) {
  const found = await findExecutableUnder(
    path.join(process.env.LOCALAPPDATA, "Microsoft", "WinGet", "Packages"),
    [toolName === "gcc" ? "gcc.exe" : "g++.exe"],
    7
  );
  compileToolCache.set(toolName, found || "");
  return found || "";
}
```

```

}

if (!["javac", "java"].includes(toolName)) {
  const found = await findExecutableUnder("C:\\Program Files\\Eclipse Adoptium", `${toolName}.exe`, 5);
  compileToolCache.set(toolName, found || "");
  return found || "";
}

compileToolCache.set(toolName, "");
return "";
}

```

The function then loops through known candidates. Some candidates are ordinary command names, which `execFile` can test through the operating system. Other candidates are explicit Windows paths, which can be checked directly. This dual approach handles both clean developer machines and ordinary user machines where `PATH` may not include every tool.

If the obvious candidates fail, the helper performs targeted searches under likely installation folders. It then stores the resolved path in `compileToolCache`. Returning an empty string is useful too: the UI can tell the user exactly which compiler or simulator is missing rather than letting a later command fail with a cryptic message.

Running external programs

`runProcess` is the terminal engine. It starts a program, passes arguments, captures stdout and stderr, tracks elapsed time, handles timeouts, and returns a structured result.

This function is where JavaScript leaves pure language behavior and coordinates with the operating system. The backend is no longer just calculating values; it is supervising another process.

The result object is designed for the UI. Terminal text can be shown immediately, messages can be logged, and errors can be explained without asking the user to inspect a hidden PowerShell window.

How `runProcess` works inside this feature

`runProcess` is the backend's controlled way to run an external program. It uses `spawn` rather than string-built shell commands, so the executable and argument list stay separated.

This is the implementation at the point where the feature needs it.

```

async function runProcess(command, args = [], options = {})

```

The function creates a promise around the child process life cycle. It records the start time, starts a timeout, gathers stdout and stderr through data events, optionally writes stdin, and resolves only after the process closes or errors.

The Windows `taskkill` branch is there because compiler or simulator processes can spawn children. Killing only the parent may leave a stuck subprocess alive. The function therefore tries to stop the whole process tree before marking the run as timed out.

The returned object has the fields the rest of the compile system needs: `ok`, `code`, `stdout`, `stderr`, `timedOut`, and `elapsedMs`. By returning a normalized object, the compile coordinator can treat Node, Python, gcc, javac, iverilog, and vvp with the same display logic.

Building the compile workspace

Before a compiler can run, the backend has to turn the UI's workspace payload into real files. `compileWorkspaceFilesFromPayload` normalizes the requested language, checks filenames, and collects the source files that should exist for the operation.

`writeCompileWorkspaceSources` then creates the project workspace and writes those files to disk. This separation is useful: one function interprets the incoming data, while the other performs the filesystem side effect.

For languages like C, C++, Verilog, and SystemVerilog, this matters because one file may include or instantiate another. The project is treated as a workspace rather than a single pasted text box.

How `compileWorkspaceFilesFromPayload` works inside this feature

`compileWorkspaceFilesFromPayload` turns the browser's workspace payload into a clean list of files. It accepts multiple workspace files, adds the active file, ignores empty code, normalizes language and role, and gives each item a safe id and filename.

This is the implementation at the point where the feature needs it.

```
function compileWorkspaceFilesFromPayload(payload = {})
```

The result is sorted by filename so build order and terminal display are predictable. More importantly, this function treats Compile Code as a workspace, not a single text area. That is what lets one file include, import, instantiate, or call another.

How `writeCompileWorkspaceSources` works inside this feature

`writeCompileWorkspaceSources` materializes the workspace on disk. It creates the target directory, optionally filters by languages or extensions, prevents filename collisions by appending a count, writes each source file, and returns the written file records.

This is the implementation at the point where the feature needs it.

```
async function writeCompileWorkspaceSources(files = [], targetDir = "", options = {})
```

This is the moment where in-memory editor state becomes a real compiler workspace. Returning `sourcePath` and `uniqueName` gives later commands the exact files that were actually written, not merely the files the browser intended to write.

Compile, build, run, simulate, and scope

`compileAndRunCode` is the feature coordinator. It receives the compile payload, prepares the workspace, chooses the toolchain, builds command arguments, runs the compiler or simulator, and returns output plus artifacts.

For Verilog and SystemVerilog, simulation depends on a testbench and vvp output. The backend can parse VCD waveform text so the frontend can show a signal scope rather than only plain terminal output.

The important feature boundary is clear: Add Code stores presentable code in the portfolio, while Compile Code creates a runnable local workspace. Later, Append code to project can move successful source into the selected project overview as formatted content.

How `compileAndRunCode` works inside this feature

`compileAndRunCode` is the compiler feature's main conductor. It receives the browser payload, decides the language and action, saves the active source, builds a run workspace, resolves tools,

executes compile/build/run/simulate steps, and returns terminal text plus artifacts.

This is the implementation at the point where the feature needs it.

```
async function compileAndRunCode(payload = {})
```

The early part of the function prepares shared state: language, profile, safe filename, saved metadata, source path, action, workspace files, stdin, and terminal lines. These variables are the common vocabulary used by every language branch.

The nested ensureRunSource function is a local helper because it belongs only to this workflow. It creates a unique run directory once, writes the selected workspace files, finds the active source path, and copies the saved source if the active file was not already written. Keeping it inside compileAndRunCode makes it clear that run directories are temporary execution spaces, not permanent project records.

Each language branch then maps the user's requested action to real commands. HTML performs lightweight validation. JavaScript runs node --check and optionally node. Python runs the interpreter. Java compiles with javac and runs the detected class. C and C++ compile to binaries. Verilog and SystemVerilog use iverilog and vvp, then attempt waveform parsing.

The function's return object is shaped for the frontend. It includes ok, language, saved metadata, terminal output, and language-specific extras such as artifacts or waveform information. That makes the Compile, Build, Run, Simulate, Show Output, Messages Log, and Scope UI possible without each button inventing a different backend contract.

How parseVcdScopeText works inside this feature

parseVcdScopeText is the waveform parser behind the scope icon. It reads VCD text line by line, captures timescale, follows scope nesting, records signal definitions, then collects value changes after definitions end.

This is the implementation at the point where the feature needs it.

```
function parseVcdScopeText(text = "", source = "waveform.vcd") {
  const lines = String(text || "").split(/\r?\n/);
  const scopes = [];
  const signalsBvCode = new Map();
  const timeScaleParts = [];
  let inTimescale = false;
  let time = 0;
  let maxTime = 0;
  let definitionDone = false;
  for (const rawLine of lines) {
    const line = rawLine.trim();
    if (!line) continue;
    if (line.startsWith("$timescale")) {
      inTimescale = true;
      const inline = line.replace("$timescale", "").replace("$end", "").trim();
      if (inline) timeScaleParts.push(inline);
      continue;
    }
    if (inTimescale) {
      if (line === "$end") {
        inTimescale = false;
      } else {
        timeScaleParts.push(line.replace("$end", "").trim());
        if (line.includes("$end")) inTimescale = false;
      }
      continue;
    }
    const scopeMatch = line.match(/^$scope\s+\S+\s+(.+?)\s+$end$/);
    if (scopeMatch) {
      scopes.push(scopeMatch[1]);
      continue;
    }
  }
}
```

```

}
if (/^\$unscone\b/.test(line)) {
  scopes.pop();
  continue;
}
const varMatch = line.match(/^\$var\s+\S+\s+(\d+)\s+(\S+)\s+(.+?)\s+\$end$/);
if (varMatch) {
  const [, width, code, reference] = varMatch;
  if (!signalsBvCode.has(code) && signalsBvCode.size < 64) {
    const cleanReference = reference.replace(/\s+\[^\]]+\]/, "");
    signalsBvCode.set(code, {
      code,
      name: [...scopes, cleanReference].filter(Boolean).join("."),
      reference: cleanReference,
      width: Number(width) || 1,
      changes: []
    });
  }
  continue;
}
if (line.startsWith("$enddefinitions")) {
  definitionDone = true;
  continue;
}
if (!definitionDone) continue;
if (line.startsWith("#")) {
  time = Number(line.slice(1)) || 0;
  maxTime = Math.max(maxTime, time);
  continue;
}
let code = "";
let value = "";
const vectorMatch = line.match(/^(br|01xz |+)\s+(\S+)\$/i);
if (vectorMatch) {
  value = normalizeVcdValue(vectorMatch[1]);
  code = vectorMatch[2];
} else if (/^01xz/i.test(line)) {
  value = normalizeVcdValue(line[0]);
  code = line.slice(1).trim();
}
const signal = signalsBvCode.get(code);
if (!signal || !value) continue;
const previous = signal.changes[signal.changes.length - 1];
if (!previous || previous.value !== value || previous.time !== time) {
  if (signal.changes.length < 600) signal.changes.push({ time, value });
}
}
const signals = [...signalsBvCode.values()]
  .filter((signal) => signal.changes.length)
  .slice(0, 32);
return {
  source: path.basename(source),
  timeScale: timeScaleParts.join(" ").replace(/\s+g, " ").trim() || "ticks",
  maxTime,
  signals
};
}

```

The parser limits signal count and change count. That is a practical UI decision: a waveform file can be enormous, but the portfolio builder scope needs a responsive educational view, not a full commercial simulator database.

The returned object contains the source filename, timescale, maximum time, and a list of signals with changes. That is exactly the shape a frontend scope view needs to draw signal names and transitions over time.

How installCompilerTools works inside this feature

installCompilerTools is the assisted setup route for compilers. It looks at the selected language profile, reads its winget install hints, and starts the appropriate installer command when possible.

This is the implementation at the point where the feature needs it.

```

asvnc function installCompilerTools(language = "") {
  const lang = normalizeCodeLanguage(language);
  const profile = compileLanguageProfiles[lang] || compileLanguageProfiles.javascript;
  if (!profile.winget?.length) {
    return {
      ok: false,
      language: lang,
      terminal: `${profile.label} does not have an automatic Winget compiler install configured. Install the
        required tools manually, then restart the builder.`
    };
  }
  const winget = await findTool("winget") || "winget";
  const terminal = [];
  let ok = true;
  for (const packageId of profile.winget) {
    const args = [
      "install",
      "--source",
      "winget",
      "--accept-source-agreements",
      "--accept-package-agreements",
      "--silent",
      "--id",
      packageId,
      "--exact"
    ];
    const result = await runProcess(winget, args, { cwd: root, timeoutMs: 600000 });
    terminal.push(terminalLine(`winget ${args.join(" ")}`.processTerminalText(result)));
    ok = ok && result.ok;
  }
  compileToolCache.clear();
  return { ok, language: lang, terminal: terminal.join("\n\n"), tools: await compileToolStatus() };
}

```

The function does not claim that every language needs installation. JavaScript may use the bundled Node runtime, HTML needs no compiler, and some tools may already be found. The helper exists for languages where a missing external compiler would otherwise leave the user stuck.

Language detection before compiler selection

The compile workspace cannot choose a compiler until it has a normalized language. The detection path starts with explicit language values, then looks at filenames, then inspects the source text. This lets a pasted SystemVerilog module, a Java class, a C header, or a Python snippet land in the right toolchain more often without making the user fight the interface.

This group is deliberately layered. `normalizeCodeLanguage` converts human variants into internal ids. `languageFromFileName` uses extensions. `detectCodeLanguageFromSource` looks for syntax clues. `safeCodeFileName` then checks that the final filename extension makes sense for the selected language. Together, they keep the compile workspace forgiving at the input edge and strict before writing files.

How compileLanguageProfiles shapes this workflow

`compileLanguageProfiles` is the compile workspace's language table. Each entry names the default file, legal extensions, display label, required tools, and installer hints for one language mode.

The object is easier to maintain than scattered if statements. Adding a new compile language later should start here because this is where the frontend's language choice becomes backend rules.

The code below is the actual shared value used by this feature group.

```

const compileLanguageProfiles = {
  c: {
    defaultFile: "main.c",
    extensions: [".c", ".h"],
    label: "C",
    primaryTools: ["gcc"],

```

```

    winget: ["BrechtSanders.WinLibs.POSIX.UCRT"]
  },
  cpp: {
    defaultFile: "main.cpp",
    extensions: [".cpp", ".cc", ".cxx", ".hpp", ".h"],
    label: "C++",
    primaryTools: ["g++"],
    winget: ["BrechtSanders.WinLibs.POSIX.UCRT"]
  },
  verilog: {
    defaultFile: "design.v",
    extensions: [".v"],
    label: "Verilog",
    primaryTools: ["iverilog", "vvp"],
    winget: ["Icarus.Verilog"]
  },
  systemverilog: {
    defaultFile: "design.sv",
    extensions: [".sv", ".svh"],
    label: "SystemVerilog",
    primaryTools: ["iverilog", "vvp"],
    winget: ["Icarus.Verilog"]
  },
  ltspice: {
    defaultFile: "simulation.cir",
    extensions: [".cir", ".net", ".sp", ".asc"],
    label: "LTspice",
    primaryTools: ["ltspice"],
    winget: []
  },
  java: {
    defaultFile: "Main.java",
    extensions: [".java"],
    label: "Java",
    primaryTools: ["javac", "java"],
    winget: ["EclipseAdoptium.Temurin.21.JDK"]
  },
  javascript: {
    defaultFile: "main.js",
    extensions: [".js", ".mjs", ".cjs"],
    label: "JavaScript",
    primaryTools: ["node"],
    winget: []
  },
  python: {
    defaultFile: "main.py",
    extensions: [".py"],
    label: "Python",
    primaryTools: ["python"],
    winget: []
  },
  html: {
    defaultFile: "index.html",
    extensions: [".html", ".htm"],
    label: "HTML",
    primaryTools: [],
    winget: []
  }
}
};

```

How normalizeCodeLanguage works inside this feature

normalizeCodeLanguage translates messy user-facing language names into the internal language keys used by the compile workspace. A person may type C++, cpp, JavaScript, node, sv, spice, or py; the compiler feature needs one stable key.

This is the implementation at the point where the feature needs it.

```

function normalizeCodeLanguage(value = "") {
  const clean = String(value || "").trim().toLowerCase().replace(/[ \s-]+/g, "");
  const aliases = {
    c: "c",
    cpp: "cpp",
    colusolus: "cpp",
    "c++": "cpp",

```

```

svsystemverilog: "svsystemverilog".
sv: "svsystemverilog".
verilog: "verilog".
v: "verilog".
ltspace: "ltspace".
space: "ltspace".
cir: "ltspace".
iava: "java".
javascript: "javascript".
is: "javascript".
node: "javascript".
nvthon: "python".
ov: "python".
html: "html".
htm: "html"
}:
return aliases[clean] || (compileLanguageProfiles[clean] ? clean : "javascript");
}

```

The aliases object is the polite front door. It accepts common spellings and abbreviations, then maps them to compileLanguageProfiles. The final fallback to JavaScript is intentional because a text-like unknown snippet is more likely to be safely syntax-checked by Node than treated as a compiler language with missing rules.

This function is called before filenames are validated, before tools are selected, before code is beautified, and before compile actions are chosen. A wrong value here would ripple through the whole compile feature, so the function keeps the rest of the code from repeatedly solving the same naming problem.

How sourceLooksCpp works inside this feature

sourceLooksCpp is a heuristic reader. It scans the source for C++-specific includes and language marks such as std::, cout, templates, classes, constexpr, nullptr, and namespace usage.

This is the implementation at the point where the feature needs it.

```

function sourceLooksCpp(source = "") {
  const text = String(source || "");
  return
    /#include\s*<(?:iostream|string|vector|arravl|map|unordered map|memory|algorithm|optional|variant)>/.test(text)
    ||

    /\b(std::|cout|cin|cerr|namespace|template\s*<|class\s+\w+|new\s+\w+|delete\s+|constexpr|nullptr|using\s+namespace)\b/.test(text):
}

```

The function is needed because header files are ambiguous. A .h file may be C or C++, and the builder has to choose whether gcc or g++ should become the likely tool. The heuristic is not a formal parser, but it is good enough to avoid many frustrating wrong guesses.

How sourceLooksC works inside this feature

sourceLooksC performs the same kind of rough reading for C. It looks for standard C headers and functions such as printf, scanf, malloc, free, structs, and typedef struct patterns.

This is the implementation at the point where the feature needs it.

```

function sourceLooksC(source = "") {
  const text = String(source || "");
  return /#include\s*<(?:stdio|stdlib|string|stdint|stdbool|math)\.h>/.test(text) ||
    /\b(printf|scanf|malloc|calloc|realloc|free|struct\s+\w+|typedef\s+struct)\b/.test(text):
}

```

When both C and C++ could be plausible, the caller can compare these heuristics and choose the safer profile. This is part of making Compile Code feel like an IDE instead of a dumb text box.

How languageFromFileName works inside this feature

languageFromFileName treats the filename as a clue. The extension is compared against the supported language profiles, which lets main.cpp, design.sv, Main.java, index.html, and simulation.cir select the correct mode without inspecting the entire body first.

This is the implementation at the point where the feature needs it.

```
function languageFromFileName(fileName = "", code = "") {
  const ext = path.extname(String(fileName || "").toLowerCase());
  if (ext === ".h") {
    if (sourceLooksCpp(code)) return "cpp";
    if (sourceLooksC(code)) return "c";
    return "c";
  }
  return Object.entries(compileLanguageProfiles).find(([, profile]) => profile.extensions.includes(ext))[0] ||
    "";
}
```

The .h branch is special because header files are shared by C and C++. The function asks the source heuristics to resolve that ambiguity. If no strong C++ clue exists, it returns C, which is the more conservative answer for a plain .h file.

How detectCodeLanguageFromSource works inside this feature

detectCodeLanguageFromSource is the compile workspace's fallback reader. It first trusts filename evidence because extensions are usually intentional. If that is not enough, it scans the text for language-specific patterns.

This is the implementation at the point where the feature needs it.

```
function detectCodeLanguageFromSource(code = "", fileName = "") {
  const bvName = languageFromFileName(fileName, code);
  if (bvName) return bvName;
  const source = String(code || "");
  if (/<\/?[a-z]\s\S\*?>/i.test(source) || /<!doctype\s+html/i.test(source)) return "html";
  if (
    (/b(always_ff|always_comb|always_latch|logic|interface|covergroup|assert\s+property|typedef\s+enum|class\s+\w+)\b/.test(source))
    return "systemverilog";
  if (/b(module|endmodule|always|assign|reg|wire|initial|posedge|negedge)\b/.test(source)) return "verilog";
  if (/^s*\.(?!(tran|ac|dc|loop|model|subckt|ends|param))\b/im.test(source) ||
    /\bVw+\s+\w+\s+\w+\s+(?:DC|SIN|PULSE)?/i.test(source)) return "ltspice";
  if (/b(def|elif|from\s+\w+\s+import|self|None|True|False)\b/.test(source)) return "python";
  if (/b(public\s+class|static\s+void\s+main|System.out)\b/.test(source)) return "java";
  if (sourceLooksCpp(source) || sourceLooksC(source) ||
    /\b(#include|main\s*\(|puts\s*\(|fprintf|sizeof\s*\(\)\b/.test(source)) {
    return sourceLooksCpp(source) ? "cpp" : "c";
  }
  if (/b(const|let|var|function|=>|console\.log|document\.|window\.)\b/.test(source)) return "javascript";
  return "javascript";
}
```

The order of checks matters. HTML is detected early because angle-bracket markup can otherwise look like operators. SystemVerilog is checked before Verilog because SystemVerilog contains many Verilog words plus richer constructs like logic, always_ff, interfaces, classes, and covergroups.

The C and C++ check is split through sourceLooksCpp and sourceLooksC so that includes and common library features can steer the choice. If no strong evidence appears, the function lands on JavaScript, which is a practical default for a browser-centered builder.

How safeCodeFileName works inside this feature

safeCodeFileName turns a requested code filename into something the compile workspace can safely write. It keeps the name readable through safeSegment, but it forces the extension to belong to the selected language profile.

This is the implementation at the point where the feature needs it.

```
function safeCodeFileName(value = "", language = "javascript") {
  const profile = compileLanguageProfiles[normalizeCodeLanguage(language)] || compileLanguageProfiles.javascript;
  const fallback = profile.defaultFile;
  const parsed = path.parse(String(value || fallback));
  const safeName = safeSegment(parsed.name, path.parse(fallback).name);
  let ext = String(parsed.ext || path.extname(fallback)).toLowerCase();
  if (!profile.extensions.includes(ext)) ext = path.extname(fallback);
  return `${safeName}${ext}`;
}
```

That extension enforcement prevents a mismatch like saving SystemVerilog as .txt or Java as .cpp after the language has already been selected. The fallback default file comes from compileLanguageProfiles, so each language keeps a natural filename even when the user leaves the field blank.

Beautifying code without becoming a full IDE parser

The beautifier is intentionally modest. It normalizes line endings, replaces tabs, performs brace-based indentation for C-like and HDL-like languages, and applies a simple HTML line split. It is not trying to replace clang-format, prettier, or a full language server.

That tradeoff is appropriate for this builder. Add Code needs presentable source blocks inside a portfolio, while Compile Code needs runnable files. `beautifyCode` improves readability for common pasted snippets without risking heavy dependencies or surprising rewrites.

How indentBraceCode works inside this feature

`indentBraceCode` is a lightweight formatter for brace-shaped languages. It walks one line at a time, lowers indentation before a closing brace or bracket, writes the trimmed line with the current indentation depth, then adjusts depth based on opening and closing delimiters.

This is the implementation at the point where the feature needs it.

```
function indentBraceCode(code = "") {
  let depth = 0;
  return String(code || "")
    .replace(/\\r\\n?/g, "\\n")
    .split("\\n")
    .map((rawLine) => {
      const line = rawLine.trim();
      if (!line) return "";
      if (/^(\\{\\}|/\\.test(line)) depth = Math.max(0, depth - 1);
      const formatted = `${" ".repeat(depth)}${line}`;
      const opens = (line.match(/\\{\\|/g) || []).length;
      const closes = (line.match(/\\}\\|/g) || []).length;
      depth = Math.max(0, depth + opens - closes);
      return formatted;
    })
}
```

This does not pretend to be a full compiler-aware formatter. It is a portfolio-friendly cleanup pass. Its value is that pasted code becomes easier to read quickly without introducing a heavy dependency or trying to rewrite language semantics.

How beautifyCode works inside this feature

beautifyCode chooses the formatting strategy after normalizing line endings and tabs. C, C++, Java, JavaScript, Verilog, and SystemVerilog use the brace indentation helper. HTML receives a simple tag line break pass. Other languages keep trailing-space cleanup and blank-line compression.

This is the implementation at the point where the feature needs it.

```
function beautifyCode(code = "", language = "javascript") {
  const normalized = String(code || "").replace(/\r\n?/g, "\n").replace(/\t/g, " ");
```

```

const lang = normalizeCodeLanguage(language):
if ([ "c". "cpp". "java". "javascript". "verilog". "systemverilog" ].includes(lang)) {
  return indentBraceCode(normalized)
    .replace(/[\t]+$/gm, "")
    .replace(/\n{4,}/g, "\n\n\n");
}
if (lang === "html") {
  return normalized
    .replace(/>\s+</g, ">\n<")
    .split("\n")
    .map((line) => line.trim())
    .join("\n")
    .trim();
}
return normalized
  .split("\n")
  .map((line) => line.trimEnd())
  .join("\n")
  .replace(/\n{4,}/g, "\n\n\n")
  .trimEnd();
}

```

The function belongs in the backend because the same formatter can be used from the desktop app no matter which browser view or Electron window is active. It also lets the future parser reuse the same cleanup rule before code is appended into a project section.

C and C++ as compiled workspace languages

C and C++ need more than a source file. The backend chooses a standard, warning flags, an output binary name, and a run command. The C-family helpers keep those decisions grouped so `compileAndRunCode` does not have to bury every flag directly inside its main body.

`cFamilyCompileProfile` chooses language-specific compiler flags. `cFamilyBinaryName` creates a predictable executable name. `cFamilyWorkspaceSources` decides which workspace files should participate. `cFamilyRunOutput` reshapes the compiler/run result for terminal display. These helpers make C and C++ feel like first-class IDE modes rather than one-off command strings.

How `cFamilyCompileProfile` works inside this feature

`cFamilyCompileProfile` builds the C or C++ compiler decision. It chooses the executable, language standard, warnings, and output arguments appropriate for the selected language and active file.

This is the implementation at the point where the feature needs it.

```

function cFamilyCompileProfile(language = "c", fileName = "main.c") {
  const isCpp = language === "cpp";
  const ext = path.extname(String(fileName || "").toLowerCase());
  const header = [ ".h". ".hpp". ".hh". ".hxx" ].includes(ext);
  return {
    compiler: isCpp ? "g++" : "gcc",
    standard: isCpp ? "-std=c++20" : "-std=c17",
    languageFlag: isCpp ? "c++" : "c",
    label: isCpp ? "C++" : "C",
    header,
    flags: [
      isCpp ? "-std=c++20" : "-std=c17",
      "-Wall",
      "-Wextra",
      "-Wpedantic",
      "-Wshadow",
      "-O0",
      "-g3",
      "-fdiagnostics-color=never"
    ]
  };
}

```

The reason this lives outside `compileAndRunCode` is readability. C and C++ have enough special rules that burying flags directly in the main coordinator would make the central workflow harder to follow.

How `cFamilyBinaryName` works inside this feature

`cFamilyBinaryName` creates a predictable executable name from the source file. That keeps compiled artifacts understandable in the cache and terminal output.

This is the implementation at the point where the feature needs it.

```
function cFamilyBinaryName(fileName = "program") {
  const parsed = path.parse(String(fileName || "program"));
  return `${safeSegment(parsed.name, "program")}.exe`;
}
```

The filename is not only cosmetic. Later run commands need to know exactly which binary was produced, and stable naming avoids accidentally running an older artifact from a previous compile.

How `cFamilyWorkspaceSources` works inside this feature

`cFamilyWorkspaceSources` selects the files that should participate in a C or C++ build. It uses extension sets for each language and ensures the active file remains included if the extension filtering missed it.

This is the implementation at the point where the feature needs it.

```
function cFamilyWorkspaceSources(files = [], language = "c", activeFileName = "") {
  const exts = language === "cpp"
    ? new Set([".cpp", ".cc", ".cxx", ".c++", ".hpp", ".hh", ".hxx", ".h"])
    : new Set([".c", ".h"]);
  const selected = files.filter((file) => exts.has(path.extname(file.fileName).toLowerCase()));
  if (!selected.some((file) => file.fileName === activeFileName)) {
    selected.push(...files.filter((file) => file.fileName === activeFileName));
  }
  return selected;
}
```

This is what lets a C++ project compile several `.cpp` and `.h` files as a workspace instead of only the active editor buffer. It also avoids sending JavaScript or HDL files to gcc by accident.

How `cFamilyRunOutput` works inside this feature

`cFamilyRunOutput` translates a finished C or C++ program run into terminal text. It reports timeout or exit code first, then appends stdout and stderr or a clear no-output message.

This is the implementation at the point where the feature needs it.

```
function cFamilyRunOutput(result = {}) {
```

This is separate from generic process text because compiled programs are user-authored. A successful program with no output should feel different from a missing terminal field; the helper tells the user the run completed even when the code printed nothing.

HDL simulation as a design plus testbench workflow

Verilog and SystemVerilog are different from ordinary run buttons because a design usually needs a testbench to produce meaningful behavior. The HDL helpers separate design files, testbench files, waveform files, and simulator output. That matches the way digital hardware verification is actually done.

hdlFilesFromPayload reads the workspace and classifies files. writeHdlSimulationSources writes them into the cache directory. clearHdlWaveforms removes stale VCD files before a simulation. readHdlWaveform finds the new waveform after vvp runs. parseVcdScopeText turns that waveform text into signal data for the scope view.

How isHdlLanguage works inside this feature

isHdlLanguage answers a narrow but important question: does this language behave like hardware description code? Verilog and SystemVerilog need simulation roles, testbenches, VCD waveforms, and a simulator flow.

This is the implementation at the point where the feature needs it.

```
function isHdlLanguage(language = "") {
    return ["verilog", "systemverilog"].includes(normalizeCodeLanguage(language));
}
```

Keeping the check in one helper avoids scattering arrays of HDL names across compile logic. If VHDL is added later, this is one of the places the new mode would join the HDL path.

How inferCompileFileRole works inside this feature

inferCompileFileRole classifies HDL files as design or testbench. It reads both the filename and the code, looking for common testbench clues such as tb, testbench, dumpfile, dumpvars, and module names that begin like a bench.

This is the implementation at the point where the feature needs it.

```
function inferCompileFileRole(fileName = "", code = "", language = "") {
    if (!isHdlLanguage(language)) return "source";
    const havstack = `${fileName}\n${code}`.toLowerCase();
    if (
        (/\\b(tb|testbench)\\b|(\\b| -)tb(\\b| -|\\.|)test(\\b| -)?bench|\\$dumpfile|\\$dumpvars|module\\s+tb[ a-z0-9]*\\/i.test(havstack))
    ) {
        return "testbench";
    }
    return "design";
}
```

That role matters because a hardware simulation needs both the design being tested and the testbench driving it. The builder should not make the user manually label every file when the common clues are obvious.

How normalizeCompileFileRole works inside this feature

normalizeCompileFileRole turns user-facing role words into the internal design/testbench distinction. Non-HDL languages always return source because they do not need that split.

This is the implementation at the point where the feature needs it.

```
function normalizeCompileFileRole(value = "", language = "") {
    if (!isHdlLanguage(language)) return "source";
    const clean = String(value || "").trim().toLowerCase().replace(/\\s-|+\\/g, "");
    return ["tb", "testbench", "bench"].includes(clean) ? "testbench" : "design";
}
```

This helper lets the UI offer friendly terms while the simulator pipeline receives predictable values.

How hdlFilesFromPayload works inside this feature

hdlFilesFromPayload builds the HDL file set for simulation. It reads workspaceFiles plus the currently active file, rejects non-HDL or empty entries, normalizes filenames and roles, and stores the result in a Map keyed by file id.

This is the implementation at the point where the feature needs it.

```
function hdlFilesFromPayload(payload = {})
```

The Map prevents duplicate entries when the active file is also present in the workspace list. The final sort places design files before testbenches, which matches the mental model of compiling the circuit before the bench that drives it.

How writeHdlSimulationSources works inside this feature

writeHdlSimulationSources creates a clean HDL sources folder for a simulation run. It removes the previous sources directory, recreates it, writes unique filenames, and returns the written records.

This is the implementation at the point where the feature needs it.

```
asvnc function writeHdlSimulationSources(files = [], cacheDir = "") {
  const sourceDir = path.join(cacheDir, "sources");
  await rm(sourceDir, { recursive: true, force: true });
  await mkdir(sourceDir, { recursive: true });
  const seen = new Map();
  const written = [];
  for (const file of files) {
    const parsed = path.parse(file.fileName);
    const count = seen.get(file.fileName) || 0;
    seen.set(file.fileName, count + 1);
    const uniqueName = count ? `${parsed.name} ${count}${parsed.ext}` : file.fileName;
    const sourcePath = path.join(sourceDir, uniqueName);
    await writeFile(sourcePath, file.code, "utf8");
    written.push({ ...file, uniqueName, sourcePath });
  }
  return written;
}
```

The cleanup step is important. HDL simulations can be misleading if old files stay in the source folder and the simulator silently compiles stale modules from an earlier attempt.

How clearHdlWaveforms works inside this feature

clearHdlWaveforms removes old VCD files before a new simulation run. That prevents the scope from accidentally showing yesterday's waveform after today's simulation fails to produce one.

This is the implementation at the point where the feature needs it.

```
asvnc function clearHdlWaveforms(cacheDir = "") {
  const vcdFiles = await findFilesByExtension(cacheDir, ".vcd", 4);
  await Promise.all(vcdFiles.map((filePath) => rm(filePath, { force: true }).catch(() => {})));
}
```

The function deliberately ignores individual deletion errors. A stale file should not stop the whole simulation attempt, but the backend should still make a best effort to start clean.

How readHdlWaveform works inside this feature

readHdlWaveform searches the simulation cache for VCD files, chooses a short path as the likely main waveform, limits extremely large text, and passes it to parseVcdScopeText.

This is the implementation at the point where the feature needs it.

```
asvnc function readHdlWaveform(cacheDir = "") {
  const vcdFiles = await findFilesByExtension(cacheDir, ".vcd", 4);
  if (!vcdFiles.length) return null;
  const selected = vcdFiles.sort((a, b) => a.length - b.length)[0];
  const text = await readFile(selected, "utf8");
  const limited = text.length > 6000000 ? text.slice(0, 6000000) : text;
  return parseVcdScopeText(limited, selected);
}
```

This function is the bridge between the simulator's file output and the UI scope. Without it, a successful simulation would still leave the user hunting through folders for a waveform file.

How parseVcdScopeText works inside this feature

parseVcdScopeText is the waveform parser behind the scope icon. It reads VCD text line by line, captures timescale, follows scope nesting, records signal definitions, then collects value changes after definitions end.

This is the implementation at the point where the feature needs it.

```
function parseVcdScopeText(text = "", source = "waveform.vcd") {
  const lines = String(text || "").split(/\r?\n/);
  const scopes = [];
  const signalsBvCode = new Map();
  const timeScaleParts = [];
  let inTimescale = false;
  let time = 0;
  let maxTime = 0;
  let definitionDone = false;
  for (const rawLine of lines) {
    const line = rawLine.trim();
    if (!line) continue;
    if (line.startsWith("$timescale")) {
      inTimescale = true;
      const inline = line.replace("$timescale", "").replace("$end", "").trim();
      if (inline) timeScaleParts.push(inline);
      continue;
    }
    if (inTimescale) {
      if (line === "$end") {
        inTimescale = false;
      } else {
        timeScaleParts.push(line.replace("$end", "").trim());
        if (line.includes("$end")) inTimescale = false;
      }
      continue;
    }
    const scopeMatch = line.match(/^$scope\s+\S+(\.+)s+\$end$/);
    if (scopeMatch) {
      scopes.push(scopeMatch[1]);
      continue;
    }
    if (/^$upscope\b/.test(line)) {
      scopes.pop();
      continue;
    }
    const varMatch = line.match(/^$var\s+\S+(\d+)\s+(\S+)\s+(\.+)s+\$end$/);
    if (varMatch) {
      const [, width, code, reference] = varMatch;
      if (!signalsBvCode.has(code) && signalsBvCode.size < 64) {
        const cleanReference = reference.replace(/\s+\[^\]]+\]/, "");
        signalsBvCode.set(code, {
          code,
          name: [...scopes, cleanReference].filter(Boolean).join("."),
          reference: cleanReference,
          width: Number(width) || 1,
          changes: []
        });
      }
      continue;
    }
    if (line.startsWith("$enddefinitions")) {
      definitionDone = true;
      continue;
    }
    if (!definitionDone) continue;
    if (line.startsWith("#")) {
      time = Number(line.slice(1)) || 0;
      maxTime = Math.max(maxTime, time);
      continue;
    }
    let code = "";
    let value = "";
    const vectorMatch = line.match(/^([br][01xz_+])s+(\S+)$/i);
```

```

if (vectorMatch) {
  value = normalizeVcdValue(vectorMatch[1]);
  code = vectorMatch[2];
} else if (/^[01xz]/i.test(line)) {
  value = normalizeVcdValue(line[0]);
  code = line.slice(1).trim();
}
const signal = signalsBvCode.get(code);
if (!signal || !value) continue;
const previous = signal.changes[signal.changes.length - 1];
if (!previous || previous.value !== value || previous.time !== time) {
  if (signal.changes.length < 600) signal.changes.push({ time, value });
}
}
const signals = [...signalsBvCode.values()]
  .filter((signal) => signal.changes.length)
  .slice(0, 32);
return {
  source: path.basename(source),
  timeScale: timeScaleParts.join(" ").replace(/\s+/g, " ").trim() || "ticks",
  maxTime,
  signals
};
}

```

The parser limits signal count and change count. That is a practical UI decision: a waveform file can be enormous, but the portfolio builder scope needs a responsive educational view, not a full commercial simulator database.

The returned object contains the source filename, timescale, maximum time, and a list of signals with changes. That is exactly the shape a frontend scope view needs to draw signal names and transitions over time.

Terminal output as the user's proof of execution

A compiler feature is not useful if the output appears only in a hidden process or only after the user clicks somewhere else. The backend captures stdout, stderr, exit code, elapsed time, generated artifacts, and timeout state so the frontend terminal can repaint immediately.

The terminal helpers are small but important. `terminalLine` gives command output a readable shell-like heading. `processTerminalText` produces a standard status block. `replacePathReferences` can shorten machine-specific paths in displayed output. `cleanHdlSimulationOutput` removes simulator noise so HDL results are easier to read.

How `terminalLine` works inside this feature

`terminalLine` gives the terminal output a readable rhythm. It places a label above a block of command text so the frontend can show which command produced which stdout or stderr.

This is the implementation at the point where the feature needs it.

```

function terminalLine(label, text = "") {
  return [`$ ${label}`.trim(), String(text || "").trim()].filter(Boolean).join("\n");
}

```

That small formatting choice matters when a compile action runs several commands. A JavaScript build may syntax-check multiple files, and C or HDL flows may compile, then run. The terminal needs separators that feel like a real development console.

How `processTerminalText` works inside this feature

`processTerminalText` turns the raw result from `runProcess` into terminal prose. It combines exit status, timeout state, stdout, and stderr into a single display string.

This is the implementation at the point where the feature needs it.

```
function processTerminalText(result = {})
```

The helper is the reason compile output can be shown immediately and consistently. The UI does not have to understand the result object's internals; it can render the terminal field and the messages log from a stable string shape.

How pathVariantsForReplacement works inside this feature

pathVariantsForReplacement prepares several spellings of a filesystem path. Windows paths, slash-normalized paths, and basename forms can all appear in compiler output.

This is the implementation at the point where the feature needs it.

```
function pathVariantsForReplacement(value = "") {
  const clean = String(value || "");
  return Arrav.from(new Set([
    clean,
    nath.normalize(clean),
    clean.replace(/\\/g, "/"),
    path.normalize(clean).replace(/\\/g, "/")
  ]).filter(Boolean));
}
```

The compile terminal uses these variants to shorten or normalize paths before showing them to the user. That keeps output focused on project files instead of long local-machine folders.

How replacePathReferences works inside this feature

replacePathReferences takes terminal text and replaces known path spellings with cleaner labels. It is a readability helper, not a security boundary.

This is the implementation at the point where the feature needs it.

```
function replacePathReferences(text = "", replacements = []) {
  let output = String(text || "");
  for (const replacement of replacements) {
    const from = replacement?.from || "";
    const to = replacement?.to || "";
    if (!from || !to) continue;
    for (const variant of pathVariantsForReplacement(from)) {
      output = output.split(variant).join(to);
    }
  }
  return output;
}
```

The function is useful after compilers report errors with absolute paths. A beginner should see the file that failed and the compiler message, not be distracted by a long AppData or compile-cache path.

How processTerminalTextWithPaths works inside this feature

processTerminalTextWithPaths combines the normal terminal formatter with path replacement. It first builds the terminal text from the process result, then cleans path references against the replacements supplied by the compile workflow.

This is the implementation at the point where the feature needs it.

```
function processTerminalTextWithPaths(result = {})
```

This is a good example of helper composition. One helper understands process status, another understands path shortening, and the combined helper gives the UI a polished message.

How cleanHdlSimulationOutput works inside this feature

`cleanHdlSimulationOutput` reshapes simulator output into something readable. HDL simulators often include finish lines, file references, and mixed stdout/stderr text. The helper separates signal-like output from finish notices and reports simulation status at the top.

This is the implementation at the point where the feature needs it.

```
function cleanHdlSimulationOutput(result = {})
```

For a hardware learner, this matters because simulation is about behavior over time. The terminal should make it obvious whether the testbench completed, timed out, or produced useful printed values.

How `runProcess` works inside this feature

`runProcess` is the backend's controlled way to run an external program. It uses `spawn` rather than string-built shell commands, so the executable and argument list stay separated.

This is the implementation at the point where the feature needs it.

```
async function runProcess(command, args = [], options = {})
```

The function creates a promise around the child process life cycle. It records the start time, starts a timeout, gathers stdout and stderr through data events, optionally writes stdin, and resolves only after the process closes or errors.

The Windows `taskkill` branch is there because compiler or simulator processes can spawn children. Killing only the parent may leave a stuck subprocess alive. The function therefore tries to stop the whole process tree before marking the run as timed out.

The returned object has the fields the rest of the compile system needs: `ok`, `code`, `stdout`, `stderr`, `timedOut`, and `elapsedMs`. By returning a normalized object, the compile coordinator can treat Node, Python, gcc, javac, iverilog, and vvp with the same display logic.

The Compile button as the full coordinator

`compileAndRunCode` is the main compile feature because it joins all the smaller pieces. It interprets the action, normalizes language, writes workspace files, finds tools, chooses commands, uses caches when possible, runs compile/build/run/simulate actions, collects terminal output, and returns a result the UI can render.

The function is long because the feature is genuinely multi-stage. A shorter version would likely hide important behavior in vague helpers or push compiler details into the browser. The current shape keeps privileged execution in Node while still organizing language-specific details in smaller support functions.

How `compileToolCandidates` shapes this workflow

`compileToolCandidates` is the Windows-aware executable search table. It records command names and likely install paths for compilers, simulators, interpreters, and LTspice.

This object exists because Windows machines often have tools installed but not on PATH. The builder should be helpful enough to find common install locations before telling the user a compiler is missing.

The code below is the actual shared value used by this feature group.

```
const compileToolCandidates = {  
  gcc: [  

```

```

    "gcc".
    process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Microsoft", "WinGet", "Packages",
      "BrechtSanders.WinLibs.POSIX.UCRT Microsoft.Winget.Source 8wekvb3d8bbwe", "mingw64", "bin", "gcc.exe") :
    ""
  ],
  "g++": [
    "g++".
    "c++".
    process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Microsoft", "WinGet", "Packages",
      "BrechtSanders.WinLibs.POSIX.UCRT Microsoft.Winget.Source 8wekvb3d8bbwe", "mingw64", "bin", "g++.exe") :
    ""
  ],
  clang: ["clang"].
  "clang++": ["clang++"].
  cl: ["cl"].
  iavac: [
    "iavac".
    "C:\\Program Files\\Eclipse Adoptium\\jdk-21.0.11.10-hotspot\\bin\\iavac.exe"
  ],
  iava: [
    "iava".
    "C:\\Program Files\\Eclipse Adoptium\\jdk-21.0.11.10-hotspot\\bin\\iava.exe"
  ],
  node: [
    /(?:^|[/\\])node(?:\.exe)?$/i.test(process.execPath || "") ? process.execPath : "",
    "node".
    "C:\\Program Files\\nodejs\\node.exe".
    process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Programs", "nodejs", "node.exe") : ""
  ],
  python: [process.env.PYTHON, "python", "py"].
  iverilog: ["iverilog", "C:\\iverilog\\bin\\iverilog.exe"].
  vvp: ["vvp", "C:\\iverilog\\bin\\vvp.exe"].
  ltspice: [
    process.env.LTSPICE_EXE.
    "C:\\Program Files\\ADI\\LTspice\\LTspice.exe".
    "C:\\Program Files\\LTC\\LTspiceXVII\\XVIIx64.exe".
    "C:\\Program Files\\Analog Devices\\LTspice\\LTspice.exe".
    process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Programs", "ADI", "LTspice", "LTspice.exe") :
    ""
  ],
  ],
};

```

How compileToolCache shapes this workflow

compileToolCache remembers resolved compiler paths. Once the backend finds gcc, g++, javac, java, iverilog, vvp, node, python, or another tool, future compile clicks can reuse the result.

This is a responsiveness feature. Tool discovery can involve shell commands and filesystem searches, and the UI should not pay that cost on every run.

The code below is the actual shared value used by this feature group.

```
const compileToolCache = new Map();
```

How compileToolVersionCache shapes this workflow

compileToolVersionCache stores version-output results. It supports tool status panels without repeatedly spawning every compiler just to display the same version string.

The code below is the actual shared value used by this feature group.

```
const compileToolVersionCache = new Map();
```

How compileActionFromPayload works inside this feature

compileActionFromPayload decides what the user's button means. It accepts compile, build, run, and simulate when explicitly provided, then chooses a default based on language.

This is the implementation at the point where the feature needs it.

```
function compileActionFromPayload(value = "", language = "") {
  const clean = String(value || "").trim().toLowerCase().replace(/[_\s-]+/g, "-");

```

```

if (!["compile", "build", "run", "simulate"].includes(clean)) return clean;
return isHdlLanguage(language) ? "simulate" : "run";
}

```

The default is intentionally different for HDL. Verilog and SystemVerilog should simulate by default because a design file alone does not behave like a software program with a main function.

How compileActionLabel works inside this feature

compileActionLabel converts the internal action into text the terminal can show. This is where run becomes Run, build becomes Build project, and HDL defaults become Simulate.

This is the implementation at the point where the feature needs it.

```

function compileActionLabel(action = "run", language = "") {
  if (action === "compile") return "Compile";
  if (action === "build") return "Build project";
  if (action === "simulate") return "Simulate";
  return isHdlLanguage(language) ? "Simulate" : "Run";
}

```

Small labeling helpers reduce UI inconsistency. The backend and frontend can display the same action wording instead of inventing labels separately.

How compileWorkspaceFilesFromPayload works inside this feature

compileWorkspaceFilesFromPayload turns the browser's workspace payload into a clean list of files. It accepts multiple workspace files, adds the active file, ignores empty code, normalizes language and role, and gives each item a safe id and filename.

This is the implementation at the point where the feature needs it.

```

function compileWorkspaceFilesFromPayload(payload = {})

```

The result is sorted by filename so build order and terminal display are predictable. More importantly, this function treats Compile Code as a workspace, not a single text area. That is what lets one file include, import, instantiate, or call another.

How writeCompileWorkspaceSources works inside this feature

writeCompileWorkspaceSources materializes the workspace on disk. It creates the target directory, optionally filters by languages or extensions, prevents filename collisions by appending a count, writes each source file, and returns the written file records.

This is the implementation at the point where the feature needs it.

```

async function writeCompileWorkspaceSources(files = [], targetDir = "", options = {})

```

This is the moment where in-memory editor state becomes a real compiler workspace. Returning sourcePath and uniqueName gives later commands the exact files that were actually written, not merely the files the browser intended to write.

How compileAndRunCode works inside this feature

compileAndRunCode is the compiler feature's main conductor. It receives the browser payload, decides the language and action, saves the active source, builds a run workspace, resolves tools, executes compile/build/run/simulate steps, and returns terminal text plus artifacts.

This is the implementation at the point where the feature needs it.

```

async function compileAndRunCode(payload = {})

```

The early part of the function prepares shared state: language, profile, safe filename, saved metadata, source path, action, workspace files, stdin, and terminal lines. These variables are the common vocabulary used by every language branch.

The nested `ensureRunSource` function is a local helper because it belongs only to this workflow. It creates a unique run directory once, writes the selected workspace files, finds the active source path, and copies the saved source if the active file was not already written. Keeping it inside `compileAndRunCode` makes it clear that run directories are temporary execution spaces, not permanent project records.

Each language branch then maps the user's requested action to real commands. HTML performs lightweight validation. JavaScript runs `node --check` and optionally `node`. Python runs the interpreter. Java compiles with `javac` and runs the detected class. C and C++ compile to binaries. Verilog and SystemVerilog use `iverilog` and `vvp`, then attempt waveform parsing.

The function's return object is shaped for the frontend. It includes `ok`, language, saved metadata, terminal output, and language-specific extras such as artifacts or waveform information. That makes the Compile, Build, Run, Simulate, Show Output, Messages Log, and Scope UI possible without each button inventing a different backend contract.

How `installCompilerTools` works inside this feature

`installCompilerTools` is the assisted setup route for compilers. It looks at the selected language profile, reads its winget install hints, and starts the appropriate installer command when possible.

This is the implementation at the point where the feature needs it.

```
asvnc function installCompilerTools(language = "") {
  const lang = normalizeCodeLanguage(language);
  const profile = compileLanguageProfiles[lang] || compileLanguageProfiles.javascript;
  if (!profile.winget?.length) {
    return {
      ok: false,
      language: lang,
      terminal: `${profile.label} does not have an automatic Winget compiler install configured. Install the
        required tools manually, then restart the builder.`
    };
  }
  const winget = await findTool("winget") || "winget";
  const terminal = [];
  let ok = true;
  for (const packageId of profile.winget) {
    const args = [
      "install",
      "--source",
      "winget",
      "--accept-source-agreements",
      "--accept-package-agreements",
      "--silent",
      "--id",
      packageId,
      "--exact"
    ];
    const result = await runProcess(winget, args, { cwd: root, timeoutMs: 60000 });
    terminal.push(terminalLine(`winget ${args.join(" ")}`, processTerminalText(result)));
    ok = ok && result.ok;
  }
  compileToolCache.clear();
  return { ok, language: lang, terminal: terminal.join("\n\n"), tools: await compileToolStatus() };
}
```

The function does not claim that every language needs installation. JavaScript may use the bundled Node runtime, HTML needs no compiler, and some tools may already be found. The helper exists for languages where a missing external compiler would otherwise leave the user stuck.

Chapter 6: Publishing, Authentication, And Loading From Target

The publishing target contract

Publishing target setup is not just saving a URL. The builder must know that the repository URL is valid, that Git can reach it, that the current user has write access, and that the chosen branch is the branch the website should use.

`validatePublishRemoteUrl` prevents obviously unsafe or unsupported target values from entering the workflow. `getPublishTargetInfo` reports what the current workspace knows about the target.

The user-facing distinction is important: Authenticate target verifies write access. Load from target imports compatible website files from that repository. Those actions should not be blurred together because one grants trust and the other moves content.

How defaultSiteRepository shapes this workflow

`defaultSiteRepository` is a deployment-time hint, not proof of authorization. It lets packaged builds carry a preferred website repository into messages and setup flows.

The backend still requires explicit target authentication before public website files can be changed. A default repository only improves guidance; it does not replace GitHub write verification.

The code below is the actual shared value used by this feature group.

```
const defaultSiteRepository = process.env.OMB_BUILDER_REPOSITORY || "";
```

How gitCandidates shapes this workflow

`gitCandidates` is the Git discovery table. It includes plain git plus common Windows install paths so publishing can work even when PATH is not perfectly configured.

The code below is the actual shared value used by this feature group.

```
const gitCandidates = [
  process.env.GIT_EXE,
  "git",
  "C:\\Program Files\\Git\\cmd\\git.exe",
  "C:\\Program Files\\Git\\bin\\git.exe",
  "C:\\Program Files (x86)\\Git\\cmd\\git.exe",
  "C:\\Program Files (x86)\\Git\\bin\\git.exe",
  process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Programs", "Git", "cmd", "git.exe") : "",
  process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Programs", "Git", "bin", "git.exe") : ""
].filter(Boolean);
```

How publishAuthorizationHelp shapes this workflow

`publishAuthorizationHelp` is the long explanation returned when publishing cannot proceed. It is centralized so every failed publishing path teaches the same distinction between Authenticate target, Load from target, and Apply to site.

Keeping this help text beside the authorization constants matters because publishing failures are user-facing. A clear backend error becomes a useful dialog instead of a vague Git failure.

The code below is the actual shared value used by this feature group.

```
const publishAuthorizationHelp = [
  "Publishing was blocked before live website files were applied.",
  "Open Publishing target. enter the repository URL. then click Authenticate target.",
  "A GitHub/Git Credential Manager browser sign-in may open. Sign in with an account that has write access to the selected Pages repository.",
  "Authenticate target only verifies write access and remembers it for the matching repository and branch.",
  "Load from target only imports compatible website files from the authenticated repository into the local builder workspace.",
]
```

```
"Until a compatible writable website repository is associated, the builder remains local-only."
].join(" ");
```

How validatePublishRemoteUrl works inside this feature

validatePublishRemoteUrl checks that a target repository URL is shaped like a supported GitHub remote. It rejects empty or unsafe values before Git commands run.

This is the implementation at the point where the feature needs it.

```
function validatePublishRemoteUrl(value = "") {
  const remoteUrl = String(value || "").trim();
  if (!remoteUrl) return "";
  if (/^git@github\.com:[A-Za-z0-9.-]+\.[A-Za-z0-9.-]+(?:\.git)?$/i.test(remoteUrl)) return remoteUrl;
  try {
    const parsed = new URL(remoteUrl);
    if (parsed.protocol !== "https:" || parsed.hostname.toLowerCase() !== "github.com") {
      throw new Error("Only GitHub HTTPS repository URLs are supported by the guided setup.");
    }
    const parts = parsed.pathname.replace(/^\/+/, "").split("/");
    if (parts.length < 2 || !parts[0] || !parts[1]) {
      throw new Error("GitHub repository URL must include owner and repository name.");
    }
    return `https://github.com/${parts[0]}/${parts[1].replace(/\.git$/i, "")}.git`;
  } catch (error) {
    if (error.message?.includes("Only GitHub") || error.message?.includes("owner")) throw error;
    throw new Error("Enter a GitHub repository URL such as https://github.com/USERNAME/USERNAME.github.io.git.");
  }
}
```

This validation makes target setup fail early with a clear message. It is better to reject an unsupported target before changing remotes, writing caches, or asking Git Credential Manager to authenticate.

How getPublishTargetInfo works inside this feature

getPublishTargetInfo collects the current publishing state: whether the workspace is a Git repository, the origin remote, current branch, repository display name, custom domain, and authorization cache status.

This is the implementation at the point where the feature needs it.

```
async function getPublishTargetInfo() {
  const info = {
    workspace: root,
    portfolioWorkspace: portfolioRoot,
    separatedWorkspace: !samePath(root, portfolioRoot),
    defaultRepository: defaultSiteRepository,
    gitBacked: false,
    compatible: await workspaceHasCompatibleSiteFiles(),
    remote: "",
    branch: "",
    repository: "",
    customDomain: ""
  };

  try {
    info.gitBacked = (await runPublishGit(["rev-parse", "--is-inside-work-tree"])).stdout.trim() === "true";
  } catch {
    return info;
  }

  try {
    info.remote = remoteUrlForDisplay((await runPublishGit(["remote", "get-url", "origin"])).stdout.trim());
  } catch {
    info.remote = "";
  }

  try {
    info.branch = (await runPublishGit(["branch", "--show-current"])).stdout.trim();
  } catch {
    info.branch = "";
  }
}
```

```

}

const parsedRemote = parseGitHubRemote(info.remote);
info.repository = parsedRemote ? `${parsedRemote.owner}/${parsedRemote.repo}` : info.remote;

try {
  info.customDomain = (await readFile(resolveInsidePortfolioRoot("CNAME"), "utf8")).trim();
} catch {
  info.customDomain = "";
}

const accessShape = {
  branch: info.branch,
  remote: info.remote,
  repository: info.repository
};

const cached = await readPublishAuthCache();
if (publishAuthCacheIsFresh(cached, accessShape)) {
  info.authorizationChecked = true;
  info.authorizationCached = true;
  info.checkedAt = cached.checkedAt;
  info.expiresAt = publishAuthCacheExpiresAt(cached);
  info.extendedTrust = Boolean(cached.extendedTrust);
  info.successCountLastWeek = cached.successCountLastWeek || 0;
  info.trustDays = cached.trustDays || (cached.extendedTrust ? 30 : 1);
} else {
  info.authorizationChecked = false;
  info.authorizationCached = false;
  info.checkedAt = "";
  info.expiresAt = "";
  info.extendedTrust = false;
  info.successCountLastWeek = cached?.successCountLastWeek || 0;
  info.trustDays = 0;
}

return info;
}

```

The frontend uses this object to explain where Apply to site will push and whether GitHub authorization is currently fresh. It is an information route, but it still matters for security because it keeps target state explicit before writes happen.

Authentication caching

The app should not ask GitHub to authenticate every time it opens. `writePublishAuthCache` stores a local trust record for the specific repository and branch after successful verification.

`assertPublishAccess` reads that record and decides whether the current publishing operation can proceed. The cache has a normal one-day window and an extended window when repeated successful authorizations prove the same device and target are consistently used.

This design balances convenience and safety. It avoids repeated prompts during normal work, but it does not grant permanent trust to an old machine state.

How `publishAuthCacheTtlMs` shapes this workflow

`publishAuthCacheTtlMs` is the normal daily trust window. It implements the user's desired behavior that GitHub authorization should be remembered for about a day, not requested every time the app reopens.

The code below is the actual shared value used by this feature group.

```
const publishAuthCacheTtlMs = 24 * 60 * 60 * 1000;
```

How `publishAuthExtendedTtlMs` shapes this workflow

`publishAuthExtendedTtlMs` is the longer trust window used after repeated successful authorizations. It is a convenience layer for a known owner and target, not a permanent bypass.

The code below is the actual shared value used by this feature group.

```
const publishAuthExtendedTtlMs = 30 * 24 * 60 * 60 * 1000;
```

How `publishAuthHistoryWindowMs` shapes this workflow

`publishAuthHistoryWindowMs` defines how far back successful authorizations count toward extended trust. The cache logic uses this to decide whether the same target has been verified repeatedly within a meaningful recent period.

The code below is the actual shared value used by this feature group.

```
const publishAuthHistoryWindowMs = 7 * 24 * 60 * 60 * 1000;
```

How `publishAuthExtendedThreshold` shapes this workflow

`publishAuthExtendedThreshold` is the number of successful authorizations needed before the longer trust window applies. It turns the security rule into an auditable constant.

The code below is the actual shared value used by this feature group.

```
const publishAuthExtendedThreshold = 3;
```

How `writePublishAuthCache` works inside this feature

`writePublishAuthCache` records a successful publishing authorization for the current remote, branch, repository, and device scope.

This is the implementation at the point where the feature needs it.

```
async function writePublishAuthCache(access = {})
```

The function also maintains success history. If the same target is authorized repeatedly within the recent window, it can grant extended trust for up to thirty days. That implements the owner's requested behavior without making authentication permanent.

After writing the JSON file, the helper attempts `chmod 600`. On Windows this may not map perfectly to Unix permissions, but the intent is clear: the authorization cache is local owner state, not a public website asset.

How `assertPublishAccess` works inside this feature

`assertPublishAccess` is the gatekeeper for live website changes. It first proves the portfolio workspace is a Git repository, then reads origin and current branch, then checks compatibility unless the caller intentionally disables that requirement.

This is the implementation at the point where the feature needs it.

```
async function assertPublishAccess(options = {})
```

After building the access shape, it tries the authorization cache. A fresh matching cache returns immediately with `authorizationCached` true. That is the daily-login behavior the user wanted: reopening the app should not force GitHub authorization again when the target is unchanged.

If the cache is missing or expired, the function synchronizes the local branch from remote, ensures there is a HEAD commit, performs the dry-run write check, writes a new auth cache, and returns a detailed access object. The function both protects Apply to site and supplies the UI with exactly what was verified.

How authenticateGitHubForTarget works inside this feature

authenticateGitHubForTarget is the guided target-authentication workflow. It validates the repository URL and optional credentials, ensures the workspace is a Git repo, captures the old state, sets the new origin, checks Git/Git Credential Manager, selects the target branch, and then verifies write access.

This is the implementation at the point where the feature needs it.

```
async function authenticateGitHubForTarget(options = {})
```

The cache shortcut is important. If the same target is already authenticated and the user did not provide new credentials, the function can return cached authorization after writing any requested domain change. That is how authentication persists across app restarts.

If explicit credentials are supplied, storeGitCredentials approves them through Git. If not, the function launches Git Credential Manager's GitHub login flow. Only after that does assertPublishAccess force the dry-run write check and write the cache.

The catch block restores the previous target state. This is what keeps a failed authentication attempt from leaving the builder pointed at an incomplete or unauthorized repository.

Loading from target

Load from target is a pull/import feature. It should run only after authentication has proven that the selected repository is writable by the user. That prevents a random public repository from being treated as the user's editable website.

syncFromPublishTarget pulls compatible website files into the local builder workspace. The purpose is to let a newly installed builder recover the latest site state from GitHub after the owner authenticates.

The function belongs in the backend because Git operations, folder synchronization, and compatibility checks require filesystem and process access that the browser should not own directly.

How syncFromPublishTarget works inside this feature

syncFromPublishTarget is Load from target. It first calls assertPublishAccess with compatibility relaxed, because a newly selected repository may need to be inspected before the local builder knows whether it has the expected files.

This is the implementation at the point where the feature needs it.

```
async function syncFromPublishTarget() {
  const publishAccess = await assertPublishAccess({ requireCompatible: false });
  const branch = publishAccess.branch || "main";
  const remote = (await runPublishGit(["remote", "get-url", "origin"])).stdout.trim();
  const importRoot = await mkdtemp(path.join(os.tmpdir(), "omb-publish-target-"));
  const cloneRoot = path.join(importRoot, "target");
  const importPaths = [
    "projects.json",
    "assets",
    "docs",
    "Backgrounds",
    "CNAME",
    ".nojekyll",
    "robots.txt",
    "sitemap.xml"
  ];

  try {
    await runGit(["clone", "--depth", "1", "--branch", branch, remote, cloneRoot], {
      cwd: importRoot,
    });
  }
```

```

    timeout: 3 * 60 * 1000
  });

  const availablePaths = [];
  for (const importPath of importPaths) {
    try {
      await fsAccess(nath.join(cloneRoot, importPath));
      availablePaths.push(importPath);
    } catch {
      // Optional path is absent in the selected site repository.
    }
  }

  if (!availablePaths.includes("projects.json")) {
    throw publishAccessError(
      "The selected repository does not contain a projects.json portfolio catalog.",
      "The target repository must contain a compatible OMB Portfolio Builder catalog before it can be imported.",
      publishAccess
    );
  }

  const backupRoot = resolveInsideRoot(
    ".omb-backups",
    `load-target-${new Date().toISOString().replace(/[:.]/g, "-")}`
  );
  await mkdir(backupRoot, { recursive: true });

  for (const importPath of availablePaths) {
    const sourcePath = path.join(cloneRoot, importPath);
    const targetPath = resolveInsideRoot(importPath);
    try {
      await fsAccess(targetPath);
      await cp(targetPath, path.join(backupRoot, importPath), { recursive: true, force: true });
      await rm(targetPath, { recursive: true, force: true });
    } catch {
      // Nothing local to back up for this path.
    }
    await cp(sourcePath, targetPath, { recursive: true, force: true });
  }

  const importedCatalog = await readFile(nath.join(cloneRoot, "projects.json"), "utf8");
  await writeFile(draftPath, importedCatalog, "utf8");
  const portfolioSvnc = await svncPortfolioPublishFiles({ removeMissing: false });

  return {
    ...publishAccess,
    imported: availablePaths,
    portfolioSvnc,
    backup: path.relative(root, backupRoot),
    draftUpdated: true
  };
} finally {
  await rm(importRoot, { recursive: true, force: true });
}
}

```

The function clones the selected branch into a temporary folder, checks for importable paths, requires projects.json, backs up existing local files, copies target files into the builder workspace, writes the imported catalog into the local draft, and synchronizes publish files.

The temporary clone is removed in a finally block. That cleanup matters because target imports can include assets and documents, and failed imports should not litter the temp directory with old repository copies.

Applying the site

publishSiteChanges is where local portfolio content becomes public repository content. It synchronizes the publish branch, copies only allowlisted website files into the publish mirror, bumps assets when needed, stages changes, commits if anything changed, and pushes.

The allowlist is the heart of the safety model. The builder has internal files, drafts, compile artifacts, docs, caches, and update support. Only the website-safe paths should be staged for the public site.

The returned object tells the UI whether a commit was created, whether a push happened, which branch was used, and what Git output came back.

How `publishPaths` shapes this workflow

`publishPaths` is a public-output allowlist. It tells the publisher which files and folders are allowed to become part of the live website.

This list is one of the most important safety objects in the backend. Without it, a broad git add could accidentally publish drafts, credentials, compile outputs, local caches, or builder-only implementation files.

The code below is the actual shared value used by this feature group.

```
const publishPaths = [
  "projects.json",
  "docs",
  "assets",
  "index.html",
  "styles.css",
  "script.js",
  "electronics-search.js",
  "Backgrounds",
  ".noiekvll",
  "headers",
  ".well-known",
  "robots.txt",
  "sitemap.xml",
  "CNAME"
];
```

How `syncPortfolioPublishFiles` works inside this feature

`syncPortfolioPublishFiles` copies public website files from the builder root into the separated portfolio workspace when those roots are different.

This is the implementation at the point where the feature needs it.

```
async function syncPortfolioPublishFiles(options = {})
```

It only copies paths from `publishPaths`. That allowlist is the central protection against publishing builder internals, local drafts, compile caches, credentials, or generated documentation that should not belong to the public site.

The `removeMissing` option controls whether missing source files should delete the corresponding target. Apply to site uses removal so the live website reflects deleted assets. Load from target can avoid aggressive deletion during import.

How `publishSiteChanges` works inside this feature

`publishSiteChanges` turns a saved catalog and public asset set into a Git push. It obtains or receives verified publish access, synchronizes the branch from remote, copies publishable files, bumps asset versions, filters stageable paths, stages them, commits only when there are changes, and pushes.

This is the implementation at the point where the feature needs it.

```
async function publishSiteChanges(publishAccess = null) {
  const access = publishAccess || await assertPublishAccess();
  const branch = access.branch || (await runPublishGit(["branch", "--show-current"])).stdout.trim();
  const remoteSvnc = await synchronizePublishBranchFromRemote(branch, access);
  const portfolioSync = await syncPortfolioPublishFiles({ removeMissing: true });
```

```

await bumpPublishedAssetVersions(portfolioRoot):

const stageablePaths = await stageablePublishPaths(publishPaths);
if (!stageablePaths.length) {
  throw new Error("No compatible site files were available to publish.");
}

await runPublishGit(["add", "-A", "--", ...stageablePaths]);
const status = await runPublishGit(["status", "--porcelain", "--", ...stageablePaths]);
const hasChanges = status.stdout.trim().length > 0;

let commit = null;
if (hasChanges) {
  const message = `Update portfolio site ${new Date().toISOString().slice(0, 10)}`;
  commit = await runPublishGit(["commit", "-m", message]);
}

const pushArgs = branch ? ["push", "origin", branch] : ["push"];
const push = await runPublishGit(pushArgs);

return {
  ...access,
  portfolioWorkspace: portfolioRoot,
  remoteSvnc,
  portfolioSvnc,
  branch: branch || "current branch",
  committed: hasChanges,
  commitOutput: commit?.stdout || commit?.stderr || "",
  pushed: true,
  pushOutput: push.stdout || push.stderr || ""
};
}

```

The function is intentionally conservative. It refuses to publish when no compatible files are stageable. It avoids empty commits. It returns commit and push output so the success dialog can show real Git evidence.

The ordering solves the earlier non-fast-forward problem: synchronize first, copy local public files second, stage and commit third, push last. That mirrors the safe human workflow for updating a GitHub Pages repository.

Git commands as backend-only side effects

Git operations are not done in the browser. `runGit`, `runPublishGit`, `runGitWithInput`, and `runOptionalCommand` are the backend's command layer around Git. They keep command execution, working directory, input, output, and failure text in one controlled area.

This grouping matters because publishing touches a real repository. If the browser assembled Git commands directly, it would mix UI state with privileged side effects. Keeping Git in the backend lets the UI ask for Authenticate target, Load from target, or Apply to site while the server handles the command details.

How `gitCandidates` shapes this workflow

`gitCandidates` is the Git discovery table. It includes plain git plus common Windows install paths so publishing can work even when `PATH` is not perfectly configured.

The code below is the actual shared value used by this feature group.

```

const gitCandidates = [
  process.env.GIT_EXE,
  "git",
  "C:\\Program Files\\Git\\cmd\\git.exe",
  "C:\\Program Files\\Git\\bin\\git.exe",
  "C:\\Program Files (x86)\\Git\\cmd\\git.exe",
  "C:\\Program Files (x86)\\Git\\bin\\git.exe",
  process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Programs", "Git", "cmd", "git.exe") : "",

```

```
process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Programs", "Git", "bin", "git.exe") : ""
].filter(Boolean);
```

How runGit works inside this feature

runGit is the general Git command wrapper. It tries each candidate executable until one works, runs the requested argument list, and returns stdout, stderr, and the Git path used.

This is the implementation at the point where the feature needs it.

```
async function runGit(args, options = {})
```

The candidate loop is Windows-friendly. Some machines have git on PATH, others have it installed in Program Files or LocalAppData. The wrapper lets the rest of the backend call Git without caring where it was installed.

How runPublishGit works inside this feature

runPublishGit is runGit with the publishing workspace as its default current directory. That single default prevents many accidental app-repository versus website-repository mistakes.

This is the implementation at the point where the feature needs it.

```
async function runPublishGit(args, options = {})
```

Whenever a command is meant to inspect, sync, stage, commit, or push the website target, this wrapper communicates that intent.

How runOptionalCommand works inside this feature

runOptionalCommand executes a command but returns an ok object instead of throwing. It is used for checks where failure is information rather than a fatal condition.

This is the implementation at the point where the feature needs it.

```
async function runOptionalCommand(command, args = [], options = {})
```

System checks and installer discovery benefit from this shape. The UI can report that winget or Git Credential Manager is missing without crashing the whole backend request.

How runGitWithInput works inside this feature

runGitWithInput runs Git commands that need stdin. Credential approval is the main example: Git expects protocol, host, path, username, and password lines through standard input.

This is the implementation at the point where the feature needs it.

```
async function runGitWithInput(args, input, options = {})
```

This helper mirrors runGit's candidate search but uses spawn so it can write to stdin and collect stdout/stderr manually. That is why credential storage can work without constructing unsafe shell strings.

How gitFailureText works inside this feature

gitFailureText extracts the most useful text from a failed Git command. Git errors may arrive through stderr, stdout, or the Error message itself.

This is the implementation at the point where the feature needs it.

```
function gitFailureText(error) {
  return [
    error?.stderr,
    error?.stdout,
    error?.message
  ]
```

```
l.filter(Boolean).join("\n").trim():
}
```

The helper improves dialogs after authentication, synchronization, or push failures. The user sees the actual Git reason rather than a generic JavaScript exception.

How `getGitStatus` works inside this feature

`getGitStatus` builds the system-readiness report for publishing. It checks Git, Git Credential Manager, the bundled Node runtime, pnpm developer requirements, and the application version.

This is the implementation at the point where the feature needs it.

```
asvnc function getGitStatus() {
  let git = { ok: false, stdout: "", stderr: "", error: "Git for Windows was not found." };
  try {
    const gitResult = await runGit(["--version"]);
    git = { ok: true, stdout: gitResult.stdout || "", stderr: gitResult.stderr || "" };
  } catch (error) {
    git = { ok: false, stdout: error.stdout || "", stderr: error.stderr || "", error: error.message || "Git for
      Windows was not found." };
  }
  let credentialManager = { ok: false, version: "", error: "Git Credential Manager was not found." };
  if (git.ok) {
    let gcm = { ok: false, stdout: "", stderr: "", error: "Git Credential Manager was not found." };
    try {
      const gcmResult = await runGit(["credential-manager", "--version"]);
      gcm = { ok: true, stdout: gcmResult.stdout || "", stderr: gcmResult.stderr || "" };
    } catch (error) {
      gcm = { ok: false, stdout: error.stdout || "", stderr: error.stderr || "", error: error.message || "Git
        Credential Manager was not found." };
    }
    credentialManager = {
      ok: gcm.ok,
      version: (gcm.stdout || gcm.stderr || "").trim(),
      error: gcm.ok ? "" : (gcm.stderr || gcm.error || "Git Credential Manager was not found.")
    };
  }
  return {
    git: {
      ok: git.ok,
      version: (git.stdout || git.stderr || "").trim(),
      error: git.ok ? "" : (git.stderr || git.error || "Git for Windows was not found.")
    },
    credentialManager,
    nodeRuntime: {
      ok: true,
      version: process.versions.node,
      bundled: true,
      note: "Bundled inside the desktop app. No manual Node.js install is required."
    },
    pnpm: {
      ok: true,
      required: false,
      note: "pnpm is only needed by developers building the installer from source."
    },
    app: {
      version: process.env.OMB_APP_VERSION || packageJson.version || "0.0.0",
      desktopApp: process.env.OMB_DESKTOP_APP === "1"
    }
  };
}
```

This function turns machine setup into structured UI data. Instead of asking the user to run shell commands, the builder can show which dependency is present, missing, or only needed for developers.

Target setup as a reversible transaction

Configuring a publishing target is risky because the builder may need to create or alter a local Git remote, check out a branch, write a CNAME, and verify write access. The code treats that as something that should be reversible if verification fails.

`capturePublishTargetState` records the current repository state before the attempt. `configurePublishTarget` applies the requested target. If access verification fails, `restorePublishTargetState` can put the workspace back. This is the kind of defensive design that prevents a failed authentication attempt from leaving the local repo half-reconfigured.

How `capturePublishTargetState` works inside this feature

`capturePublishTargetState` records the current remote, branch, and CNAME state before target authentication changes anything.

This is the implementation at the point where the feature needs it.

```
async function capturePublishTargetState() {
  const snapshot = {
    branch: "",
    customDomainExists: false,
    customDomainText: "",
    remote: "",
    remoteExists: false
  };
  try {
    snapshot.remote = (await runPublishGit(["remote", "get-url", "origin"])).stdout.trim();
    snapshot.remoteExists = true;
  } catch {
    snapshot.remote = "";
    snapshot.remoteExists = false;
  }
  try {
    snapshot.branch = (await runPublishGit(["branch", "--show-current"])).stdout.trim();
  } catch {
    snapshot.branch = "";
  }
  try {
    snapshot.customDomainText = await readFile(resolveInsidePortfolioRoot("CNAME"), "utf8");
    snapshot.customDomainExists = true;
  } catch {
    snapshot.customDomainText = "";
    snapshot.customDomainExists = false;
  }
  return snapshot;
}
```

It exists because target setup is reversible. If authentication fails after changing origin or branch, the backend can attempt to restore the previous workspace state instead of leaving the user half-switched to a bad target.

How `configurePublishTarget` works inside this feature

`configurePublishTarget` is the older target setup path that configures repository URL, optional custom domain, and optional credentials, then reports target information.

This is the implementation at the point where the feature needs it.

```
async function configurePublishTarget(options = {})
```

The current UI steers users toward Authenticate target because configuration without write verification caused confusion. The function remains useful as a lower-level setup routine and a historical bridge in the backend.

How `restorePublishTargetState` works inside this feature

restorePublishTargetState attempts to put the publishing workspace back the way it was after a failed target setup. It restores origin, branch, and custom-domain file state on a best-effort basis.

This is the implementation at the point where the feature needs it.

```
async function restorePublishTargetState(snapshot = {})
```

The function intentionally catches rollback errors. The original authentication failure is the important error for the user; rollback attempts should not hide it.

How writeTargetCustomDomain works inside this feature

writeTargetCustomDomain writes or removes CNAME in the portfolio workspace, and mirrors it to the app root when those roots differ.

This is the implementation at the point where the feature needs it.

```
async function writeTargetCustomDomain(domain, customDomainProvided) {
  const targetPaths = [resolveInsidePortfolioRoot("CNAME")];
  if (!samePath(root, portfolioRoot)) targetPaths.push(resolveInsideRoot("CNAME"));
  if (customDomainProvided && domain) {
    for (const targetPath of targetPaths) await writeFile(targetPath, `${domain}\n`, "utf8");
  } else if (customDomainProvided && !domain) {
    for (const targetPath of targetPaths) await rm(targetPath, { force: true });
  }
}
```

This lets the builder maintain custom-domain configuration as a publishable site file. The customDomainProvided flag distinguishes 'the user did not edit this field' from 'the user intentionally cleared the domain'.

How ensureGitRepository works inside this feature

ensureGitRepository makes sure the portfolio workspace is a Git work tree. It creates the folder, runs git init if needed, and creates a main branch when the repository has no named branch.

This is the implementation at the point where the feature needs it.

```
async function ensureGitRepository() {
  await mkdir(portfolioRoot, { recursive: true });
  try {
    const insideWorkTree = (await runPublishGit(["rev-parse", "--is-inside-work-tree"])).stdout.trim();
    if (insideWorkTree === "true") return;
  } catch {
    await runPublishGit(["init"]);
  }

  const branch = (await runPublishGit(["branch", "--show-current"])).stdout.trim();
  if (!branch) {
    await runPublishGit(["checkout", "-B", "main"]);
  }
}
```

This is setup plumbing for both existing targets and blank local workspaces. The builder cannot authenticate, load, or publish until the workspace can run Git commands.

Remote branch detection and synchronization

The builder should not assume every repository uses master, and it should not push over remote changes blindly. detectRemoteDefaultBranch, checkoutPublishBranch, ensurePublishHeadForWriteCheck, and synchronizePublishBranchFromRemote help the local workspace line up with the selected remote branch before writing.

This is the code path that addresses the kind of error where Git says the local branch is behind its remote counterpart. The publishing flow must fetch, inspect, checkout, and synchronize before the

final push can be trusted.

How `normalizeGitBranchName` works inside this feature

`normalizeGitBranchName` accepts a branch-like string only if it avoids characters and shapes that Git rejects or treats specially.

This is the implementation at the point where the feature needs it.

```
function normalizeGitBranchName(value = "") {
  const branch = String(value || "").trim();
  if (
    !branch ||
    branch.length > 180 ||
    branch.includes(".") ||
    branch.endsWith(".") ||
    branch.includes("@") ||
    /[\\:\s~^?*[\]\x00-\x1f]/.test(branch) ||
    branch.startsWith("/") ||
    branch.endsWith("/") ||
    branch.includes("//")
  ) {
    return "";
  }
  return branch;
}
```

Branch names become command arguments and cache keys, so the backend should not pass arbitrary text into Git. This helper catches invalid names before checkout, sync, or push.

How `detectRemoteDefaultBranch` works inside this feature

`detectRemoteDefaultBranch` asks the remote repository which branch HEAD points to. That is how the builder learns whether a target uses main, master, or another default branch.

This is the implementation at the point where the feature needs it.

```
async function detectRemoteDefaultBranch(remoteUrl = "") {
  if (!remoteUrl) return "";
  try {
    const result = await runPublishGit(["ls-remote", "--symref", remoteUrl, "HEAD"], { timeout: 45000 });
    const match = String(result.stdout || "").match(/^ref:~s+refs\/heads\/(.+?)~s+HEAD$/m);
    return normalizeGitBranchName(match?.[1] || "");
  } catch {
    return "";
  }
}
```

Using the remote's default avoids hardcoding branch assumptions during target setup. The function returns a normalized branch name or an empty string when detection fails.

How `originRemoteExists` works inside this feature

`originRemoteExists` checks whether the publishing workspace already has an origin remote. It returns a boolean instead of throwing, which makes `setOriginRemote` straightforward.

This is the implementation at the point where the feature needs it.

```
async function originRemoteExists() {
  try {
    await runPublishGit(["remote", "get-url", "origin"]);
    return true;
  } catch {
    return false;
  }
}
```

The helper is small, but it keeps remote setup readable: first ask whether origin exists, then set or add accordingly.

How setOriginRemote works inside this feature

setOriginRemote either changes the existing origin URL or creates it if it does not exist. This is the backend equivalent of connecting the builder to a website repository.

This is the implementation at the point where the feature needs it.

```
asvnc function setOriginRemote(remoteUrl) {
  if (await originRemoteExists()) {
    await runPublishGit(["remote", "set-url", "origin", remoteUrl]);
  } else {
    await runPublishGit(["remote", "add", "origin", remoteUrl]);
  }
}
```

The function does not authenticate by itself. It only changes Git configuration. Authentication and write verification happen later through authenticateGitHubForTarget and assertPublishAccess.

How checkoutPublishBranch works inside this feature

checkoutPublishBranch switches the publishing workspace to the selected branch, creating it locally if necessary.

This is the implementation at the point where the feature needs it.

```
asvnc function checkoutPublishBranch(branch) {
  const cleanBranch = normalizeGitBranchName(branch) || "main";
  const current = (await runPublishGit(["branch", "--show-current"])).stdout.trim();
  if (current === cleanBranch) return cleanBranch;
  try {
    await runPublishGit(["checkout", cleanBranch]);
  } catch {
    await runPublishGit(["checkout", "-B", cleanBranch]);
  }
  return cleanBranch;
}
```

The clean branch name is important because this function passes the branch to Git. Returning the branch also gives callers a stable value to include in status responses.

How ensurePublishHeadForWriteCheck works inside this feature

ensurePublishHeadForWriteCheck makes sure the publishing workspace has a commit before dry-run write verification. A repository with no HEAD cannot push HEAD to a temporary branch.

This is the implementation at the point where the feature needs it.

```
asvnc function ensurePublishHeadForWriteCheck() {
  try {
    await runPublishGit(["rev-parse", "--verify", "HEAD"]);
    return { created: false };
  } catch {
    await runPublishGit([
      "-c",
      "user.name=OMB Portfolio Builder",
      "-c",
      "user.email=omb-builder@localhost",
      "commit",
      "--allow-emptv",
      "-m",
      "Initialize publish authorization check"
    ]);
    return { created: true };
  }
}
```

If no commit exists, the helper creates an empty initialization commit with local builder identity. That gives Git a concrete object to test during authorization.

How synchronizePublishBranchFromRemote works inside this feature

synchronizePublishBranchFromRemote makes the local publishing branch match the remote branch before writes happen. It checks whether the remote branch exists, fetches it, checks out the local branch, and hard-resets to origin/branch.

This is the implementation at the point where the feature needs it.

```
async function synchronizePublishBranchFromRemote(branch, access = {})
```

This is the answer to non-fast-forward failures. Before the builder writes new website files, it starts from the remote's current state so Apply to site does not push from behind.

Authentication cache scope and expiration

The authorization cache is not a global permission slip. It is scoped to the target repository, branch, and local conditions the code records. publishAuthCacheScope defines that scope. publishAuthCachesFresh then decides whether a stored session still matches the current access request.

The expiration logic gives the user convenience without turning a one-time sign-in into permanent trust. One day is the normal window. Repeated successful authorizations can extend trust for the same target, but the cache still has an expiration moment and a matching check.

How publishAuthCachePath shapes this workflow

publishAuthCachePath points to the local authorization session file. It is not a public website asset and should never be pushed as portfolio content.

The cache file lets the builder avoid asking for GitHub authorization on every Apply to site click while still limiting trust to a matching target and branch.

The code below is the actual shared value used by this feature group.

```
const publishAuthCachePath = path.join(root, ".omb-publish-session.json");
```

How publishAuthCacheTtlMs shapes this workflow

publishAuthCacheTtlMs is the normal daily trust window. It implements the user's desired behavior that GitHub authorization should be remembered for about a day, not requested every time the app reopens.

The code below is the actual shared value used by this feature group.

```
const publishAuthCacheTtlMs = 24 * 60 * 60 * 1000;
```

How publishAuthExtendedTtlMs shapes this workflow

publishAuthExtendedTtlMs is the longer trust window used after repeated successful authorizations. It is a convenience layer for a known owner and target, not a permanent bypass.

The code below is the actual shared value used by this feature group.

```
const publishAuthExtendedTtlMs = 30 * 24 * 60 * 60 * 1000;
```

How publishAuthHistoryWindowMs shapes this workflow

publishAuthHistoryWindowMs defines how far back successful authorizations count toward extended trust. The cache logic uses this to decide whether the same target has been verified repeatedly within a meaningful recent period.

The code below is the actual shared value used by this feature group.

```
const publishAuthHistoryWindowMs = 7 * 24 * 60 * 60 * 1000;
```

How publishAuthExtendedThreshold shapes this workflow

publishAuthExtendedThreshold is the number of successful authorizations needed before the longer trust window applies. It turns the security rule into an auditable constant.

The code below is the actual shared value used by this feature group.

```
const publishAuthExtendedThreshold = 3;
```

How readPublishAuthCache works inside this feature

readPublishAuthCache loads the local authorization session file and returns null when it cannot be read.

This is the implementation at the point where the feature needs it.

```
async function readPublishAuthCache() {
  try {
    return JSON.parse(await readFile(publishAuthCachePath, "utf8"));
  } catch {
    return null;
  }
}
```

Returning null is intentional. Missing auth cache is normal on a new device, after cleanup, or after a target change. The caller decides whether that absence means 'ask the user to authenticate' or simply 'show not authenticated'.

How writePublishAuthCache works inside this feature

writePublishAuthCache records a successful publishing authorization for the current remote, branch, repository, and device scope.

This is the implementation at the point where the feature needs it.

```
async function writePublishAuthCache(access = {}) {
```

The function also maintains success history. If the same target is authorized repeatedly within the recent window, it can grant extended trust for up to thirty days. That implements the owner's requested behavior without making authentication permanent.

After writing the JSON file, the helper attempts chmod 600. On Windows this may not map perfectly to Unix permissions, but the intent is clear: the authorization cache is local owner state, not a public website asset.

How publishAuthCacheScope works inside this feature

publishAuthCacheScope creates a hash from machine and workspace identity. The cache should not be valid merely because a remote and branch match; it should also belong to this device and installation context.

This is the implementation at the point where the feature needs it.

```
function publishAuthCacheScope() {
  let username = process.env.USERNAME || process.env.USER || "";
  try {
    username = os.userInfo().username || username;
  } catch {
    // Environment username is a sufficient fallback for cache scoping.
  }
  return createHash("sha256")
    .update([
      os.hostname(),
      username,
      root,
    ])
```

```

    portfolioRoot
    1.join("\n"))
    .digest("hex");
}

```

The hash avoids storing the raw username and path as the visible scope while still letting `publishAuthCachelsFresh` detect whether the cache was created for the same local environment.

How `parsePublishAuthTimestamp` works inside this feature

`parsePublishAuthTimestamp` converts stored ISO date strings into timestamps. It also normalizes over-precise fractional seconds so `Date.parse` can handle them.

This is the implementation at the point where the feature needs it.

```

function parsePublishAuthTimestamp(value = "") {
  const direct = Date.parse(String(value || ""));
  if (Number.isFinite(direct)) return direct;
  const normalized = String(value || "").replace(/(\.\d{3})\d+(Z|[-+]\d{2}:\d{2})$/i, "$1$2");
  return Date.parse(normalized);
}

```

This helper makes the auth cache more tolerant of timestamp formatting differences between versions without weakening the expiration rule.

How `publishAuthCacheExpiresAt` works inside this feature

`publishAuthCacheExpiresAt` calculates the expiration instant for a cache. It prefers an explicit `expiresAt` field, then falls back to `checkedAt` plus the normal TTL.

This is the implementation at the point where the feature needs it.

```

function publishAuthCacheExpiresAt(cache) {
  const explicitExpiresAt = parsePublishAuthTimestamp(cache?.expiresAt || "");
  if (Number.isFinite(explicitExpiresAt)) return new Date(explicitExpiresAt).toISOString();
  const checkedAt = parsePublishAuthTimestamp(cache?.checkedAt || "");
  if (!Number.isFinite(checkedAt)) return "";
  return new Date(checkedAt + publishAuthCacheTtlMs).toISOString();
}

```

The fallback supports older cache files. That lets updated builders read previous authorization records instead of forcing a fresh GitHub login just because the cache schema gained a field.

How `publishAuthCachelsFresh` works inside this feature

`publishAuthCachelsFresh` decides whether cached authorization can be trusted for the requested access. It compares remote, branch, repository, scope, and expiration time.

This is the implementation at the point where the feature needs it.

```

function publishAuthCacheIsFresh(cache, access) {
  if (!cache || !access) return false;
  if (cache.remote !== access.remote || cache.branch !== access.branch || cache.repository !== access.repository)
    return false;
  if (cache.scope !== publishAuthCacheScope()) return false;
  const expiresAt = Date.parse(publishAuthCacheExpiresAt(cache));
  if (!Number.isFinite(expiresAt)) return false;
  return Date.now() < expiresAt;
}

```

Every part of that comparison matters. A cache for one repository should not publish another. A cache from one branch should not silently apply to a different branch. A cache from another installation should not grant this one trust.

How `assertPublishAccess` works inside this feature

assertPublishAccess is the gatekeeper for live website changes. It first proves the portfolio workspace is a Git repository, then reads origin and current branch, then checks compatibility unless the caller intentionally disables that requirement.

This is the implementation at the point where the feature needs it.

```
async function assertPublishAccess(options = {})
```

After building the access shape, it tries the authorization cache. A fresh matching cache returns immediately with authorizationCached true. That is the daily-login behavior the user wanted: reopening the app should not force GitHub authorization again when the target is unchanged.

If the cache is missing or expired, the function synchronizes the local branch from remote, ensures there is a HEAD commit, performs the dry-run write check, writes a new auth cache, and returns a detailed access object. The function both protects Apply to site and supplies the UI with exactly what was verified.

Load from target as a controlled import

syncFromPublishTarget is the opposite direction from Apply to site. It uses an authenticated target as a source of truth and pulls compatible site files into the local builder. That is how a fresh install can recover the latest website state after the owner authenticates.

The function needs the same seriousness as publishing because importing the wrong repository could overwrite local builder state. It checks access, synchronizes the branch, verifies compatible files, copies allowed assets, and reports what happened.

How workspaceHasCompatibleSiteFiles works inside this feature

workspaceHasCompatibleSiteFiles checks whether a target folder looks like an OMB-compatible static portfolio. It expects core website files such as index.html, styles.css, and script.js.

This is the implementation at the point where the feature needs it.

```
async function workspaceHasCompatibleSiteFiles(baseRoot = portfolioRoot) {
  for (const fileName of ["index.html", "styles.css", "script.js"]) {
    try {
      await readFile(path.join(baseRoot, fileName));
    } catch {
      return false;
    }
  }
  return true;
}
```

The function prevents Apply to site from treating an arbitrary repository as a valid website target. Load from target can be more flexible during authentication, but publishing should require compatible files.

How syncFromPublishTarget works inside this feature

syncFromPublishTarget is Load from target. It first calls assertPublishAccess with compatibility relaxed, because a newly selected repository may need to be inspected before the local builder knows whether it has the expected files.

This is the implementation at the point where the feature needs it.

```
async function syncFromPublishTarget() {
  const publishAccess = await assertPublishAccess({ requireCompatible: false });
  const branch = publishAccess.branch || "main";
  const remote = (await runPublishGit(["remote", "get-url", "origin"]).stdout.trim());
  const importRoot = await mkdtemp(path.join(os.tmpdir(), "omb-publish-target-"));
```

```

const cloneRoot = path.join(importRoot, "target");
const importPaths = [
  "projects.json",
  "assets",
  "docs",
  "backgrounds",
  "CNAME",
  ".nojekyll",
  "robots.txt",
  "sitemap.xml"
];

try {
  await runGit(["clone", "--depth", "1", "--branch", branch, remote, cloneRoot], {
    cwd: importRoot,
    timeout: 3 * 60 * 1000
  });

  const availablePaths = [];
  for (const importPath of importPaths) {
    try {
      await fsAccess(path.join(cloneRoot, importPath));
      availablePaths.push(importPath);
    } catch {
      // Optional path is absent in the selected site repository.
    }
  }

  if (!availablePaths.includes("projects.json")) {
    throw publishAccessError(
      "The selected repository does not contain a projects.json portfolio catalog.",
      "The target repository must contain a compatible OMB Portfolio Builder catalog before it can be imported.",
      publishAccess
    );
  }

  const backupRoot = resolveInsideRoot(
    ".omb-backups",
    `load-target-${new Date().toISOString().replace(/[:.]/g, "-")}`
  );
  await mkdir(backupRoot, { recursive: true });

  for (const importPath of availablePaths) {
    const sourcePath = path.join(cloneRoot, importPath);
    const targetPath = resolveInsideRoot(importPath);
    try {
      await fsAccess(targetPath);
      await cp(targetPath, path.join(backupRoot, importPath), { recursive: true, force: true });
      await rm(targetPath, { recursive: true, force: true });
    } catch {
      // Nothing local to back up for this path.
    }
    await cp(sourcePath, targetPath, { recursive: true, force: true });
  }

  const importedCatalog = await readFile(path.join(cloneRoot, "projects.json"), "utf8");
  await writeFile(draftPath, importedCatalog, "utf8");
  const portfolioSvnc = await svncPortfolioPublishFiles({ removeMissing: false });

  return {
    ...publishAccess,
    imported: availablePaths,
    portfolioSvnc,
    backup: path.relative(root, backupRoot),
    draftUpdated: true
  };
} finally {
  await rm(importRoot, { recursive: true, force: true });
}
}

```

The function clones the selected branch into a temporary folder, checks for importable paths, requires projects.json, backs up existing local files, copies target files into the builder workspace, writes the imported catalog into the local draft, and synchronizes publish files.

The temporary clone is removed in a finally block. That cleanup matters because target imports can include assets and documents, and failed imports should not litter the temp directory with old repository copies.

How `syncPortfolioPublishFiles` works inside this feature

`syncPortfolioPublishFiles` copies public website files from the builder root into the separated portfolio workspace when those roots are different.

This is the implementation at the point where the feature needs it.

```
async function syncPortfolioPublishFiles(options = {})
```

It only copies paths from `publishPaths`. That allowlist is the central protection against publishing builder internals, local drafts, compile caches, credentials, or generated documentation that should not belong to the public site.

The `removeMissing` option controls whether missing source files should delete the corresponding target. Apply to site uses removal so the live website reflects deleted assets. Load from target can avoid aggressive deletion during import.

How `stageablePublishPaths` works inside this feature

`stageablePublishPaths` filters the `publishPaths` allowlist down to paths that actually matter for this publish run.

This is the implementation at the point where the feature needs it.

```
asvnc function stageablePublishPaths(pathsToCheck) {
  const stageable = [];
  for (const relativePath of pathsToCheck) {
    if (await publishPathIsStageable(relativePath)) stageable.push(relativePath);
  }
  return stageable;
}
```

The function keeps git add focused. It avoids failing on missing optional paths while still allowing deletions of tracked files to be staged.

How `publishSiteChanges` works inside this feature

`publishSiteChanges` turns a saved catalog and public asset set into a Git push. It obtains or receives verified publish access, synchronizes the branch from remote, copies publishable files, bumps asset versions, filters stageable paths, stages them, commits only when there are changes, and pushes.

This is the implementation at the point where the feature needs it.

```
asvnc function publishSiteChanges(publishAccess = null) {
  const access = publishAccess || await assertPublishAccess();
  const branch = access.branch || (await runPublishGit(["branch", "--show-current"])).stdout.trim();
  const remoteSvnc = await svncSynchronizePublishBranchFromRemote(branch, access);
  const portfolioSvnc = await svncPortfolioPublishFiles({ removeMissing: true });
  await bumpPublishedAssetVersions(portfolioRoot);

  const stageablePaths = await stageablePublishPaths(publishPaths);
  if (!stageablePaths.length) {
    throw new Error("No compatible site files were available to publish.");
  }

  await runPublishGit(["add", "-A", "--", ...stageablePaths]);
  const status = await runPublishGit(["status", "--porcelain", "--", ...stageablePaths]);
  const hasChanges = status.stdout.trim().length > 0;

  let commit = null;
  if (hasChanges) {
```

```

const message = `Update portfolio site ${new Date().toISOString().slice(0, 10)}:`;
commit = await runPublishGit(["commit", "-m", message]);
}

const pushArgs = branch ? ["push", "origin", branch] : ["push"];
const push = await runPublishGit(pushArgs);

return {
  ...access,
  portfolioWorkspace: portfolioRoot,
  remoteSvnc,
  portfolioSvnc,
  branch: branch || "current branch",
  committed: hasChanges,
  commitOutput: commit?.stdout || commit?.stderr || "",
  pushed: true,
  pushOutput: push.stdout || push.stderr || ""
};
}

```

The function is intentionally conservative. It refuses to publish when no compatible files are stageable. It avoids empty commits. It returns commit and push output so the success dialog can show real Git evidence.

The ordering solves the earlier non-fast-forward problem: synchronize first, copy local public files second, stage and commit third, push last. That mirrors the safe human workflow for updating a GitHub Pages repository.

Chapter 7: Portfolio AI And Public Source Context

The AI assistant as a routed backend feature

The Ask My Portfolio panel should answer greetings, general engineering questions, portfolio-specific questions, and public source-code questions differently. That judgment starts in the frontend but is completed by the backend because the backend can enrich context and call model providers safely.

`portfolioAiInstructions` gives the model its behavior contract. It tells the model not to invent projects, not to expose hidden reasoning, not to append random links, and to answer general questions directly before dragging in project context.

The backend owns private keys and local model calls. A browser should not hold an OpenAI key, and a static portfolio page cannot directly supervise local Ollama.

How `portfolioAiInstructions` shapes this workflow

`portfolioAiInstructions` is not ordinary explanatory text. In an AI-backed feature, prompt instructions are application behavior. They define how the assistant should distinguish greetings, general questions, project-specific questions, source-code requests, uncertainty, and portfolio links.

The backend keeps this instruction block because the browser should not own private model behavior or API-key routes. The public page asks a question; the backend assembles the trusted instruction and context.

The code below is the actual shared value used by this feature group.

```

const portfolioAiInstructions = [
  "You are the AI assistant for the portfolio website described by the supplied portfolio context.",
  "Behave like a careful senior electrical and computer engineering mentor who can also navigate the saved portfolio.",
  "Answer the visitor's question first in a concise ChatGPT-like format. then add portfolio links or context only when they help.",
]

```

```

"Use the supplied question intent to decide whether this is a general engineering question or a
portfolio-specific question.".
"For general conversation intent, respond naturally and briefly. Use the recent conversation instead of a fixed
template. If the visitor says hi, a good answer is a short greeting such as: Hi, what can I do for you?".
"If the visitor asks 'what is my name?' or 'who am I?' and they have not identified themselves, do not pretend
to know the visitor. Say you are an AI agent for this portfolio and identify the portfolio owner only if the
supplied profile context includes a name.".
"For general knowledge intent, answer the question directly using broad general knowledge. Do not force
portfolio context unless the visitor asks to connect the answer to the portfolio owner's work.".
"For general engineering intent, begin with the general electronics or engineering explanation. Do not lead with
project context unless the visitor asks to connect it.".
"For portfolio specific intent, answer from the portfolio context first, then explain related engineering
concepts only when useful.".
"Use recent conversation history for follow-up questions, pronouns, comparisons, and corrections.".
"Use the supplied portfolio context as the trusted source for the portfolio owner's projects, links, files,
resume, and contact information.".
"Use sourceExcerpts when present as higher-detail evidence from unloaded text files, extracted resume text,
same-site files, GitHub pages, or other safe public sources.".
"Use knowledgeManifest to understand project files, image evidence, public profiles, resumes, and project areas.
Treat filenames, captions, surrounding text, and descriptions as evidence.".
"Do not claim to visually inspect an image unless actual image analysis is provided. If only image metadata is
supplied, say what the caption/path/context suggests.".
"For GitHub, LinkedIn, resume, or uploaded-file questions, cite what is present in the supplied context or
fetched excerpts and then point to the link when useful. Access public pages and fetched excerpts when the
website allows it.".
"When a visitor asks for source code, use public GitHub source excerpts when provided. Show concise relevant
snippets with file paths, and explain what the code is doing. Do not imply private repository access.".
"If sourceExcerpts include lines labeled Source file, include at least one fenced code block with the file path
immediately before it, unless the visitor explicitly asks for links only.".
"Never invent, infer, or write hypothetical code for the portfolio owner's repositories. Every fenced code block
about repository work must be copied from a fetched Source file excerpt. If no source file excerpt is
available, say that the code was not available in the fetched public sources and give the repository link
instead.".
"For questions about public GitHub repositories, prefer fetched public GitHub source excerpts over portfolio
summaries or project descriptions. You may fetch and display concise code from public GitHub URLs supplied
in the portfolio context or safe source fetches, but never imply access to private repositories.".
"You may answer related electronics, hardware, analog, mixed-signal, digital, embedded, FPGA, ASIC, PCB, and
firmware questions even when the answer is broader than the saved portfolio.".
"Do not invent portfolio projects, credentials, employers, files, or test results that are not in the context.".
"If context is missing, say what is missing and answer generally only for the engineering concept.".
"When useful, state assumptions, define terms, explain signal or data flow, identify tradeoffs, and name what
evidence would prove the claim.".
"Only mention or recommend portfolio links that are directly relevant to the visitor's current question or the
active follow-up context. Do not append random links.".
"Keep the answer recruiter-friendly, specific, and easy to skim. Use short paragraphs and bullets when
helpful.".
"Do not expose chain-of-thought. Give the polished answer only."
].join("\n");

```

How handlePortfolioAi works inside this feature

handlePortfolioAi is the AI request coordinator. It reads the visitor's question, clamps it, cleans recent conversation, validates intent, enriches portfolio context, and then chooses the model provider.

This is the implementation at the point where the feature needs it.

```

asvnc function handlePortfolioAi(request, response) {
  const body = await readRequestJson(request);
  const question = clampText(body.question, 1200).trim();
  const conversation = cleanConversationHistory(body.conversation);
  const validIntents = new Set(["general conversation", "general engineering", "general knowledge",
    "portfolio specific"]);
  const intent = validIntents.has(body.intent) ? body.intent : "portfolio specific";
  const allowWebSearch = body.allowWebSearch === true;

  if (!question) {
    sendJson(response, 400, { error: "Question is required." });
    return;
  }

  const context = await enrichPortfolioContext({ ...(body.context || {}), question });

  const apiKey = process.env.OPENAI_API_KEY || "";
  if (!apiKey) {
    const ollama = await callOllamaPortfolioAi({ question, intent, conversation, context, allowWebSearch });
    if (ollama.ok) {

```

```

    sendJson(response, 200, {
      answer: ollama.answer,
      model: ollama.model,
      provider: "ollama",
      usedWebSearch: false
    });
    return;
  }
  sendJson(response, 503, { error: ollama.error || "No local Ollama model or OPENAI API KEY is available for the
    local backend." });
  return;
}

const model = process.env.OPENAI_MODEL || "gpt-5.4";
const fallbackModel = process.env.OPENAI_FALLBACK_MODEL || "gpt-4.1";
const webSearchMode = String(process.env.OPENAI_ENABLE_WEB_SEARCH || "auto").toLowerCase();
const enableWebSearch = webSearchMode === "true" || (webSearchMode !== "false" && allowWebSearch);
const buildPayload = (selectedModel) => {
  model: selectedModel,
  input: [
    {
      role: "developer",
      content: [{ type: "input text", text: portfolioAiInstructions }]
    },
    {
      role: "user",
      content: [{
        type: "input text",
        text: [
          `Visitor question: ${question}`,
          `Question intent: ${intent}`,
          `Web search allowed for this question: ${enableWebSearch ? "yes" : "no"}`,
          "",
          "Recent conversation JSON:",
          clampText(JSON.stringify(conversation, null, 2), 8000),
          "",
          "Portfolio context JSON:",
          clampText(JSON.stringify(context, null, 2), 18000)
        ].join("\n")
      ]
    }
  ]
};

max output tokens: Number(process.env.OPENAI_MAX_OUTPUT_TOKENS || 1100),
reasoning: { effort: process.env.OPENAI_REASONING_EFFORT || "medium" },
text: { verbosity: process.env.OPENAI_VERBOSITY || "medium" }
});

const callModel = async (selectedModel) => {
  const payload = buildPayload(selectedModel);
  if (enableWebSearch) {
    payload.tools = [{ type: "web search", search context size: process.env.OPENAI_WEB_SEARCH_CONTEXT_SIZE ||
      "low" }];
  }

  const openAiResponse = await fetch("https://api.openai.com/v1/responses", {
    method: "POST",
    headers: {
      "Authorization": `Bearer ${apiKey}`,
      "Content-Type": "application/json"
    },
    body: JSON.stringify(payload)
  });

  const data = await openAiResponse.json().catch(() => ({}));
  return { data, openAiResponse, selectedModel };
};

let { data, openAiResponse, selectedModel } = await callModel(model);
if (!openAiResponse.ok && !process.env.OPENAI_MODEL && fallbackModel && fallbackModel !== model && [400,
  404].includes(openAiResponse.status)) {
  ({ data, openAiResponse, selectedModel } = await callModel(fallbackModel));
}
if (!openAiResponse.ok) {
  sendJson(response, openAiResponse.status, {
    error: data?.error?.message || "OpenAI request failed."
  });
  return;
}
}

```

```

sendJson(response, 200, {
  answer: extractOpenAiText(data),
  model: selectedModel,
  usedWebSearch: enableWebSearch
});
}

```

When no OpenAI key is configured, it tries the local Ollama route. That makes the backend useful for local development and machines without cloud credentials. If Ollama also fails, the endpoint returns a clear service-unavailable response instead of pretending to answer.

When OpenAI is available, the function builds a Responses API payload with developer instructions, question intent, web-search permission, conversation JSON, and portfolio context JSON. The model choice and fallback model come from environment variables so deployments can change models without rewriting the code.

The enableWebSearch decision is also explicit. Web search can be forced, disabled, or allowed per question. This lets the assistant answer portfolio-specific questions from local context while still using broader knowledge when a general electronics question needs it.

The response returns answer, model, and whether web search was used. The frontend does not need to know about prompt assembly, provider fallback, Ollama, OpenAI payload shape, or context-fetching internals.

Public GitHub and website context

enrichPortfolioContext expands the question context. It can include portfolio catalog data, uploaded text evidence, resume text, same-site files, public GitHub source excerpts, and safe public links.

fetchGitHubRepositorySource is intentionally public-source oriented. If a visitor asks for code, the assistant should cite fetched public source excerpts rather than hallucinating code that was never retrieved.

This is where the AI feature becomes more than a canned FAQ. It can combine the owner's saved portfolio data with public repository evidence and general engineering knowledge, while still admitting when a file was not available.

How fetchGitHubRepositorySource works inside this feature

fetchGitHubRepositorySource handles repository-level code retrieval. It asks GitHub for repository contents, filters out binary files, scores likely text files, fetches limited excerpts, and packages them as source evidence.

This is the implementation at the point where the feature needs it.

```

async function fetchGitHubRepositorySource(source = {})

```

The function is intentionally public-only. It should never imply private repository access. If a repository or file cannot be fetched publicly, the assistant should report that limitation rather than inventing code.

How enrichPortfolioContext works inside this feature

enrichPortfolioContext prepares the information the AI will see. It starts with the frontend-supplied context, then adds source excerpts when links or file descriptors can be safely fetched.

This is the implementation at the point where the feature needs it.

```
async function enrichPortfolioContext(context = {})
```

The function's job is not to answer the question. Its job is to assemble trustworthy evidence before the model answers. That separation keeps prompt construction different from response generation.

Choosing OpenAI or Ollama

handlePortfolioAi first enriches context, then checks whether an OpenAI API key is configured. If no key exists, it attempts the local Ollama path. If OpenAI is configured, it builds a Responses API payload with developer instructions, conversation history, question intent, web-search preference, and portfolio context.

callOllamaPortfolioAi gives the local backend a fallback model path. This makes the builder useful on machines where a local model is installed and a cloud key is not configured.

The response shape stays simple for the frontend: answer text, model, provider, and whether web search was used.

How callOllamaPortfolioAi works inside this feature

callOllamaPortfolioAi is the local-model path. It builds a prompt from instructions, question, intent, conversation, and portfolio context, then sends it to the local Ollama API.

This is the implementation at the point where the feature needs it.

```
async function callOllamaPortfolioAi({ question, intent, conversation, context, allowWebSearch })
```

The function is important for offline or keyless machines. If OPENAI_API_KEY is not available, the builder can still answer through a local model when Ollama is running.

The returned object includes ok, answer, model, and error fields so handlePortfolioAi can either send the local answer or report that no backend model is available.

How handlePortfolioAi works inside this feature

handlePortfolioAi is the AI request coordinator. It reads the visitor's question, clamps it, cleans recent conversation, validates intent, enriches portfolio context, and then chooses the model provider.

This is the implementation at the point where the feature needs it.

```
asvnc function handlePortfolioAi(request, response) {
  const body = await readRequestJson(request);
  const question = clampText(body.question, 1200).trim();
  const conversation = cleanConversationHistory(body.conversation);
  const validIntents = new Set(["general conversation", "general engineering", "general knowledge",
    "portfolio specific"]);
  const intent = validIntents.has(body.intent) ? body.intent : "portfolio specific";
  const allowWebSearch = body.allowWebSearch === true;

  if (!question) {
    sendJson(response, 400, { error: "Question is required." });
    return;
  }

  const context = await enrichPortfolioContext({ ...(body.context || {}), question });

  const apiKey = process.env.OPENAI_API_KEY || "";
  if (!apiKey) {
    const ollama = await callOllamaPortfolioAi({ question, intent, conversation, context, allowWebSearch });
    if (ollama.ok) {
      sendJson(response, 200, {
        answer: ollama.answer,
        model: ollama.model,
        provider: "ollama",
        usedWebSearch: false
      });
    }
  }
}
```

```

    return:
  }
  sendJson(response, 503, { error: ollama.error || "No local Ollama model or OPENAI API KEY is available for the
    local backend." });
  return:
}

const model = process.env.OPENAI_MODEL || "gpt-5.4";
const fallbackModel = process.env.OPENAI_FALLBACK_MODEL || "gpt-4.1";
const webSearchMode = String(process.env.OPENAI_ENABLE_WEB_SEARCH || "auto").toLowerCase();
const enableWebSearch = webSearchMode === "true" || (webSearchMode !== "false" && allowWebSearch);
const buildPavload = (selectedModel) => ({
  model: selectedModel,
  input: [
    {
      role: "developer",
      content: [{ type: "input text", text: portfolioAiInstructions }]
    },
    {
      role: "user",
      content: [{
        type: "input text",
        text: [
          `Visitor question: ${question}`,
          `Question intent: ${intent}`,
          `Web search allowed for this question: ${enableWebSearch ? "yes" : "no"}`,
          "",
          "Recent conversation JSON:",
          clampText(JSON.stringify(conversation, null, 2), 8000),
          "",
          "Portfolio context JSON:",
          clampText(JSON.stringify(context, null, 2), 18000)
        ].join("\n")
      ]
    }
  ]
},
  max output tokens: Number(process.env.OPENAI_MAX_OUTPUT_TOKENS || 1100),
  reasoning: { effort: process.env.OPENAI_REASONING_EFFORT || "medium" },
  text: { verbosity: process.env.OPENAI_VERBOSITY || "medium" }
});

const callModel = async (selectedModel) => {
  const pavload = buildPavload(selectedModel);
  if (enableWebSearch) {
    pavload.tools = [{ type: "web search", search context size: process.env.OPENAI_WEB_SEARCH_CONTEXT_SIZE ||
      "low" }];
  }

  const openAiResponse = await fetch("https://api.openai.com/v1/responses", {
    method: "POST",
    headers: {
      "Authorization": `Bearer ${apiKey}`,
      "Content-Type": "application/json"
    },
    body: JSON.stringify(pavload)
  });

  const data = await openAiResponse.json().catch(() => ({}));
  return { data, openAiResponse, selectedModel };
};

let { data, openAiResponse, selectedModel } = await callModel(model);
if (!openAiResponse.ok && !process.env.OPENAI_MODEL && fallbackModel && fallbackModel !== model && [400,
  404].includes(openAiResponse.status)) {
  ({ data, openAiResponse, selectedModel } = await callModel(fallbackModel));
}
if (!openAiResponse.ok) {
  sendJson(response, openAiResponse.status, {
    error: data?.error?.message || "OpenAI request failed."
  });
  return:
}

sendJson(response, 200, {
  answer: extractOpenAiText(data),
  model: selectedModel,
  usedWebSearch: enableWebSearch
});

```

```
}
```

When no OpenAI key is configured, it tries the local Ollama route. That makes the backend useful for local development and machines without cloud credentials. If Ollama also fails, the endpoint returns a clear service-unavailable response instead of pretending to answer.

When OpenAI is available, the function builds a Responses API payload with developer instructions, question intent, web-search permission, conversation JSON, and portfolio context JSON. The model choice and fallback model come from environment variables so deployments can change models without rewriting the code.

The enableWebSearch decision is also explicit. Web search can be forced, disabled, or allowed per question. This lets the assistant answer portfolio-specific questions from local context while still using broader knowledge when a general electronics question needs it.

The response returns answer, model, and whether web search was used. The frontend does not need to know about prompt assembly, provider fallback, Ollama, OpenAI payload shape, or context-fetching internals.

Public source URLs before AI context is built

The AI feature is stronger when it can read public evidence, but public-source fetching needs guardrails. `sourceUrlAllowed` rejects URLs that should not be fetched. `parseGitHubSourceUrl` recognizes GitHub profiles, repositories, and raw source paths. `githubQuestionTokens` and `scoring` helpers decide which files are most likely relevant to the visitor's question.

This means the assistant can answer a repository-code question from actual public source excerpts instead of inventing snippets. The code deliberately separates URL parsing, allowed-source checking, repository scoring, file scoring, and text fetching so each step has a narrow responsibility.

How `sourceLooksTextual` works inside this feature

`sourceLooksTextual` is part of the AI source-ingestion boundary. It decides whether a source descriptor appears to represent text that can be fetched or read into an AI prompt.

This is the implementation at the point where the feature needs it.

```
function sourceLooksTextual(source = {})
```

The function keeps binary-heavy evidence such as images, archives, executables, and office files out of text fetch paths. Those files may still be linked or downloaded, but they should not be treated as plain source text.

How `sourceUrlAllowed` works inside this feature

`sourceUrlAllowed` decides whether a URL is safe enough for backend fetching. It allows known public source contexts and rejects URLs that do not match the permitted shape.

This is the implementation at the point where the feature needs it.

```
function sourceUrlAllowed(url) {
  try {
    const parsed = new URL(url);
    const hostName = parsed.hostname.toLowerCase();
    return [
      "localhost",
      "127.0.0.1",
      "github.com",
      "api.github.com",
      "raw.githubusercontent.com",
    ].includes(hostName);
  } catch {
    return false;
  }
}
```

```

    "gist.githubusercontent.com".
    "linkedin.com".
    "www.linkedin.com"
  }.includes(hostName):
} catch {
  return false:
}
}

```

This matters because the AI assistant can fetch public GitHub and same-site evidence. The backend needs a gate so a portfolio visitor cannot turn the AI endpoint into a general private-network fetcher.

How `gitHubHeaders` works inside this feature

`gitHubHeaders` prepares the headers used for GitHub API requests. The accept type can change depending on whether the request wants JSON metadata or raw content, while the user-agent identifies the builder.

This is the implementation at the point where the feature needs it.

```

function gitHubHeaders(accept = "application/vnd.github+json") {
  return {
    "Accept": accept,
    "User-Agent": "OMB-Portfolio-Builder-AI"
  };
}

```

Putting headers in a helper avoids subtle differences between GitHub fetch calls. Consistent headers reduce API surprises and make future token support easier to add in one place.

How `parseGitHubSourceUrl` works inside this feature

`parseGitHubSourceUrl` reads GitHub-style URLs and extracts owner, repository, branch, and file path when possible. It handles repository pages and blob/raw-style source links differently.

This is the implementation at the point where the feature needs it.

```

function parseGitHubSourceUrl(url) {
  try {
    const parsed = new URL(url);
    const host = parsed.hostname.toLowerCase();
    const parts = parsed.pathname.split("/").filter(Boolean);
    if (host === "raw.githubusercontent.com" && parts.length >= 4) {
      return {
        type: "file",
        owner: parts[0],
        repo: parts[1],
        branch: parts[2],
        filePath: parts.slice(3).join("/")
      };
    }
    if (host !== "github.com" || !parts.length) return null;
    if (parts.length === 1) return { type: "profile", owner: parts[0] };
    const base = { owner: parts[0], repo: parts[1] };
    if (parts[2] === "blob" && parts.length >= 5) {
      return { ...base, type: "file", branch: parts[3], filePath: parts.slice(4).join("/") };
    }
    if (parts[2] === "tree" && parts.length >= 4) {
      return { ...base, type: "repo", branch: parts[3] };
    }
    return { ...base, type: "repo" };
  } catch {
    return null;
  }
}

```

The AI code-fetching feature needs this parser because a portfolio may store a normal repository URL, a branch URL, or a direct file URL. The backend turns those shapes into a common object

before scoring or fetching files.

How githubQuestionTokens works inside this feature

githubQuestionTokens reduces the visitor's question to searchable words. It removes punctuation-like noise and keeps tokens that can help match repositories and files.

This is the implementation at the point where the feature needs it.

```
function githubQuestionTokens(question = "") {
  return [...new Set(String(question || "").
    .toLowerCase()
    .replace(/[\^a-z0-9 #+.\s-!+/_g. " "])
    .split(/\s+/)
    .filter((token) => token.length >= 3 && ![ "the", "and", "with", "from", "show", "code", "file",
      "github" ].includes(token)))];
```

This is simple information retrieval. The model should receive source excerpts that are relevant to the question, and token matching is a cheap first pass before any AI reasoning happens.

How scoreGitHubFile works inside this feature

scoreGitHubFile assigns a relevance score to a file path based on the question. Files with matching names, extensions, or likely source-code relevance rise higher.

This is the implementation at the point where the feature needs it.

```
function scoreGitHubFile(filePath = "", question = "") {
  const cleanPath = filePath.toLowerCase();
  const tokens = githubQuestionTokens(question);
  let score = githubTextFilePattern.test(cleanPath) ? 10 : 0;
  if (/readme\.md$/i.test(cleanPath)) score += 55;
  if (/\/\.(c|h|c++|h++|py|is|mis|ts|v|sv|vhd|?|ino|tcl|xdc|sdc)$/i.test(cleanPath)) score += 28;
  if (/\/b(test|tbl|bench|sim|simulation|src|firmware|hardware|rtl|driver|main)\b/i.test(cleanPath)) score += 12;
  tokens.forEach((token) => {
    if (cleanPath.includes(token)) score += 18;
  });
  return score;
}
```

The score does not prove that a file is the answer. It decides what to fetch first so the AI prompt contains useful evidence within a bounded context size.

How wantsGitHubCode works inside this feature

wantsGitHubCode recognizes questions that are asking for source, implementation, repository code, functions, or files. That signal lets the assistant prefer GitHub source excerpts over high-level portfolio summaries.

This is the implementation at the point where the feature needs it.

```
function wantsGitHubCode(question = "") {
  return
    /\b(code|source|snippet|implementation|firmware|driver|module|verilog|vhd|python|javascript|typescript|c\+\+|cpp|c\s
+code|pull|show|display)\b/i.test(question);
}
```

This prevents generic answers when a visitor specifically asks to see code. If public source is available, the backend should fetch it; if it is not, the assistant should say so clearly.

How scoreGitHubRepo works inside this feature

scoreGitHubRepo ranks repositories against the visitor's question. Names, descriptions, languages, and topics can all make one repository more relevant than another.

This is the implementation at the point where the feature needs it.

```
function scoreGitHubRepo(repo = {})
```

A portfolio owner may link several GitHub repositories. Scoring helps the AI avoid dumping random links and instead focus on the repository most related to the current prompt.

Fetching GitHub and local source text

The source-reading pipeline has several paths. GitHub profile fetches discover repositories. Repository fetches collect likely files and source snippets. Same-site or local source fetches can read uploaded text evidence when allowed. `fetchSourceText` becomes the dispatcher that chooses the right path.

Each fetch is size-limited because an AI prompt should not be flooded by an entire repository or huge file. The purpose is to provide enough evidence for a grounded answer, not to mirror every byte of a public project.

How `fetchGitHubJson` works inside this feature

`fetchGitHubJson` is the small GitHub API fetch wrapper. It requests JSON, checks response status, and returns parsed data.

This is the implementation at the point where the feature needs it.

```
async function fetchGitHubJson(url) {
  const response = await fetch(url, { headers: githubHeaders() });
  if (!response.ok) return null;
  return response.json().catch(() => null);
}
```

The helper keeps error behavior consistent across profile, repository, and contents requests. If GitHub rejects a request, callers can handle one predictable failure shape.

How `fetchLimitedText` works inside this feature

`fetchLimitedText` downloads text with a maximum length. It is used when a source file or public page might be larger than the AI prompt should receive.

This is the implementation at the point where the feature needs it.

```
async function fetchLimitedText(url, maxLength = 12000) {
  const response = await fetch(url, {
    headers: githubHeaders("text/plain;text/markdown;text/html,application/json;q=0.8,*/*;q=0.1")
  });
  if (!response.ok) return "";
  const rawText = clampText(await response.text(), maxLength);
  const contentType = response.headers.get("Content-Type") || "";
  return /html/i.test(contentType) ? stripHtmlToText(rawText) : rawText;
}
```

The limit is a product decision. The assistant needs evidence, not an entire repository or huge generated file. Bounded text keeps answers responsive and reduces model context waste.

How `fetchGitHubProfileSource` works inside this feature

`fetchGitHubProfileSource` handles profile-level GitHub links. It can discover repositories from a GitHub user profile and rank them against the question.

This is the implementation at the point where the feature needs it.

```
async function fetchGitHubProfileSource(source = {}) {
```

This allows a broad GitHub profile link in the portfolio to become useful evidence. Instead of only showing the profile URL, the backend can find likely repositories when the visitor asks a code-related

question.

How fetchGitHubRepositorySource works inside this feature

fetchGitHubRepositorySource handles repository-level code retrieval. It asks GitHub for repository contents, filters out binary files, scores likely text files, fetches limited excerpts, and packages them as source evidence.

This is the implementation at the point where the feature needs it.

```
async function fetchGitHubRepositorySource(source = {})
```

The function is intentionally public-only. It should never imply private repository access. If a repository or file cannot be fetched publicly, the assistant should report that limitation rather than inventing code.

How fetchGitHubSourceText works inside this feature

fetchGitHubSourceText is the direct GitHub source path. When a portfolio link already points at a specific file, this helper fetches the relevant text without first ranking a whole repository.

This is the implementation at the point where the feature needs it.

```
async function fetchGitHubSourceText(source = {})
```

That direct path is useful for resumes, examples, HDL files, scripts, or documentation links that the owner intentionally exposed as public evidence.

How readLocalSourceText works inside this feature

readLocalSourceText reads same-site text files from the portfolio workspace when the URL maps to local published content. It keeps local reads inside the approved root and applies text limits.

This is the implementation at the point where the feature needs it.

```
async function readLocalSourceText(url) {
  let parsed:
  try {
    parsed = new URL(url);
  } catch {
    return "";
  }
  const localHosts = new Set(["localhost", "127.0.0.1"]);
  if (!localHosts.has(parsed.hostname.toLowerCase())) return "";
  const pathname = decodeURIComponent(parsed.pathname.replace(/^\/+/, ""));
  if (!pathname || pathname.includes("..")) return "";
  const ext = path.extname(pathname).toLowerCase();
  if (![".txt", ".md", ".json", ".csv", ".log", ".xml", ".is", ".mis", ".css", ".c", ".h", ".cpp", ".hpp", ".py", ".v", ".sv", ".spice", ".cir", ".net", ".asc", ".sch", ".kicad sch", ".kicad pcb"].includes(ext)) return "";
  try {
    return await readFile(resolveInsideRoot(pathname), "utf8");
  } catch {
    return "";
  }
}
```

This lets uploaded text evidence, Markdown files, source files, or simulation logs become AI context without requiring GitHub fetching.

How fetchSourceText works inside this feature

fetchSourceText is the dispatcher for source evidence. It looks at a source descriptor and chooses whether to read local text, fetch a same-site URL, fetch a GitHub file, or expand a GitHub repository.

This is the implementation at the point where the feature needs it.

```
async function fetchSourceText(source = {})
```

That dispatcher is what lets the AI assistant combine uploaded project files, public repository files, resume excerpts, and safe website links through one enrichment path.

How enrichPortfolioContext works inside this feature

enrichPortfolioContext prepares the information the AI will see. It starts with the frontend-supplied context, then adds source excerpts when links or file descriptors can be safely fetched.

This is the implementation at the point where the feature needs it.

```
async function enrichPortfolioContext(context = {})
```

The function's job is not to answer the question. Its job is to assemble trustworthy evidence before the model answers. That separation keeps prompt construction different from response generation.

Conversation cleanup before a model sees it

AI chat is not just the current question. Follow-up questions need recent context, but unlimited history can become noisy, expensive, and privacy-unfriendly. cleanConversationHistory reduces the chat history to a bounded, predictable shape before it is sent to Ollama or OpenAI.

The extraction helpers then turn provider-specific responses into plain assistant text. extractOpenAiText understands the Responses API shape. extractOllamaText understands the local Ollama response shape. The rest of the UI does not need to know which provider produced the answer.

How cleanConversationHistory works inside this feature

cleanConversationHistory keeps recent chat context useful and bounded. It accepts only reasonable message shapes, trims text, and limits how much history travels to the model.

This is the implementation at the point where the feature needs it.

```
function cleanConversationHistory(value) {
  if (!Array.isArray(value)) return [];
  return value
    .slice(-8)
    .map((item) => ({
      role: item?.role === "assistant" ? "assistant" : "user",
      content: clampText(item?.content, 1400).trim()
    }))
    .filter((item) => item.content);
}
```

A conversational assistant needs memory for follow-up questions, but unlimited chat history would make prompts noisy and slower. This helper preserves continuity without letting the conversation become an unbounded payload.

How extractOpenAiText works inside this feature

extractOpenAiText reads the OpenAI Responses API output shape and returns the assistant's text. Provider responses can contain arrays, nested content items, and metadata; the frontend only needs the final answer.

This is the implementation at the point where the feature needs it.

```
function extractOpenAiText(data) {
  if (typeof data?.output?.text === "string" && data.output?.text?.trim()) return data.output?.text?.trim();
  const output = Array.isArray(data?.output) ? data.output : [];
  const chunks = [];

  output.forEach((item) => {
    (item?.content || []).forEach((content) => {
      if (typeof content?.text === "string" && content?.text?.trim()) chunks.push(content?.text?.trim());
    });
  });
}
```

```

    }):
  }):

  return chunks.join("\n\n").trim():
}

```

This helper isolates provider-specific parsing. If OpenAI changes response details later, the rest of `handlePortfolioAi` can remain focused on routing and error handling.

How `extractOllamaText` works inside this feature

`extractOllamaText` performs the same normalization for the local Ollama response. It returns the model's message text while hiding provider-specific response details from the rest of the app.

This is the implementation at the point where the feature needs it.

```
function extractOllamaText(data = {})
```

Keeping OpenAI and Ollama extraction separate lets the assistant support both cloud and local models without forcing the UI to understand either response format.

General conversation fallback before model calls

`ruleBasedConversationAnswer` exists because not every chat needs a large model. A greeting, a simple identity prompt, or an empty conversational opener can be answered safely and quickly. This prevents the assistant from sounding like a canned project search result when the visitor is only starting a conversation.

The function does not replace the AI model. It handles the small social edges that make the chat feel alive before deeper questions are routed to portfolio context and model reasoning.

How `ruleBasedConversationAnswer` works inside this feature

`ruleBasedConversationAnswer` handles tiny conversational cases that should not need a model call. Greetings and simple identity prompts can be answered immediately and naturally.

This is the implementation at the point where the feature needs it.

```
function ruleBasedConversationAnswer(question = "") {
  const clean = String(question || "").toLowerCase():
  if (/\/b(thanks|thank you|appreciate it)\b/.test(clean)) {
    return "You're welcome. I can keep helping with this portfolio. project evidence. resume links. or related electronics and embedded-svstems questions.":
  }

  if (/\/b(who are you|what are you)\b/.test(clean)) {
    return "I am this portfolio's assistant. I can help visitors explore the saved engineering work. explain project details. summarize portfolio evidence. and answer related electronics. embedded. analog. digital. FPGA. ASIC. PCB. and firmware questions.":
  }

  return "Hi. I can help you explore this portfolio. explain projects. summarize project evidence. open relevant sections. or answer related electronics and embedded-svstems questions. You can ask about a specific project. a tool like KiCad or STM32CubeIDE. or a general topic such as embedded svstems. op amps. FPGA design. ASICs. or PCB testing.":
}

```

This helper improves the feel of the chatbot. Without it, a visitor saying hi could receive an awkward portfolio-search answer or an unnecessary model failure when the backend provider is unavailable.

How `callOllamaPortfolioAi` works inside this feature

`callOllamaPortfolioAi` is the local-model path. It builds a prompt from instructions, question, intent, conversation, and portfolio context, then sends it to the local Ollama API.

This is the implementation at the point where the feature needs it.

```
async function callOllamaPortfolioAi({ question, intent, conversation, context, allowWebSearch })
```

The function is important for offline or keyless machines. If OPENAI_API_KEY is not available, the builder can still answer through a local model when Ollama is running.

The returned object includes ok, answer, model, and error fields so handlePortfolioAi can either send the local answer or report that no backend model is available.

How handlePortfolioAi works inside this feature

handlePortfolioAi is the AI request coordinator. It reads the visitor's question, clamps it, cleans recent conversation, validates intent, enriches portfolio context, and then chooses the model provider.

This is the implementation at the point where the feature needs it.

```
async function handlePortfolioAi(request, response) {
  const body = await readRequestJson(request);
  const question = clampText(body.question, 1200).trim();
  const conversation = cleanConversationHistory(body.conversation);
  const validIntents = new Set(["general conversation", "general engineering", "general knowledge",
    "portfolio specific"]);
  const intent = validIntents.has(body.intent) ? body.intent : "portfolio specific";
  const allowWebSearch = body.allowWebSearch === true;

  if (!question) {
    sendJson(response, 400, { error: "Question is required." });
    return;
  }

  const context = await enrichPortfolioContext({ ...(body.context || {}), question });

  const apiKey = process.env.OPENAI_API_KEY || "";
  if (!apiKey) {
    const ollama = await callOllamaPortfolioAi({ question, intent, conversation, context, allowWebSearch });
    if (ollama.ok) {
      sendJson(response, 200, {
        answer: ollama.answer,
        model: ollama.model,
        provider: "ollama",
        usedWebSearch: false
      });
      return;
    }
    sendJson(response, 503, { error: ollama.error || "No local Ollama model or OPENAI API KEY is available for the local backend." });
    return;
  }

  const model = process.env.OPENAI_MODEL || "gpt-5.4";
  const fallbackModel = process.env.OPENAI_FALLBACK_MODEL || "gpt-4.1";
  const webSearchMode = String(process.env.OPENAI_ENABLE_WEB_SEARCH || "auto").toLowerCase();
  const enableWebSearch = webSearchMode === "true" || (webSearchMode !== "false" && allowWebSearch);
  const buildPayload = (selectedModel) => ({
    model: selectedModel,
    input: [
      {
        role: "developer",
        content: [{ type: "input text", text: portfolioAiInstructions }]
      },
      {
        role: "user",
        content: [{
          type: "input text",
          text: [
            `Visitor question: ${question}`,
            `Question intent: ${intent}`,
            `Web search allowed for this question: ${enableWebSearch ? "yes" : "no"}`,
            "",
            "Recent conversation JSON:",
            clampText(JSON.stringify(conversation, null, 2), 8000),
            "",
            "Portfolio context JSON:",

```

```

        clampText(JSON.stringify(context, null, 2), 18000)
    }.join("\n")
  }
}
},
max output tokens: Number(process.env.OPENAI_MAX_OUTPUT_TOKENS || 1100),
reasoning: { effort: process.env.OPENAI_REASONING_EFFORT || "medium" },
text: { verbosity: process.env.OPENAI_VERBOSITY || "medium" }
});

const callModel = async (selectedModel) => {
  const payload = buildPayload(selectedModel);
  if (enableWebSearch) {
    payload.tools = [{ type: "web search", search context size: process.env.OPENAI_WEB_SEARCH_CONTEXT_SIZE ||
      "low" }];
  }

  const openAiResponse = await fetch("https://api.openai.com/v1/responses", {
    method: "POST",
    headers: {
      "Authorization": `Bearer ${apiKey}`,
      "Content-Type": "application/json"
    },
    body: JSON.stringify(payload)
  });

  const data = await openAiResponse.json().catch(() => ({}));
  return { data, openAiResponse, selectedModel };
};

let { data, openAiResponse, selectedModel } = await callModel(model);
if (!openAiResponse.ok && !process.env.OPENAI_MODEL && fallbackModel && fallbackModel !== model && [400,
  404].includes(openAiResponse.status)) {
  ({ data, openAiResponse, selectedModel } = await callModel(fallbackModel));
}
if (!openAiResponse.ok) {
  sendJson(response, openAiResponse.status, {
    error: data?.error?.message || "OpenAI request failed."
  });
  return;
}

sendJson(response, 200, {
  answer: extractOpenAiText(data),
  model: selectedModel,
  usedWebSearch: enableWebSearch
});
}

```

When no OpenAI key is configured, it tries the local Ollama route. That makes the backend useful for local development and machines without cloud credentials. If Ollama also fails, the endpoint returns a clear service-unavailable response instead of pretending to answer.

When OpenAI is available, the function builds a Responses API payload with developer instructions, question intent, web-search permission, conversation JSON, and portfolio context JSON. The model choice and fallback model come from environment variables so deployments can change models without rewriting the code.

The enableWebSearch decision is also explicit. Web search can be forced, disabled, or allowed per question. This lets the assistant answer portfolio-specific questions from local context while still using broader knowledge when a general electronics question needs it.

The response returns answer, model, and whether web search was used. The frontend does not need to know about prompt assembly, provider fallback, Ollama, OpenAI payload shape, or context-fetching internals.

Chapter 8: Updates, Security Reports, And Maintenance

Update checks

getUpdateInfo compares the installed builder version with GitHub release information. The UI can then show whether the app is current or whether an update is available.

The backend blocks known bad versions through blockedAppUpdateVersions. That small Set gives the maintainer a way to skip a release without deleting every artifact.

The user-facing idea is simple: the app should know when a better release exists, but it should not install blindly or downgrade itself.

How packageJson shapes this workflow

packageJson carries the builder's package metadata into runtime code. The updater uses its version field to decide whether the installed app is current or behind a release.

The code below is the actual shared value used by this feature group.

```
const packageJson = JSON.parse(await readFile(path.join(root, "package.json"), "utf8").catch(() => "{}"));
```

How blockedAppUpdateVersions shapes this workflow

blockedAppUpdateVersions is a release safety switch. If a version is known to be broken, the updater can skip it without deleting GitHub release files.

The code below is the actual shared value used by this feature group.

```
const blockedAppUpdateVersions = new Set(["0.2.16"]);
```

How getUpdateInfo works inside this feature

getUpdateInfo asks GitHub Releases for the latest builder release, compares it with the installed version, finds installer and portable assets, and reports whether an update is available.

This is the implementation at the point where the feature needs it.

```
asvnc function getUpdateInfo() {
  const currentVersion = process.env.OMB_APP_VERSION || packageJson.version || "0.0.0";
  const owner = "otienomaurice";
  const repo = "omb-portfolio-builder";
  try {
    const response = await fetch(`https://api.github.com/repos/${owner}/${repo}/releases/latest`, {
      headers: { "Accept": "application/vnd.github+json", "User-Agent": "OMB-Portfolio-Builder" }
    });
    if (!response.ok) throw new Error(`GitHub returned ${response.status}.`);
    const release = await response.json();
    const latestVersion = String(release.tag.name || "").replace(/^builder-v/i, "");
    const installer = (release.assets || []).find((asset) => /Setup-.*\.exe$/i.test(asset.name));
    const nortable = (release.assets || []).find((asset) => /Portable-.*\.exe$/i.test(asset.name));
    const updateBlocked = blockedAppUpdateVersions.has(latestVersion);
    return {
      ok: true,
      currentVersion,
      latestVersion,
      updateAvailable: !updateBlocked && compareVersions(latestVersion, currentVersion) > 0,
      updateBlocked,
      blockedReason: updateBlocked
        ? `Builder ${latestVersion} is skipped because its installer can hang while running the old uninstaller
          during in-app updates. Wait for builder 0.2.17 or newer, then run Update again.`
        : "",
      releaseUrl: release.html_url || "",
      installerUrl: installer?.browser_download_url || "",
      installerName: installer?.name || "",
      nortableUrl: nortable?.browser_download_url || ""
    };
  } catch (error) {
    return {
      ok: false,
      currentVersion,
      latestVersion: currentVersion,
    };
  }
}
```

```

    updateAvailable: false,
    error: error.message || "Could not check for updates."
  };
}
}

```

The blocked version check is a safety escape hatch. If a release is known to have installer problems, the backend can refuse to offer it for automatic update while still reporting why it was skipped.

Launching an update

`downloadAndLaunchAppUpdate` is more than a download helper. It downloads the installer, writes a detached launcher script, closes the running builder, starts the installer, waits for completion, finds the installed executable, and relaunches the app.

The feature is complicated because a running app cannot safely replace all of its own files. The handoff script survives the app shutdown and performs the update from outside the process being replaced.

This is why update behavior belongs in the backend rather than the browser view. It touches the filesystem, process table, installer, and Windows application paths.

How `downloadAndLaunchAppUpdate` works inside this feature

`downloadAndLaunchAppUpdate` performs the automatic update handoff. It checks update availability, rejects blocked or missing installer releases, downloads the installer to a temp folder, validates that the file is large enough to be plausible, and writes it to disk.

This is the implementation at the point where the feature needs it.

```

asvnc function downloadAndLaunchAppUpdate() {
  const update = await getUpdateInfo();
  if (!update.ok) throw new Error(update.error || "Could not check for updates.");
  if (update.updateBlocked) {
    throw new Error(update.blockedReason || `Builder ${update.latestVersion} is temporarily blocked from automatic updates.`);
  }
  if (!update.updateAvailable) {
    throw new Error(`OMB Portfolio Builder ${update.currentVersion || ""} is already up to date.`);
  }
  if (!update.installerUrl) {
    throw new Error("The latest GitHub Release does not include a Windows setup installer.");
  }
  if (process.platform !== "win32") {
    throw new Error("Automatic installer updates are currently available only on Windows.");
  }

  const updateRoot = path.join(os.tmpdir(), "omb-portfolio-builder-updates");
  await mkdir(updateRoot, { recursive: true });
  const installerName = update.installerName && /\.exe$/i.test(update.installerName)
    ? update.installerName
    : `OMB-Portfolio-Builder-Setup-${safeUpdateFileSegment(update.latestVersion)}-x64.exe`;
  const installerPath = path.join(updateRoot, installerName);

  const response = await fetch(update.installerUrl, {
    headers: { "Accept": "application/octet-stream", "User-Agent": "OMB-Portfolio-Builder" }
  });
  if (!response.ok) throw new Error(`GitHub returned ${response.status} while downloading the installer.`);
  const installerBuffer = Buffer.from(await response.arrayBuffer());
  if (installerBuffer.length < 1024 * 1024) {
    throw new Error("The downloaded installer was unexpectedly small. so the update was stopped.");
  }
  await writeFile(installerPath, installerBuffer);

  const updateLogPath = path.join(updateRoot, `update-${safeUpdateFileSegment(update.latestVersion)}.log`);
  const currentExecutable = /OMB Portfolio Builder\.exe$/i.test(process.execPath || "") ? process.execPath : "";
  const currentInstallDirectory = currentExecutable ? path.dirname(currentExecutable) : "";

```

```

const localAppDataExecutable = process.env.LOCALAPPDATA
  ? path.join(process.env.LOCALAPPDATA, "Programs", "OMB Portfolio Builder", "OMB Portfolio Builder.exe")
  : "";
const legacyManagedApplicationExecutable = path.join(os.homedir(), "OMB", "application", "OMB Portfolio
  Builder", "OMB Portfolio Builder.exe");
const legacyFlatApplicationExecutable = path.join(os.homedir(), "OMB", "application", "OMB Portfolio
  Builder.exe");
const relaunchCandidates = Array.from(new Set([
  localAppDataExecutable,
  currentExecutable,
  currentInstallDirectory ? path.join(currentInstallDirectory, "OMB Portfolio Builder.exe") : "",
  legacyFlatApplicationExecutable,
  legacyManagedApplicationExecutable,
  process.env.ProgramFiles ? path.join(process.env.ProgramFiles, "OMB Portfolio Builder", "OMB Portfolio
    Builder.exe") : "",
  process.env["ProgramFiles(x86)"] ? path.join(process.env["ProgramFiles(x86)"], "OMB Portfolio Builder", "OMB
    Portfolio Builder.exe") : ""
].filter(Boolean)));
const relaunchCandidatesPs = `@($relaunchCandidates.map(powershellSingleQuoted).join(", "));`;
const launcherPath = path.join(updateRoot, `run-update-${safeUpdateFileSegment(update.latestVersion)}.ps1`);
const launcherCommandPath = path.join(updateRoot,
  `run-update-${safeUpdateFileSegment(update.latestVersion)}.cmd`);
const launcherScript = [
  "$ErrorActionPreference = 'Continue'",
  "$parentPid = ${process.pid}",
  "$installer = ${powershellSingleQuoted(installerPath)}",
  "$log = ${powershellSingleQuoted(updateLogPath)}",
  "$candidates = $relaunchCandidatesPs",
  "function Write-UpdateLog([string]$Message) {",
  "  trv { Add-Content -LiteralPath $log -Value ((Get-Date).ToString('s') + ' ' + $Message) } catch {}",
  "}",
  "function Wait-ForExecutable([string[]]$Paths, [int]$Seconds) {",
  "  $deadline = (Get-Date).AddSeconds($Seconds)",
  "  do {",
  "    foreach ($candidate in $Paths) {",
  "      if ($candidate -and (Test-Path -LiteralPath $candidate)) { return $candidate }",
  "    }",
  "    Start-Sleep -Seconds 1",
  "  } while ((Get-Date) -lt $deadline)",
  "  return $null",
  "}",
  "function Stop-BuilderProcesses([int]$ProcessId, [string[]]$Paths) {",
  "  trv {",
  "    $normalizedPaths = @($Paths | Where-Object { $_ } | ForEach-Object {",
  "      trv { [System.IO.Path]::GetFullPath($_).ToLowerInvariant() } catch { $_.ToLowerInvariant() }",
  "    })",
  "    $processes = Get-CimInstance Win32_Process | Where-Object {",
  "      $_.ProcessId -eq $ProcessId -or",
  "      $_.Name -eq 'OMB Portfolio Builder.exe' -or",
  "      ($_.ExecutablePath -and ($normalizedPaths -contains",
  "        ([System.IO.Path]::GetFullPath($_.ExecutablePath).ToLowerInvariant()))",
  "      } | Sort-Object ProcessId -Unique",
  "    }",
  "    foreach ($process in $processes) {",
  "      trv {",
  "        Write-UpdateLog ('Stopping previous builder process PID ' + $process.ProcessId + ' at ' +",
  "          $process.ExecutablePath)",
  "        Stop-Process -Id $process.ProcessId -Force -ErrorAction SilentlyContinue",
  "      } catch { Write-UpdateLog ('Could not stop PID ' + $process.ProcessId + ': ' + $_.Exception.Message)",
  "        }",
  "      Start-Sleep -Seconds 2",
  "    } catch { Write-UpdateLog ('Could not stop previous builder processes: ' + $_.Exception.Message) }",
  "  }",
  "function Run-Installer([switch]$Elevated) {",
  "  $process = $null",
  "  if ($Elevated) {",
  "    Write-UpdateLog 'Starting installer with Windows elevation prompt.'",
  "    $process = Start-Process -FilePath $installer -ArgumentList @('/S') -Verb RunAs -PassThru",
  "  } else {",
  "    Write-UpdateLog 'Starting installer silently.'",
  "    $process = Start-Process -FilePath $installer -ArgumentList @('/S') -WindowStyle Hidden -PassThru",
  "  }",
  "  if (-not $process) { return 0 }",
  "  if (-not $process.WaitForExit(900000)) {",
  "    Write-UpdateLog ('Installer timed out after 15 minutes. PID: ' + $process.Id)",
  "    trv { Stop-Process -Id $process.Id -Force -ErrorAction SilentlyContinue } catch { Write-UpdateLog ('Could",
  "      not stop timed-out installer: ' + $_.Exception.Message) }",
  "    return 124",

```

```

    " }".
    " return $process.ExitCode".
    "}".
    "Write-UpdateLog 'Update launcher started.'".
    "Start-Sleep -Seconds 3".
    "try { Wait-Process -Id $parentPid -Timeout 8; Write-UpdateLog 'Previous app process exited.' } catch {
        Write-UpdateLog ('Previous app wait finished: ' + $ .Exception.Message) }".
    "Stop-BuilderProcesses $parentPid $candidates".
    "Start-Sleep -Seconds 1".
    "$installerProcess = $null".
    "$installerExit = 1".
    "try {".
    "   Write-UpdateLog ('Starting installer: ' + $installer)".
    "   $installerExit = Run-Installer".
    "   Write-UpdateLog ('Installer exit code: ' + $installerExit)".
    "} catch {".
    "   Write-UpdateLog ('Installer launch failed: ' + $ .Exception.Message)".
    "   exit 1".
    "}".
    "if ($installerExit -ne 0) {".
    "   try {".
    "       Write-UpdateLog 'Installer did not finish cleanly: retrieving with Windows elevation prompt.'".
    "       $installerExit = Run-Installer -Elevated".
    "       Write-UpdateLog ('Elevated installer exit code: ' + $installerExit)".
    "       } catch { Write-UpdateLog ('Elevated installer launch failed: ' + $ .Exception.Message) }".
    "   }".
    "if ($installerExit -ne 0) { exit $installerExit }".
    "$target = Wait-ForExecutable $candidates 90".
    "if ($target) {".
    "   try {".
    "       Start-Sleep -Seconds 2".
    "       Start-Process -FilePath $target -WorkingDirectory ($Split-Path -Parent $target)".
    "       Write-UpdateLog ('Relaunched app: ' + $target)".
    "       exit 0".
    "   } catch {".
    "       Write-UpdateLog ('Relaunch failed: ' + $ .Exception.Message)".
    "       exit 3".
    "   }".
    "   }".
    "Write-UpdateLog 'No installed app executable was found after update.'".
    "exit 2"
1. ioin("\r\n"):
await writeFile(launcherPath, launcherScript, "utf8"):
const launcherCommandScript = [
    "@echo off".
    `start "" /min powershell.exe -NoProfile -ExecutionPolicy Bypass -WindowStyle Hidden -File "${launcherPath}"`
1. ioin("\r\n"):
await writeFile(launcherCommandPath, launcherCommandScript, "utf8"):

const child = spawn("cmd.exe", ["/d", "/", "/c", launcherCommandPath], {
    detached: true,
    stdio: "ignore",
    windowsHide: true
}):
child.unref():

setTimeout(() => {
    process.emit("omb-builder-update-started", { installerPath, launcherPath, updateLogPath });
}, 600):
setTimeout(() => {
    process.exit(0);
}, 3000):

return {
    ...update,
    installerPath,
    launcherPath,
    updateLogPath,
    launcherCommandPath,
    started: true
};
}

```

The function then calculates relaunch candidates. It includes the per-user AppData install path, the current executable, legacy OMB application paths, and Program Files paths. That broad list is what

lets the updater find the app again after installation, even as the project migrates away from older folder layouts.

The generated PowerShell script is the real bridge. It waits for the current process, stops any builder processes, runs the installer silently, retries elevated if necessary, waits for an executable to appear, and relaunches it. The Node process cannot safely replace itself, so the detached script continues after Node exits.

The final timers emit an update-started event and exit the backend. The HTTP response returns before the process closes, giving the frontend a chance to tell the user that the update has started.

Security reporting

`getSecurityReport` exposes local visibility into access and download-related information for an authenticated builder. The report is local-only because security information should not be readable from arbitrary network clients.

This feature is the beginning of operational awareness. It does not turn a static site into a full analytics platform by itself, but it gives the builder a place to surface trusted local security and publishing state.

The same local-only principle used for write actions appears here: even read-only security information can be sensitive, so the API route checks the request source before responding.

How `getSecurityReport` works inside this feature

`getSecurityReport` combines release download counts, authorization cache status, publish target information, and documented security controls into one local report.

This is the implementation at the point where the feature needs it.

```
asvnc function getSecurityReport() {
  const [downloadResult, authCacheResult, targetResult] = await Promise.allSettled([
    getBuilderReleaseDownloadReport(),
    readPublishAuthCache(),
    getPublishTargetInfo()
  ]);
  const authCache = authCacheResult.status === "fulfilled" ? authCacheResult.value : null;
  const target = targetResult.status === "fulfilled" ? targetResult.value : null;
  return {
    ok: true,
    generatedAt: new Date().toISOString(),
    appDownloads: downloadResult.status === "fulfilled"
      ? { ok: true, ...downloadResult.value }
      : { ok: false, error: downloadResult.reason?.message || "GitHub release download counts were not available." },
    publishingAuth: {
      targetRepository: target?.repository || "",
      branch: target?.branch || "",
      authenticated: Boolean(authCache && publishAuthCacheIsFresh(authCache, {
        remote: target?.remote || "",
        repository: target?.repository || "",
        branch: target?.branch || ""
      })),
      expiresAt: publishAuthCacheExpiresAt(authCache) || "",
      trustDays: Number(authCache?.trustDays || 0),
      successCountLastWeek: Number(authCache?.successCountLastWeek || 0)
    },
    websiteAccess: {
      available: false,
      summary: "Visitor counts and visitor IP addresses are not available from a static GitHub Pages site or GitHub release download counts.",
      recommendedSetup: [
        "Proxy the custom domain through Cloudflare instead of DNS-only.",
        "Enable Cloudflare Web Analytics for aggregate page-view counts.",
        "Use Cloudflare Logpush, Analytics Engine, or a Worker-backed endpoint if you need IP-level security"
      ]
    }
  };
}
```

```

    logs.".
    "Publish a privacy notice before collecting raw IP addresses or persistent identifiers."
  ]
},
securityControls: [
  "Local builder write APIs are restricted to local requests.",
  "Publishing target details are only returned to the local machine.",
  "GitHub publishing authorization is cached for one day by default, or longer after repeated successful
  authorizations.",
  "The desktop app runs with Electron nodeIntegration disabled, contextIsolation enabled, and sandbox
  enabled.",
  "Denlovable website security headers and security.txt metadata are included in the publishable site files."
]
}:
}

```

The website access section is deliberately honest. A static GitHub Pages site does not provide raw visitor identity or IP logs by itself. The report recommends Cloudflare analytics or Worker-backed logging if the owner wants deeper visibility with a privacy notice.

Version comparison and release selection

The update checker needs to compare semantic-looking version strings, skip known bad releases, and find useful release assets. `compareVersions` gives the updater a deterministic way to decide whether a release is newer. `blockedAppUpdateVersions` gives the maintainer a quick blocklist for releases that should not be offered.

`getUpdateInfo` combines package metadata, GitHub release information, and download asset names into the object the UI can show. A good updater is not just a download button; it tells the user what version they have, what version is available, and whether the app is already current.

How `packageJson` shapes this workflow

`packageJson` carries the builder's package metadata into runtime code. The updater uses its version field to decide whether the installed app is current or behind a release.

The code below is the actual shared value used by this feature group.

```
const packageJson = JSON.parse(await readFile(path.join(root, "package.json"), "utf8").catch(() => "{}"));
```

How `blockedAppUpdateVersions` shapes this workflow

`blockedAppUpdateVersions` is a release safety switch. If a version is known to be broken, the updater can skip it without deleting GitHub release files.

The code below is the actual shared value used by this feature group.

```
const blockedAppUpdateVersions = new Set(["0.2.16"]);
```

How `compareVersions` works inside this feature

`compareVersions` compares dotted version strings part by part. The update checker uses it to decide whether the GitHub release is newer than the installed app.

This is the implementation at the point where the feature needs it.

```
function compareVersions(left = "", right = "") {
  const leftParts = String(left || "0").split(/[.-]/).map((part) => Number.parseInt(part, 10) || 0);
  const rightParts = String(right || "0").split(/[.-]/).map((part) => Number.parseInt(part, 10) || 0);
  const length = Math.max(leftParts.length, rightParts.length);
  for (let index = 0; index < length; index += 1) {
    const delta = (leftParts[index] || 0) - (rightParts[index] || 0);
    if (delta !== 0) return delta;
  }
  return 0;
}
```

This is deliberately simpler than a full semantic-version library because the builder release tags follow a predictable builder-vX.Y.Z pattern. The helper keeps updater logic dependency-light.

How getUpdateInfo works inside this feature

getUpdateInfo asks GitHub Releases for the latest builder release, compares it with the installed version, finds installer and portable assets, and reports whether an update is available.

This is the implementation at the point where the feature needs it.

```
asvnc function getUpdateInfo() {
  const currentVersion = process.env.OMB_APP_VERSION || packageJson.version || "0.0.0";
  const owner = "otienomaurice";
  const repo = "omb-portfolio-builder";
  try {
    const response = await fetch(`https://api.github.com/repos/${owner}/${repo}/releases/latest`, {
      headers: { "Accept": "application/vnd.github+json", "User-Agent": "OMB-Portfolio-Builder" }
    });
    if (!response.ok) throw new Error(`GitHub returned ${response.status}.`);
    const release = await response.json();
    const latestVersion = String(release.tag_name || "").replace(/^builder-v/i, "");
    const installer = (release.assets || []).find((asset) => /Setup-.*\.exe$/i.test(asset.name));
    const portable = (release.assets || []).find((asset) => /Portable-.*\.exe$/i.test(asset.name));
    const updateBlocked = blockedAppUpdateVersions.has(latestVersion);
    return {
      ok: true,
      currentVersion,
      latestVersion,
      updateAvailable: !updateBlocked && compareVersions(latestVersion, currentVersion) > 0,
      updateBlocked,
      blockedReason: updateBlocked
        ? `Builder ${latestVersion} is skipped because its installer can hang while running the old uninstaller during in-app updates. Wait for builder 0.2.17 or newer, then run Update again.`
        : "",
      releaseUrl: release.html_url || "",
      installerUrl: installer?.browser_download_url || "",
      installerName: installer?.name || "",
      portableUrl: portable?.browser_download_url || ""
    };
  } catch (error) {
    return {
      ok: false,
      currentVersion,
      latestVersion: currentVersion,
      updateAvailable: false,
      error: error.message || "Could not check for updates."
    };
  }
}
```

The blocked version check is a safety escape hatch. If a release is known to have installer problems, the backend can refuse to offer it for automatic update while still reporting why it was skipped.

How getBuilderReleaseDownloadReport works inside this feature

getBuilderReleaseDownloadReport reads GitHub release asset download counts. It returns asset names, download counts, sizes, update times, URLs, and a total download count.

This is the implementation at the point where the feature needs it.

```
asvnc function getBuilderReleaseDownloadReport() {
  const owner = "otienomaurice";
  const repo = "omb-portfolio-builder";
  const response = await fetch(`https://api.github.com/repos/${owner}/${repo}/releases/latest`, {
    headers: { "Accept": "application/vnd.github+json", "User-Agent": "OMB-Portfolio-Builder" }
  });
  if (!response.ok) throw new Error(`GitHub returned ${response.status}.`);
  const release = await response.json();
  const assets = (release.assets || []).map((asset) => ({
    name: asset.name || "",
    downloadCount: Number(asset.download_count || 0),
  }));
}
```

```

    sizeBytes: Number(asset.size || 0),
    undatedAt: asset.undated at || "",
    url: asset.browser download url || ""
  }));
  return {
    releaseName: release.name || release.tag name || "latest",
    releaseTag: release.tag name || "",
    releaseUrl: release.html url || "",
    totalDownloads: assets.reduce((total, asset) => total + asset.downloadCount, 0),
    assets
  };
}

```

This is the only download visibility GitHub Releases provides directly. It can tell how many downloads occurred per asset, but not who downloaded them or their IP addresses.

PowerShell quoting and detached update handoff

downloadAndLaunchAppUpdate writes a PowerShell handoff script. That script has to survive after the app exits, wait for the old process to close, run the installer, find the installed executable, and relaunch it. powershellSingleQuoted and safeUpdateFileSegment are small helpers that keep generated script text and filenames predictable.

The update path is intentionally not a normal in-process function call. The running app cannot replace itself cleanly while its files are in use. The detached script is the bridge between the old process and the new installed version.

How safeUpdateFileSegment works inside this feature

safeUpdateFileSegment cleans version names and other update fragments before they become temp filenames. It keeps letters, numbers, dots, underscores, and hyphens, then trims the result.

This is the implementation at the point where the feature needs it.

```

function safeUpdateFileSegment(value = "") {
  return String(value || "")
    .replace(/[^\a-z0-9_ -]/gi, "-")
    .replace(/^-+|-+$/g, "")
    .slice(0, 80) || "latest";
}

```

The updater writes downloaded installers, logs, PowerShell scripts, and command launchers. Predictable filenames make cleanup and troubleshooting much easier.

How powershellSingleQuoted works inside this feature

powershellSingleQuoted safely places a JavaScript string inside a single-quoted PowerShell literal by doubling embedded single quotes.

This is the implementation at the point where the feature needs it.

```

function powershellSingleQuoted(value = "") {
  return ` '${String(value).replaceAll("'", "'')} `;
}

```

The update launcher is generated as text. This helper prevents paths containing apostrophes from breaking the script.

How downloadAndLaunchAppUpdate works inside this feature

downloadAndLaunchAppUpdate performs the automatic update handoff. It checks update availability, rejects blocked or missing installer releases, downloads the installer to a temp folder, validates that the file is large enough to be plausible, and writes it to disk.

This is the implementation at the point where the feature needs it.

```
asvnc function downloadAndLaunchAppUpdate() {
  const update = await getUpdateInfo();
  if (!update.ok) throw new Error(update.error || "Could not check for updates.");
  if (update.updateBlocked) {
    throw new Error(update.blockedReason || `Builder ${update.latestVersion} is temporarily blocked from automatic updates.`);
  }
  if (!update.updateAvailable) {
    throw new Error(`OMB Portfolio Builder ${update.currentVersion} || "" is already up to date.`);
  }
  if (!update.installerUrl) {
    throw new Error("The latest GitHub Release does not include a Windows setup installer.");
  }
  if (process.platform !== "win32") {
    throw new Error("Automatic installer updates are currently available only on Windows.");
  }

  const updateRoot = path.join(os.tmpdir(), "omb-portfolio-builder-updates");
  await mkdir(updateRoot, { recursive: true });
  const installerName = update.installerName && /\.exe$/i.test(update.installerName)
    ? update.installerName
    : `OMB-Portfolio-Builder-Setup-${safeUpdateFileSegment(update.latestVersion)}-x64.exe`;
  const installerPath = path.join(updateRoot, installerName);

  const response = await fetch(update.installerUrl, {
    headers: { "Accept": "application/octet-stream", "User-Agent": "OMB-Portfolio-Builder" }
  });
  if (!response.ok) throw new Error(`GitHub returned ${response.status} while downloading the installer.`);
  const installerBuffer = Buffer.from(await response.arrayBuffer());
  if (installerBuffer.length < 1024 * 1024) {
    throw new Error("The downloaded installer was unexpectedly small, so the update was stopped.");
  }
  await writeFile(installerPath, installerBuffer);

  const updateLogPath = path.join(updateRoot, `update-${safeUpdateFileSegment(update.latestVersion)}.log`);
  const currentExecutable = /OMB Portfolio Builder\.exe$/i.test(process.execPath || "") ? process.execPath : "";
  const currentInstallDirectory = currentExecutable ? path.dirname(currentExecutable) : "";
  const localAppDataExecutable = process.env.LOCALAPPDATA
    ? path.join(process.env.LOCALAPPDATA, "Programs", "OMB Portfolio Builder", "OMB Portfolio Builder.exe")
    : "";
  const legacyManagedApplicationExecutable = path.join(os.homedir(), "OMB", "application", "OMB Portfolio Builder", "OMB Portfolio Builder.exe");
  const legacyFlatApplicationExecutable = path.join(os.homedir(), "OMB", "application", "OMB Portfolio Builder.exe");
  const relaunchCandidates = Array.from(new Set([
    localAppDataExecutable,
    currentExecutable,
    currentInstallDirectory ? path.join(currentInstallDirectory, "OMB Portfolio Builder.exe") : "",
    legacyFlatApplicationExecutable,
    legacyManagedApplicationExecutable,
    process.env.ProgramFiles ? path.join(process.env.ProgramFiles, "OMB Portfolio Builder", "OMB Portfolio Builder.exe") : "",
    process.env["ProgramFiles(x86)"] ? path.join(process.env["ProgramFiles(x86)"], "OMB Portfolio Builder", "OMB Portfolio Builder.exe") : ""
  ]).filter(Boolean)));
  const relaunchCandidatesPs = `@($${relaunchCandidates.map(powershellSingleQuoted).join(", ")});`;
  const launcherPath = path.join(updateRoot, `run-update-${safeUpdateFileSegment(update.latestVersion)}.ps1`);
  const launcherCommandPath = path.join(updateRoot, `run-update-${safeUpdateFileSegment(update.latestVersion)}.cmd`);
  const launcherScript = [
    "$ErrorActionPreference = 'Continue'",
    "`$parentPid = ${process.pid}`",
    "`$installer = ${powershellSingleQuoted(installerPath)}",
    "`$log = ${powershellSingleQuoted(updateLogPath)}",
    "`$candidates = ${relaunchCandidatesPs}",
    "function Write-UpdateLog([string]$Message) {",
    "  trv { Add-Content -LiteralPath $log -Value ((Get-Date).ToString('s') + ' ' + $Message) } catch {}",
    "}",
    "function Wait-ForExecutable([string[]]$Paths, [int]$Seconds) {",
    "  $deadline = (Get-Date).AddSeconds($Seconds)",
    "  do {",
    "    foreach ($candidate in $Paths) {",
    "      if ($candidate -and (Test-Path -LiteralPath $candidate)) { return $candidate }",
    "    }",
    "    Start-Sleep -Seconds 1",
    "  } while ((Get-Date) -lt $deadline)",
  ]
```

```

" return $null".
"}".
"function Stop-BuilderProcesses([int]$ProcessId, [string[]]$Paths) {".
"  trv {".
"    $normalizedPaths = @($Paths | Where-Object { $ } | ForEach-Object {".
"      trv { [System.IO.Path]::GetFullPath($ ).ToLowerInvariant() } catch { $ .ToLowerInvariant() }".
"    }".
"    $processes = Get-CimInstance Win32 Process | Where-Object {".
"      $ .ProcessId -eq $ProcessId -or".
"      $ .Name -eq 'OMB Portfolio Builder.exe' -or".
"      ($ .ExecutablePath -and ($normalizedPaths -contains".
"        ([System.IO.Path]::GetFullPath($ .ExecutablePath).ToLowerInvariant()))".
"      } | Sort-Object ProcessId -Unique".
"      foreach ($process in $processes) {".
"        trv {".
"          Write-UpdateLog ('Stopping previous builder process PID ' + $process.ProcessId + ' at ' +".
"            $process.ExecutablePath)".
"          Stop-Process -Id $process.ProcessId -Force -ErrorAction SilentlyContinue".
"          } catch { Write-UpdateLog ('Could not stop PID ' + $process.ProcessId + ': ' + $ .Exception.Message)".
"        }".
"      }".
"      Start-Sleep -Seconds 2".
"    } catch { Write-UpdateLog ('Could not stop previous builder processes: ' + $ .Exception.Message) }".
"  }".
"function Run-Installer([switch]$Elevated) {".
"  $process = $null".
"  if ($Elevated) {".
"    Write-UpdateLog 'Starting installer with Windows elevation prompt.'".
"    $process = Start-Process -FilePath $installer -ArgumentList @('/S') -Verb RunAs -PassThru".
"  } else {".
"    Write-UpdateLog 'Starting installer silently.'".
"    $process = Start-Process -FilePath $installer -ArgumentList @('/S') -WindowStyle Hidden -PassThru".
"  }".
"  if (-not $process) { return 0 }".
"  if (-not $process.WaitForExit(900000)) {".
"    Write-UpdateLog ('Installer timed out after 15 minutes. PID: ' + $process.Id)".
"    trv { Stop-Process -Id $process.Id -Force -ErrorAction SilentlyContinue } catch { Write-UpdateLog ('Could".
"      not stop timed-out installer: ' + $ .Exception.Message) }".
"    return 124".
"  }".
"  return $process.ExitCode".
"}.
"Write-UpdateLog 'Update launcher started.'".
"Start-Sleep -Seconds 3".
"trv { Wait-Process -Id $parentPid -Timeout 8: Write-UpdateLog 'Previous app process exited.' } catch {".
"  Write-UpdateLog ('Previous app wait finished: ' + $ .Exception.Message) }".
"Stop-BuilderProcesses $parentPid $candidates".
"Start-Sleep -Seconds 1".
"$installerProcess = $null".
"$installerExit = 1".
"trv {".
"  Write-UpdateLog ('Starting installer: ' + $installer)".
"  $installerExit = Run-Installer".
"  Write-UpdateLog ('Installer exit code: ' + $installerExit)".
"} catch {".
"  Write-UpdateLog ('Installer launch failed: ' + $ .Exception.Message)".
"  exit 1".
"}.
"if ($installerExit -ne 0) {".
"  trv {".
"    Write-UpdateLog 'Installer did not finish cleanly: retrying with Windows elevation prompt.'".
"    $installerExit = Run-Installer -Elevated".
"    Write-UpdateLog ('Elevated installer exit code: ' + $installerExit)".
"  } catch { Write-UpdateLog ('Elevated installer launch failed: ' + $ .Exception.Message) }".
"}.
"if ($installerExit -ne 0) { exit $installerExit }".
"$target = Wait-ForExecutable $candidates 90".
"if ($target) {".
"  trv {".
"    Start-Sleep -Seconds 2".
"    Start-Process -FilePath $target -WorkingDirectory (Split-Path -Parent $target)".
"    Write-UpdateLog ('Relaunched app: ' + $target)".
"    exit 0".
"  } catch {".
"    Write-UpdateLog ('Relaunch failed: ' + $ .Exception.Message)".
"    exit 3".
"  }".
"}.
"}",

```

```

    "Write-UpdateLog 'No installed app executable was found after update.'".
    "exit 2"
  l.ioin("\r\n"):
  await writeFile(launcherPath, launcherScript, "utf8");
  const launcherCommandScript = [
    "@echo off",
    `start "" /min powershell.exe -NoProfile -ExecutionPolicy Bypass -WindowStyle Hidden -File "${launcherPath}"`
  ].join("\r\n");
  await writeFile(launcherCommandPath, launcherCommandScript, "utf8");

  const child = spawn("cmd.exe", ["/d", "/c", launcherCommandPath], {
    detached: true,
    stdio: "ignore",
    windowsHide: true
  });
  child.unref();

  setTimeout(() => {
    process.emit("omb-builder-update-started", { installerPath, launcherPath, updateLogPath });
  }, 600);
  setTimeout(() => {
    process.exit(0);
  }, 3000);

  return {
    ...update,
    installerPath,
    launcherPath,
    updateLogPath,
    launcherCommandPath,
    started: true
  };
}

```

The function then calculates relaunch candidates. It includes the per-user AppData install path, the current executable, legacy OMB application paths, and Program Files paths. That broad list is what lets the updater find the app again after installation, even as the project migrates away from older folder layouts.

The generated PowerShell script is the real bridge. It waits for the current process, stops any builder processes, runs the installer silently, retries elevated if necessary, waits for an executable to appear, and relaunches it. The Node process cannot safely replace itself, so the detached script continues after Node exits.

The final timers emit an update-started event and exit the backend. The HTTP response returns before the process closes, giving the frontend a chance to tell the user that the update has started.

Security reports as local visibility

`getSecurityReport` gives the builder a local view of security-relevant state such as release download reporting and publishing status. It is guarded by `isLocalRequest` through the API route because even read-only operational details can be sensitive.

This is not a public analytics dashboard. It is a local owner-facing report. The distinction matters: public website visitors should not be able to inspect builder internals, but the authenticated builder owner needs enough visibility to understand access and download behavior.

How `getSecurityReport` works inside this feature

`getSecurityReport` combines release download counts, authorization cache status, publish target information, and documented security controls into one local report.

This is the implementation at the point where the feature needs it.

```

  async function getSecurityReport() {
    const [downloadResult, authCacheResult, targetResult] = await Promise.allSettled([

```

```

getBuilderReleaseDownloadReport().
readPublishAuthCache().
getPublishTargetInfo()
}):
const authCache = authCacheResult.status === "fulfilled" ? authCacheResult.value : null:
const target = targetResult.status === "fulfilled" ? targetResult.value : null:
return {
  ok: true,
  generatedAt: new Date().toISOString(),
  appDownloads: downloadResult.status === "fulfilled"
    ? { ok: true, ...downloadResult.value }
    : { ok: false, error: downloadResult.reason?.message || "GitHub release download counts were not available."
      },
  publishingAuth: {
    targetRepository: target?.repository || "",
    branch: target?.branch || "",
    authenticated: Boolean(authCache && nublishAuthCacheIsFresh(authCache, {
      remote: target?.remote || "",
      repository: target?.repository || "",
      branch: target?.branch || ""
    })),
    expiresAt: nublishAuthCacheExpiresAt(authCache) || "",
    trustDays: Number(authCache?.trustDays || 0),
    successCountLastWeek: Number(authCache?.successCountLastWeek || 0)
  },
  websiteAccess: {
    available: false,
    summary: "Visitor counts and visitor IP addresses are not available from a static GitHub Pages site or
      GitHub release download counts.",
    recommendedSetup: [
      "Proxv the custom domain through Cloudflare instead of DNS-only.",
      "Enable Cloudflare Web Analytics for aggregate page-view counts.",
      "Use Cloudflare Logpush, Analytics Engine, or a Worker-backed endpoint if you need IP-level security
        logs.",
      "Publish a privacy notice before collecting raw IP addresses or persistent identifiers."
    ]
  },
  securityControls: [
    "Local builder write APIs are restricted to local requests.",
    "Publishing target details are only returned to the local machine.",
    "GitHub publishing authorization is cached for one day by default, or longer after repeated successful
      authorizations.",
    "The desktop app runs with Electron nodeIntegration disabled, contextIsolation enabled, and sandbox
      enabled.",
    "Deployable website security headers and security.txt metadata are included in the publishable site files."
  ]
}
}

```

The website access section is deliberately honest. A static GitHub Pages site does not provide raw visitor identity or IP logs by itself. The report recommends Cloudflare analytics or Worker-backed logging if the owner wants deeper visibility with a privacy notice.